

**A PROJECT REPORT ON  
CODEINGO**

*A Gamified, Secure Coding-Learning Platform*

**A PROJECT REPORT SUBMITTED TO  
THE UNIVERSITY OF MUMBAI IN PARTIAL FULFILMENT  
FOR THE AWARD OF THE DEGREE OF  
BACHELOR OF SCIENCE  
(INFORMATION TECHNOLOGY)**

**By**

**[STUDENT NAME]**

**[SEAT NO. e.g. BSIT-V-2627-XXXXX]**

**UNDER THE ESTEEMED GUIDANCE OF  
ASSISTANT PROFESSOR**

**[GUIDE NAME]**

**DEPARTMENT OF INFORMATION TECHNOLOGY  
& COMPUTER SCIENCE**

**KERALEEYA SAMAJAM (REGD.) DOMBIVLI'S  
MODEL COLLEGE  
(EMPOWERED AUTONOMOUS)**

*(Affiliated to University of Mumbai)*

**DOMBIVLI, 421201  
MAHARASHTRA  
OCTOBER 2026**

**DEPARTMENT OF INFORMATION TECHNOLOGY & COMPUTER SCIENCE**

**CERTIFICATE**

This is to certify that the project entitled, "**CODEINGO**", is bonafide work of **[STUDENT NAME]** bearing Seat No: **[SEAT NO]** submitted in FIELD PROJECT of SEM V of BACHELOR OF SCIENCE in INFORMATION TECHNOLOGY.

|                |                             |
|----------------|-----------------------------|
| Internal Guide | Head of the Dept./Principal |
|----------------|-----------------------------|

|                   |                   |
|-------------------|-------------------|
| Internal Examiner | External Examiner |
|-------------------|-------------------|

**Keraleeya Samajam (Regd.) Dombivli's**

**MODEL COLLEGE**

(Empowered Autonomous)

(Affiliated to University of Mumbai)

Re-Accredited Grade "A" by NAAC

**CERTIFICATE**

This is to certify that the following students of the B.Sc.I.T. Program studying in Semester V, have successfully completed a group project titled: "**CODEINGO**" in the area of Information Technology specialization, during the academic year 2026-2027. The students listed below have contributed to this project work and to the best of our knowledge, the work is original and all information provided is accurate and relevant.

**List of Group members:**

| Sr. No | Name          | Roll/Seat No     |
|--------|---------------|------------------|
| 1      | [MEMBER NAME] | [ROLL / SEAT NO] |
| 2      | [MEMBER NAME] | [ROLL / SEAT NO] |
| 3      | [MEMBER NAME] | [ROLL / SEAT NO] |
| 4      | [MEMBER NAME] | [ROLL / SEAT NO] |
| 5      | [MEMBER NAME] | [ROLL / SEAT NO] |

|                |                             |
|----------------|-----------------------------|
| Internal Guide | Head of the Dept./Principal |
|----------------|-----------------------------|

|                   |                   |
|-------------------|-------------------|
| Internal Examiner | External Examiner |
|-------------------|-------------------|

**Keraleeya Samajam (Regd.) Dombivli's**  
**MODEL COLLEGE**  
(Empowered Autonomous)  
(Affiliated to University of Mumbai)  
Re-Accredited Grade "A" by NAAC  
Conferred with "Best College Award" by University of Mumbai

## **CERTIFICATE**

I hereby certify that Mr./Ms. **[STUDENT NAME]**, Student of B.Sc.I.T. Program studying in V Semester, has completed a project titled **CODEINGO** in the area of Information Technology specialization/Major for the academic year 2026-2027.

To the best of my knowledge the work of the student is original and the information included in the project is correct.

|                |                             |
|----------------|-----------------------------|
| Internal Guide | Head of the Dept./Principal |
|----------------|-----------------------------|

|                   |                   |
|-------------------|-------------------|
| Internal Examiner | External Examiner |
|-------------------|-------------------|

## **GUIDE INTERACTION DECLARATION**

I, the undersigned **[STUDENT NAME]** Roll/Seat No. **[ROLL NO / SEAT NO]** studying in the V Semester of B.Sc.I.T. Program, hereby declare that I am carrying out my Field Project under the guidance of **[GUIDE NAME]** and I have interacted with my Guide on the following dates as part of my project development process:

### **Guide Interaction Record**

| <b>Sr. No.</b> | <b>Date</b>   | <b>Signature of the Internal Guide</b> |
|----------------|---------------|----------------------------------------|
| 1              | <b>[DATE]</b> |                                        |
| 2              | <b>[DATE]</b> |                                        |
| 3              | <b>[DATE]</b> |                                        |
| 4              | <b>[DATE]</b> |                                        |
| 5              | <b>[DATE]</b> |                                        |
| 6              | <b>[DATE]</b> |                                        |
| 7              | <b>[DATE]</b> |                                        |
| 8              | <b>[DATE]</b> |                                        |
| 9              | <b>[DATE]</b> |                                        |
| 10             | <b>[DATE]</b> |                                        |

I affirm that the information provided above is true to the best of my knowledge and that I have received timely academic guidance throughout the project duration.

**Signature of the Student**

### **Guide Certification**

Certified that the above interactions have been held as mentioned.

**Signature of Internal Guide**

# ABSTRACT

**CODEINGO** is a gamified web platform that teaches programming through short, interactive lessons, in the spirit of language-learning apps such as Duolingo. A learner signs in with a Google account, protects it with mandatory 2-step verification (an emailed code, an authenticator app, or a passkey), and follows a structured path of units and lessons that run from beginner to advanced level.

The platform offers ten courses - Python, JavaScript, TypeScript, Java, C++, C, SQL, HTML & CSS, Git, and Data Structures & Algorithms - with 438 lessons, 2,130 exercises and 30 hands-on projects. Learners do not only read: in every lesson they write real programs that run safely inside the browser (Python through WebAssembly, JavaScript and TypeScript in sandboxed workers, SQL on an in-browser SQLite engine, Git in a built-in simulator). Answers are graded on the server, and the expected results are never sent to the browser, so they cannot be copied.

To keep learners motivated the system adds XP, streaks, hearts, badges, a daily challenge, chapter tests with jump-ahead readiness tests, spaced-repetition practice for mistakes, public certificates, friends and weekly leagues, timed weekly contests with a live leaderboard, and optional streak reminder e-mails. The design was guided by a requirement survey of 43 young learners, which showed that 65% find game-like learning appealing, that confusing syntax and lack of guidance are the main reasons for quitting, and that building real projects is the strongest motivator.

Codeingo is built with Next.js and React on the front end and FastAPI with PostgreSQL on the back end. It is installable as a progressive web app, available in a light (white/blue) and a dark (black/green) theme, protected by a strict security design (CSRF protection, content-security policy, rate limiting, audit log, hidden admin panel) and verified by 95 automated tests and by executing every one of its 492 runnable reference solutions.

## ACKNOWLEDGEMENT

It gives me pleasure to present my Field Project on "**CODEINGO**". This is my milestone in Bachelor of Science (Information Technology). I would like to express my sincere thanks to all the teachers who helped me throughout the project. I would like to acknowledge the help and guidance provided by our guide **[GUIDE NAME]** in all places during the presentation of this project.

I am thankful to our honorable Principal **[PRINCIPAL NAME]**. Onwards my project works, I am also thankful to the staff members of the IT-CS department for their moral support. I extend my gratitude to **[COORDINATOR NAME]**, Coordinator of IT & CS Department for the support and guidance.

I also thank the 43 learners who took the time to answer the requirement survey; their honest answers shaped the features of this platform. Finally, I thank my family and friends for their constant encouragement.

**Name and Signature of Student**

**[STUDENT NAME]**

## **DECLARATION**

I hereby declare that the project entitled, "**CODEINGO**" done at Keraleeya Samajam (Regd.) Dombivli's Model College (Empowered Autonomous), has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university.

The project is done in partial fulfilment of the requirements for the award of degree of BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY) to be submitted as a Fifth Semester Field Project as part of our curriculum.

**Name and Signature of Student**

**[STUDENT NAME]**

# TABLE OF CONTENTS

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| <b>Chapter 1 Introduction.....</b>                                            | <b>1</b>  |
| 1.1 Background.....                                                           | 1         |
| 1.2 Objective.....                                                            | 1         |
| 1.3 Purpose, Scope, Applicability.....                                        | 2         |
| 1.3.1 Purpose.....                                                            | 2         |
| 1.3.2 Scope.....                                                              | 3         |
| 1.3.3 Applicability.....                                                      | 3         |
| <b>Chapter 2 Survey of Technologies.....</b>                                  | <b>6</b>  |
| 2.1 Existing Systems.....                                                     | 6         |
| 2.2 List of Technologies.....                                                 | 8         |
| 2.3 Comparative Study.....                                                    | 9         |
| 2.4 Selected Technologies.....                                                | 10        |
| <b>Chapter 3 Requirement and Analysis.....</b>                                | <b>12</b> |
| 3.1 Problem Definition.....                                                   | 12        |
| 3.2 Requirement Specification.....                                            | 12        |
| 3.2.1 Functional Requirements.....                                            | 12        |
| 3.2.2 Non-Functional Requirements.....                                        | 13        |
| 3.3 Software and Hardware Requirement.....                                    | 14        |
| 3.4 Preliminary Product Description.....                                      | 15        |
| 3.5 Conceptual Model.....                                                     | 16        |
| 3.6 Requirement Gathering Survey.....                                         | 17        |
| 3.6.1 Method.....                                                             | 17        |
| 3.6.2 Results.....                                                            | 18        |
| 3.6.3 From Findings to Features.....                                          | 23        |
| <b>Chapter 4 System Design.....</b>                                           | <b>25</b> |
| 4.1 Basic Modules.....                                                        | 25        |
| 4.2 Data Design.....                                                          | 26        |
| 4.3 Logic Design.....                                                         | 35        |
| 4.3.1 Sign-in and 2-step verification flow.....                               | 36        |
| 4.3.2 Answer grading flow.....                                                | 36        |
| 4.3.3 Security design.....                                                    | 37        |
| 4.4 User Interface Design.....                                                | 37        |
| 4.5 Work Break-Down Structure.....                                            | 38        |
| 5.1 Coding.....                                                               | 41        |
| 5.1.0 Project structure.....                                                  | 41        |
| 5.1.1 Application factory - backend/app/main.py.....                          | 41        |
| 5.1.2 E-mail 2-step code: issue and verify - backend/app/security/mfa.py..... | 42        |
| 5.1.3 Server-side grading - backend/app/services/grading.py.....              | 43        |
| 5.1.4 Spaced repetition (Leitner boxes) - backend/app/services/review.py..... | 43        |
| 5.1.5 Passkey sign-in verification - backend/app/security/passkeys.py.....    | 44        |

|                       |                                                                                          |           |
|-----------------------|------------------------------------------------------------------------------------------|-----------|
| 5.1.6                 | Contest scoring and ranking - backend/app/services/contests.py.....                      | 44        |
| 5.1.7                 | TypeScript type-checking in the browser - frontend/public/runners/ts-<br>shared.mjs..... | 45        |
| 5.1.8                 | Per-request Content-Security-Policy - frontend/proxy.js.....                             | 46        |
| 5.2                   | Testing.....                                                                             | 47        |
| 5.2.1                 | Test Cases.....                                                                          | 47        |
| 5.2.2                 | Curriculum validation.....                                                               | 52        |
| 5.2.3                 | Observations from browser testing.....                                                   | 52        |
| 5.2.4                 | Defects found and fixed during testing.....                                              | 53        |
| <b>Chapter 6</b>      | <b>Results and Discussion.....</b>                                                       | <b>55</b> |
| 6.1                   | Screenshots of the system.....                                                           | 55        |
| 6.1.1                 | User sign-in and entry.....                                                              | 55        |
| 6.1.2                 | Learning.....                                                                            | 57        |
| 6.1.3                 | Tests, practice and daily challenge.....                                                 | 62        |
| 6.1.4                 | Social features and contests.....                                                        | 65        |
| 6.1.5                 | Progress, profile and certificate.....                                                   | 67        |
| 6.1.6                 | Administration.....                                                                      | 70        |
| 6.1.7                 | Mobile view.....                                                                         | 72        |
| 6.2                   | Discussion of results.....                                                               | 73        |
| <b>Chapter 7</b>      | <b>Conclusion and Future Work.....</b>                                                   | <b>75</b> |
| 7.1                   | Conclusion.....                                                                          | 75        |
| 7.2                   | Limitations.....                                                                         | 75        |
| 7.3                   | Future Scope.....                                                                        | 75        |
| 7.4                   | Bibliography.....                                                                        | 75        |
| <b>Annexures.....</b> |                                                                                          | <b>78</b> |
| A                     | Survey questionnaire.....                                                                | 78        |
| B                     | REST API endpoints.....                                                                  | 78        |
| C                     | Security controls.....                                                                   | 80        |
| D                     | Installation and user guide.....                                                         | 81        |

## LIST OF TABLES

|              |                                                                 |    |
|--------------|-----------------------------------------------------------------|----|
| Table 2.1.1  | Summary of existing systems compared with Codeingo.....         | 7  |
| Table 2.3.1  | Comparative study of the technology choices.....                | 9  |
| Table 3.3.1  | Software requirements.....                                      | 14 |
| Table 3.3.2  | Hardware requirements.....                                      | 15 |
| Table 3.6.1  | Summary of the survey results (n = 43).....                     | 17 |
| Table 3.6.2  | Survey findings and the features that answer them.....          | 23 |
| Table 4.2.1  | users - Account, security settings, gamification state.....     | 27 |
| Table 4.2.2  | courses - The ten courses.....                                  | 28 |
| Table 4.2.3  | units - Units (chapters) of a course.....                       | 28 |
| Table 4.2.4  | lessons - Lessons and projects.....                             | 29 |
| Table 4.2.5  | exercises - Questions and programs.....                         | 29 |
| Table 4.2.6  | user_lessons - A learner's lesson progress.....                 | 30 |
| Table 4.2.7  | lesson_attempts - One play-through of a lesson.....             | 30 |
| Table 4.2.8  | user_tests - Best result per test.....                          | 30 |
| Table 4.2.9  | test_attempts - One sitting of a chapter or readiness test..... | 31 |
| Table 4.2.10 | review_items - Spaced-repetition list.....                      | 31 |
| Table 4.2.11 | practice_attempts - Practice sessions.....                      | 31 |
| Table 4.2.12 | daily_claims - Daily challenge answers.....                     | 32 |
| Table 4.2.13 | certificates - Issued certificates.....                         | 32 |
| Table 4.2.14 | xp_events - XP history.....                                     | 32 |
| Table 4.2.15 | user_badges - Earned badges.....                                | 33 |
| Table 4.2.16 | follows - Friend relations.....                                 | 33 |
| Table 4.2.17 | contests - Weekly contests.....                                 | 33 |
| Table 4.2.18 | contest_entries - A learner's single timed run.....             | 33 |
| Table 4.2.19 | passkeys - WebAuthn credentials.....                            | 34 |
| Table 4.2.20 | webauthn_challenges - One-time passkey challenges.....          | 34 |
| Table 4.2.21 | recovery_codes - Authenticator recovery codes.....              | 34 |
| Table 4.2.22 | refresh_tokens - Rotating refresh tokens.....                   | 35 |
| Table 4.2.23 | audit_log - Security audit log.....                             | 35 |
| Table 4.4.1  | Main colour tokens of the two themes.....                       | 38 |
| Table 5.1.1  | Size of the code base (lines).....                              | 41 |
| Table 5.2.1  | Automated back-end tests by file.....                           | 47 |
| Table 5.2.2  | Test cases.....                                                 | 48 |
| Table 5.2.3  | Curriculum size verified by execution.....                      | 52 |
| Table 5.2.4  | Notable defects found by testing and how they were fixed.....   | 53 |
| Table 6.2.1  | Survey finding and how the finished system answers it.....      | 73 |
| Table B.1    | REST API endpoints.....                                         | 78 |

## LIST OF FIGURES

|              |                                                                         |    |
|--------------|-------------------------------------------------------------------------|----|
| Figure 3.5   | Conceptual model of Codeingo.....                                       | 17 |
| Figure 3.6.1 | Age group of the respondents.....                                       | 18 |
| Figure 3.6.2 | Earlier experience with programming.....                                | 19 |
| Figure 3.6.3 | Reasons why learning was difficult or boring.....                       | 19 |
| Figure 3.6.4 | Preferred way of learning new skills.....                               | 20 |
| Figure 3.6.5 | How appealing is learning to code through a game?.....                  | 20 |
| Figure 3.6.6 | Language the respondents want to learn first.....                       | 21 |
| Figure 3.6.7 | What would keep respondents learning.....                               | 21 |
| Figure 3.6.8 | Learning alone or with a community.....                                 | 22 |
| Figure 3.6.9 | Biggest reason to stop using a learning platform.....                   | 22 |
| Figure 4.2   | Entity-Relationship diagram (generated from the SQLAlchemy models)..... | 27 |
| Figure 4.3   | System architecture (logic diagram).....                                | 36 |
| Figure 4.4   | Screen map of Codeingo (user interface diagram).....                    | 37 |
| Figure 4.5   | Work break-down structure of Codeingo.....                              | 39 |
| Figure 6.1   | Landing page (light theme).....                                         | 55 |
| Figure 6.2   | Landing page (dark theme).....                                          | 56 |
| Figure 6.3   | 2-step verification by e-mailed code.....                               | 56 |
| Figure 6.4   | Course list (light).....                                                | 57 |
| Figure 6.5   | Learning path.....                                                      | 57 |
| Figure 6.6   | Learning path (dark).....                                               | 58 |
| Figure 6.7   | Lesson introduction.....                                                | 58 |
| Figure 6.8   | Lesson question.....                                                    | 59 |
| Figure 6.9   | Wrong answer feedback.....                                              | 59 |
| Figure 6.10  | Running code in the browser.....                                        | 60 |
| Figure 6.11  | Lesson complete.....                                                    | 60 |
| Figure 6.12  | TypeScript type error.....                                              | 61 |
| Figure 6.13  | Git terminal.....                                                       | 61 |
| Figure 6.14  | Chapter test introduction.....                                          | 62 |
| Figure 6.15  | Chapter test question.....                                              | 62 |
| Figure 6.16  | Chapter test result.....                                                | 63 |
| Figure 6.17  | Practice hub.....                                                       | 63 |
| Figure 6.18  | Practice question.....                                                  | 64 |
| Figure 6.19  | Daily challenge.....                                                    | 64 |
| Figure 6.20  | Weekly league (everyone).....                                           | 65 |
| Figure 6.21  | Weekly league (friends).....                                            | 65 |
| Figure 6.22  | Contests list.....                                                      | 66 |
| Figure 6.23  | Contest leaderboard.....                                                | 66 |
| Figure 6.24  | Contest in progress.....                                                | 67 |
| Figure 6.25  | Progress (light).....                                                   | 67 |
| Figure 6.26  | Progress (dark).....                                                    | 68 |

|             |                                      |    |
|-------------|--------------------------------------|----|
| Figure 6.27 | Profile and security.....            | 68 |
| Figure 6.28 | Certificate.....                     | 69 |
| Figure 6.29 | Your data (export and delete).....   | 69 |
| Figure 6.30 | Admin: users and roles.....          | 70 |
| Figure 6.31 | Admin: audit log.....                | 70 |
| Figure 6.32 | Admin area hidden from learners..... | 71 |
| Figure 6.33 | Database tables.....                 | 71 |
| Figure 6.34 | Phone: learning path.....            | 72 |
| Figure 6.35 | Phone: leagues.....                  | 72 |
| Figure 6.36 | Phone: dark theme.....               | 73 |

# CHAPTER 1

## INTRODUCTION

### 1.1 Background

Software now sits underneath almost every industry, and the ability to write code has become a basic skill for students, not only for those who plan a career in computing. Yet learning to program is hard to start and even harder to continue. Beginners meet an unfamiliar syntax, tools that must be installed and configured before the first line can run, and long video courses that never check whether anything was understood. Many of them give up in the first weeks.

The survey conducted for this project (Section 3.6, 43 respondents aged 15 to 22) confirms this picture: 28% of the learners had already tried programming and given up, 33% of those who answered said that confusing syntax made learning difficult or boring, and a further 28% said that they would stop using a learning platform because of a lack of guidance. At the same time 65% rated the idea of learning to code through a game-like experience as appealing (4 or 5 out of 5), and the learners named building real projects (40%) and competing with friends (23%) as the things that would keep them going.

Language-learning applications such as Duolingo showed that short daily lessons, immediate feedback, streaks, points and friendly competition can turn a difficult subject into a habit. Recent advances in web technology make the same approach possible for programming: WebAssembly lets a real Python interpreter or an SQL database run safely inside the learner's own browser, and modern web frameworks make it possible to build fast, installable applications that work on a phone as well as on a desktop.

**CODEINGO** ("Duolingo for coding") applies these ideas. It is a web platform in which a learner follows a guided path from beginner to advanced level, answers quick-check questions, writes and runs real programs in every lesson, builds a project at the end of each section, proves progress with chapter tests, and earns a public certificate when a course is complete. Security is treated as a core feature, not an add-on: every account is protected by 2-step verification, answers are graded on the server so they cannot be copied from the page, and the administrator panel is hidden from everyone who is not an administrator.

### 1.2 Objective

The primary objective of **CODEINGO** is to develop a secure, gamified web platform that teaches programming from beginner to advanced level through short interactive lessons, real code execution in the browser, instant feedback and social motivation, so that learners stay engaged and can show what they have learned.

#### Specific Objectives

1. To provide a structured learning path of courses, sections, units and lessons, unlocked in order, covering ten subjects: Python, JavaScript, TypeScript, Java, C++, C, SQL, HTML & CSS, Git and Data Structures & Algorithms.

2. To let learners write and run real code inside the browser (Python, JavaScript, TypeScript, SQL, HTML/CSS and Git) without installing anything, using sandboxed runtimes with time limits and no network access.
3. To grade every answer on the server and never send the expected output to the browser, and to block copy-and-paste into answer boxes, so that learners must understand the solution.
4. To motivate learners with XP, daily goals, streaks, hearts, badges and a daily challenge, and to offer friendly competition through friends, weekly leagues and timed weekly contests with a live leaderboard.
5. To check understanding with chapter tests, and to let confident learners skip ahead with 15-question readiness tests at the end of each section.
6. To help memory with a spaced-repetition practice mode that brings missed questions back at growing intervals.
7. To end every section with a multi-step project in which each step continues from the learner's own code, and to issue a public, verifiable, printable certificate for every completed course.
8. To secure access with Google sign-in plus mandatory 2-step verification (emailed code, authenticator app or passkey), CSRF protection, rate limiting, a strict content-security policy, hashed and encrypted secrets and a complete audit log.
9. To provide a hidden administration panel for managing lessons and users, in which only an administrator can grant or remove administrator access, and only after re-entering a 2-step code.
10. To respect privacy: collect only the data the platform needs, let every user download all of their data and delete their account permanently, and publish a clear Privacy Policy and Terms of Service.
11. To offer a modern, accessible and responsive interface in a light theme (white, blue, black) and a dark theme (black, green), usable from a phone to a 4K monitor, and installable as an app with an offline page.
12. To send optional streak-reminder e-mails with a one-click unsubscribe, at most once per day.
13. To make the platform easy to deploy (Docker and a free Vercel + Render + Neon setup), and to verify the quality of the content by automatically executing every reference solution of every exercise.
14. To base the feature set on evidence by running a requirement survey among prospective learners before and during design.

## 1.3 Purpose, Scope, Applicability

### 1.3.1 Purpose

**CODEINGO** exists to make learning to code approachable, habit-forming and trustworthy. It lowers the barrier to the first line of code (nothing to install), keeps learners returning through game mechanics that were chosen from survey evidence, gives them real problems and projects instead of passive reading, and protects their accounts and data with professional-grade

security. For teachers and mentors it provides a ready, verified curriculum and an administration panel to maintain it.

### 1.3.2 Scope

The scope of the project covers the design, development, testing and documentation of a web-based learning platform with the following functionality:

- Registration and sign-in with Google, mandatory 2-step verification, passkeys, recovery codes and session management.
- Ten courses with 438 lessons, 2,130 exercises of five types (multiple choice, fill-in-the-blank, arrange-the-code, write-a-snippet, write-and-run a program) and 30 projects.
- In-browser code runners for Python, JavaScript, TypeScript, SQL, HTML/CSS and Git; an optional sandboxed server runner for Java, C and C++.
- Server-side grading, XP, streaks, hearts, badges, daily challenge, daily goal.
- Chapter tests, readiness (jump-ahead) tests, spaced-repetition practice, section projects and public certificates.
- Friends, friend codes and invite links, global and friends leagues, weekly contests with a live leaderboard.
- Streak reminder e-mails with unsubscribe; installable progressive web app with offline page.
- Admin panel with content editing, user management, role management and a security audit log.
- Privacy features: data export, account deletion, Privacy Policy and Terms of Service.
- Light and dark themes, responsive layout, accessibility features.

The following are outside the scope of this version: native mobile applications, live video lessons, a discussion forum, payments, and an AI model that generates new lessons. An optional AI tutor ("Codi") that gives hints is supported if the administrator supplies an API key, but the platform works fully without it.

### 1.3.3 Applicability

The platform can be used by:

- School and college students (the survey respondents were aged 15 to 22) who want to start programming without installing tools.
- Self-learners and career-switchers who need a guided path from beginner to advanced level and a certificate to show.
- Teachers and training institutes that want a ready curriculum with tests, projects and a leaderboard to run a class or a coding club.
- Colleges for coding contests: the weekly timed contest with a live leaderboard can be used for inter-class competitions.
- Developers preparing for interviews (data structures and algorithms, SQL, Git, TypeScript).

- Students of web development who need practice in HTML & CSS with an instant live preview.

Because the lessons are stored in the database and edited from the administration panel, the same platform can host additional courses, or the curriculum of a single institution, without changes to the code.

# GUIDE INTERACTION REPORT

*(To be maintained by the student and submitted as part of the Field Project documentation)*

|                                |                        |
|--------------------------------|------------------------|
| Name of the Student            | [STUDENT NAME]         |
| Program /Semester              | B.Sc I.T./ Semester V  |
| Roll No/Seat No                | [ROLL NO / SEAT NO]    |
| Major Subject / Specialization | Information Technology |
| Field project Title            | CODEINGO               |
| Name of the Faculty mentor     | [GUIDE NAME]           |

## Overview (Max 150 Words)

*(Brief summary of key activities, tasks and projects undertaken during the period)*

## CHAPTER 1: INTRODUCTION

This chapter introduces why a gamified coding platform is needed. Many beginners quit because of confusing syntax, installation problems and lack of guidance, while game mechanics such as streaks, points and friendly competition keep people learning. It describes Codeingo as "Duolingo for coding", lists fourteen objectives (learning path, in-browser code running, server-side grading, gamification, tests, spaced repetition, projects and certificates, strong security, admin panel, privacy, themes, reminders, deployment, survey-based design), and defines purpose, scope and applicability.

## Learning Outcomes (Max 100 words):

We learned to identify a real problem for beginner programmers and to turn it into a clear objective list. We understood the difference between purpose, scope and applicability, and why a scope must also say what is not included. We learned that a project should be based on evidence, so we ran a survey of 43 learners instead of guessing which features are important. We also learned that security and privacy are objectives of the project, not extras.

## Challenges Faced (Max 100 words):

Our first idea was very wide: many languages, many game features and many social features. It was difficult to decide what belongs to this version and what must wait. Writing objectives that can be tested, and not only wishes, took several attempts. We also had to decide how a free project can still be trusted with real accounts, which is why 2-step verification became mandatory early.

## Mentor Feedback and Suggestions (Max 100 words):

[To be filled in by the guide]

Signature of the Internal Guide

# CHAPTER 2

## SURVEY OF TECHNOLOGIES

### 2.1 Existing Systems

Many websites and apps help people learn programming. Before designing Codeingo we studied the kinds of systems that the survey respondents said they already use (Codecademy, freeCodeCamp, Scratch, LeetCode, YouTube channels, and local compilers such as Turbo C). A summary of the existing systems is given below.

#### 1. Interactive course platforms (for example Codecademy)

##### Features:

- Lessons with an in-browser editor and automatic checking.
- Structured courses and career paths.
- Projects and quizzes.

##### Limitations:

- Many projects, certificates and advanced content require a paid subscription.
- Little social or competitive motivation; progress is mostly private.
- The path is largely text-based and can feel dry to a beginner.

#### 2. Free curriculum sites (for example freeCodeCamp)

##### Features:

- Free and very large curriculum with projects and certifications.
- Runs in the browser without installation.

##### Limitations:

- Minimal game elements, so motivation depends on the learner.
- Beginners often report too little guidance when a task is hard.
- Mostly web development; few languages such as C, C++ or Java.

#### 3. Visual block-based tools (for example Scratch)

##### Features:

- Drag-and-drop blocks, immediate visual results.
- Excellent for young children and for learning logic.

##### Limitations:

- Does not teach real text syntax, so the next step to Python or Java is a big jump.
- Not suitable for learning databases, version control or algorithms in depth.

#### 4. Practice and contest sites (for example LeetCode, HackerRank)

##### Features:

- Thousands of problems, contests and interview preparation.
- Strong community and rankings.

**Limitations:**

- Assume that the learner already knows the language; unsuitable for absolute beginners.
- Little teaching content; feedback is a pass/fail verdict.

**5. Video tutorials and local tools (for example YouTube channels, Turbo C, editors)****Features:**

- Free, large amount of explanation; 30% of our respondents prefer learning by watching videos.
- Local compilers show how real tools work.

**Limitations:**

- Passive: nobody checks whether the learner understood.
- Installation and setup are a barrier; old tools may be outdated.
- No progress tracking, no reminders, no motivation system.

**Table 2.1.1: Summary of existing systems compared with Codeingo**

| Feature                                     | Course platforms | Free curriculum | Block tools | Contest sites | Codeingo |
|---------------------------------------------|------------------|-----------------|-------------|---------------|----------|
| Real code runs in the browser, no install   | Yes              | Yes             | Blocks only | Yes           | Yes      |
| Beginner to advanced path                   | Yes              | Partly          | No          | No            | Yes      |
| Game mechanics (XP, streak, hearts, badges) | Partly           | No              | Partly      | Rankings      | Yes      |
| Tests and jump-ahead tests                  | Partly           | No              | No          | No            | Yes      |
| Spaced-repetition practice of mistakes      | No               | No              | No          | No            | Yes      |
| Friends, leagues and live contests          | No               | No              | No          | Yes           | Yes      |
| Projects that build on your own code        | Paid             | Yes             | Yes         | No            | Yes      |
| Verifiable public certificates              | Paid             | Yes             | No          | No            | Yes      |
| Mandatory 2-step verification / passkeys    | Optional         | Optional        | No          | Optional      | Yes      |
| Data download and account deletion in-app   | Varies           | Varies          | Varies      | Varies        | Yes      |

*The comparison is a general summary of the categories above, not a review of any specific product; individual products change over time.*

The study shows a gap: no single free system combines guided beginner-to-advanced learning, real code execution without installation, game mechanics, tests, practice, social competition and strong account security. Codeingo is designed to fill this gap.

## 2.2 List of Technologies

### 1. Frontend Technologies

- **Next.js 16 (App Router) and React 19:** Used to build the user interface as pages and reusable components; server rendering for fast first load; the Next.js proxy file creates a fresh Content-Security-Policy nonce for every request.
- **JavaScript (ES2022):** Used for all client-side behaviour, including the lesson player, the answer checking flow and the code-runner workers.
- **Recharts:** Draws the XP-over-time and lessons-by-language charts on the Progress page, with a table view for accessibility.
- **Motion (animation library):** Smooth, compositor-only animations (transform and opacity) for the feedback sheets, path highlights and confetti.

### 2. Styling Technologies

- **Tailwind CSS 3 and CSS variables:** Utility-first styling; colours are CSS variables, so the light theme (white, blue, black) and the dark theme (black, green) swap by changing one class. Accent tints re-colour whole sections.
- **Nunito (self-hosted by Next.js):** A rounded, friendly typeface loaded at build time, so no request is made to a font server at run time.

### 3. Backend Technologies

- **Python 3.11+:** The language of the server and of the curriculum definitions.
- **FastAPI and Uvicorn:** Web framework and server for the REST API; automatic request validation with Pydantic; OpenAPI description.
- **SQLAlchemy 2:** Object-relational mapper; the same models run on PostgreSQL (production) and SQLite (development and tests).
- **PyJWT, cryptography (Fernet), pyotp, qrcode, webauthn:** Access tokens, encryption of authenticator secrets, time-based one-time passwords (RFC 6238), QR codes for authenticator set-up, and passkeys (WebAuthn).

### 4. Database and Cache

- **PostgreSQL 16:** Primary database in production (SQLite is used in development).
- **Redis:** Shared, fixed-window rate limiting across several server processes.

### 5. In-browser Code Runtimes

- **Pyodide (CPython compiled to WebAssembly):** Runs learner Python programs inside a Web Worker.
- **sql.js (SQLite compiled to WebAssembly):** Runs learner SQL on an in-browser database.
- **TypeScript 6 compiler (JavaScript build):** Type-checks learner TypeScript in strict mode in the browser, then the compiled code runs like JavaScript.
- **Custom Git simulator:** A deterministic in-memory implementation of the common Git commands used by the Git course.
- **Piston (optional):** A self-hosted sandbox for compiling and running Java, C and C++ on the server side, if configured.

## 6. Communication Technologies

- **HTTP / REST APIs (JSON):** Communication between the browser and the FastAPI server; the browser always talks to one origin.
- **SMTP over TLS:** Sends sign-in codes and reminders (for example through a Gmail App Password).
- **Google OpenID Connect with PKCE:** Secure sign-in with a Google account.

## 7. Testing, DevOps and Development Tools

- **pytest and Playwright:** 95 automated back-end tests; a browser-test suite and screenshot automation.
- **Docker, nginx, GitHub Actions:** Container images, reverse proxy and continuous integration.
- **Vercel, Render, Neon:** Hosting for the web site, the API with Redis, and the PostgreSQL database in the free deployment option.
- **Visual Studio Code, Git and GitHub, npm, pip:** Development environment, version control and package management.

## 2.3 Comparative Study

**Table 2.3.1: Comparative study of the technology choices**

| Technology                  | Features                                                      | Advantages                                                                            | Disadvantages                                                        |
|-----------------------------|---------------------------------------------------------------|---------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Next.js (React)             | Server rendering, routing, build tooling, per-request headers | Fast first load, one framework for pages and proxying, large ecosystem                | Frequent major changes; larger learning curve than plain React       |
| Plain React single-page app | Client-side rendering only                                    | Simple hosting                                                                        | Slower first load; no server-side nonce for a strict CSP             |
| FastAPI (Python)            | Async REST framework, typed validation, auto docs             | Fast, concise, the same language as the curriculum tools, strong typing with Pydantic | Younger ecosystem than Django                                        |
| Django / Flask              | Full-stack framework / micro framework                        | Mature, many plugins                                                                  | Django is heavier for an API-only service; Flask needs many add-ons  |
| Node.js + Express           | JavaScript on the server                                      | One language on both sides                                                            | Less built-in validation; weaker typing                              |
| PostgreSQL                  | Relational, transactions, cascade rules, JSON columns         | Strong integrity, ideal for progress and relations, free hosting available            | Needs a server; migrations need care                                 |
| MongoDB                     | Document store                                                | Flexible schema                                                                       | Weaker joins for leaderboards and progress queries; manual integrity |
| SQLite                      | Embedded file database                                        | Zero setup for development and tests                                                  | Not for several server processes in production                       |

| Technology                                              | Features                                        | Advantages                                                | Disadvantages                                          |
|---------------------------------------------------------|-------------------------------------------------|-----------------------------------------------------------|--------------------------------------------------------|
| Pyodide / WebAssembly in the browser                    | Runs real interpreters on the learner's machine | No server cost, instant results, safe sandbox, no install | First load of the runtime is large (cached afterwards) |
| Server-side code containers                             | Run learner code on the server                  | Works for compiled languages                              | Costly, harder to secure, slower, needs Docker hosts   |
| Tailwind CSS                                            | Utility classes with design tokens              | Consistent design, small CSS, easy theming                | Long class lists in markup                             |
| Cookie sessions with short JWT + rotating refresh token | HttpOnly cookies, CSRF token                    | Not readable by scripts, revocable, reuse detection       | Needs CSRF protection                                  |
| Redis rate limiter                                      | Shared counters with expiry                     | Works with many processes                                 | An extra service to run                                |

## 2.4 Selected Technologies

Based on the requirements and the comparison above, the following technologies were selected for Codeingo:

1. **Next.js 16 with React 19 and Tailwind CSS** for the front end. The framework gives fast, server-rendered pages, a place to create a fresh Content-Security-Policy nonce for each request, and a single origin when the API is proxied. Tailwind and CSS variables make the two required colour themes simple and consistent.
2. **FastAPI with SQLAlchemy 2 and Pydantic** for the back end. It produces a typed, validated REST API quickly, and the same Python code base also contains the curriculum and the validation scripts.
3. **PostgreSQL (production) and SQLite (development, tests)** for data. The relational model fits users, progress, tests and leaderboards, and cascading deletes make account deletion reliable. The same test suite passes on both databases.
4. **Redis** for shared rate limiting, so abuse protection works with several server processes.
5. **Pyodide, sql.js, the TypeScript compiler and a Git simulator inside sandboxed Web Workers** for running learner code. Code stays on the learner's device, costs the server nothing, cannot reach the network (worker-level CSP and removal of network APIs) and is stopped by a time limit if it loops forever.
6. **Google OIDC + e-mail codes + TOTP + WebAuthn passkeys** for authentication, giving every learner a mandatory second factor without forcing them to own a special app.
7. **pytest, Playwright, GitHub Actions** for quality: automated tests, browser tests and a pipeline that also executes every reference solution of every exercise.
8. **Docker, nginx, Vercel, Render and Neon** for deployment, from a single-server container setup to a free three-service cloud setup.

Together these technologies give a clear separation between the user interface (Next.js), the server logic and security (FastAPI), the data (PostgreSQL and Redis) and the code-execution layer (browser runtimes).

# GUIDE INTERACTION REPORT

*(To be maintained by the student and submitted as part of the Field Project documentation)*

|                                       |                        |
|---------------------------------------|------------------------|
| <b>Name of the Student</b>            | [STUDENT NAME]         |
| <b>Program /Semester</b>              | B.Sc I.T./ Semester V  |
| <b>Roll No/Seat No</b>                | [ROLL NO / SEAT NO]    |
| <b>Major Subject / Specialization</b> | Information Technology |
| <b>Field project Title</b>            | CODEINGO               |
| <b>Name of the Faculty mentor</b>     | [GUIDE NAME]           |

## Overview (Max 150 Words)

*(Brief summary of key activities, tasks and projects undertaken during the period)*

## CHAPTER 2: SURVEY OF TECHNOLOGIES

As part of Chapter 2 we studied existing learning systems (course platforms, free curriculum sites, block tools, contest sites, videos) and listed their features and limitations. We listed the technologies we could use for the front end, back end, database, code execution, authentication, testing and deployment, compared them in a table, and selected Next.js, React, Tailwind, FastAPI, SQLAlchemy, PostgreSQL, Redis, Pyodide/WebAssembly runtimes, Google sign-in with 2-step verification and passkeys.

### Learning Outcomes (Max 100 words):

We learned how to do a proper survey of existing systems and to find their gaps. We learned to compare technologies in a table by features, advantages and disadvantages, and to choose by project needs and not by fashion. We understood that running learner code on the learner's own device in WebAssembly is cheaper and safer than running it on the server, and we learned how a strict content-security policy works with a per-request nonce.

### Challenges Faced (Max 100 words):

The technologies changed faster than the tutorials: Next.js 16 behaves differently from older versions, so we had to read its own documentation. On a Windows computer with application-control rules the default native compiler was blocked, and we had to switch to the webpack build. Choosing between running code on a server and in the browser needed experiments, because compiled languages such as Java and C cannot run in the browser easily.

### Mentor Feedback and Suggestions (Max 100 words):

[To be filled in by the guide]

Signature of the Internal Guide

# CHAPTER 3

## REQUIREMENT AND ANALYSIS

### 3.1 Problem Definition

Beginners who want to learn programming depend on a mixture of video tutorials, text websites and local tools. Installing a compiler or an editor is the first obstacle; reading or watching without being checked is the second; and without a plan, feedback or company, motivation fades within weeks. Existing interactive platforms are either partly paid, lightly gamified, or built for people who already know a language. Platforms that do offer accounts rarely explain what data they keep, and many treat security as optional.

Our own survey (Section 3.6) turned this problem into numbers: 28% of the respondents had tried and given up, the largest single reason for difficulty was confusing syntax (33%), the largest reason to leave was not enough guidance (28%), 65% found game-like learning appealing, and only 23% wanted to learn completely alone.

**CODEINGO** is proposed to solve this problem by combining a guided path, real code execution in the browser, instant server-side grading, game mechanics, tests, spaced-repetition practice, projects, certificates and friendly competition in one secure web application. The system reduces the effort of starting (nothing to install), gives clear feedback after every answer, and keeps the learner's account and data protected.

The main purpose of the proposed system is to make learning to code more engaging, more guided and more trustworthy than a collection of videos and text pages, and to give teachers and institutions a ready, verified curriculum that they can extend.

### 3.2 Requirement Specification

The requirements of Codeingo are divided into two categories:

#### 3.2.1 Functional Requirements

Functional requirements describe the operations and features that the system should provide to learners and administrators.

##### 1. User Authentication & Account Security

- The system shall allow sign-in with a Google account and, for a returning user, with a passkey.
- The system shall require 2-step verification for every account: a 6-digit code sent by e-mail (default), an authenticator-app code, or a passkey.
- The system shall lock further attempts after five wrong codes, throttle code requests, and provide hashed single-use recovery codes for authenticator users.
- The system shall let a user sign out of the current device or of all devices.

## **2. Learning Path & Lessons**

- The system shall present courses made of sections (Beginner, Intermediate, Advanced), units and lessons, unlocked in order.
- Each lesson shall contain a short concept explanation and exercises of five types: multiple choice, fill-in-the-blank, arrange-the-code, write-a-snippet and write-and-run a program.
- The system shall run learner programs in the browser for Python, JavaScript, TypeScript, SQL, HTML/CSS and Git, and shall grade them on the server by comparing the output with expected results that are never sent to the browser.
- The system shall show an explanation and the correct answer only after the learner has answered, and shall prevent pasting into answer boxes.

## **3. Assessment & Memory**

- The system shall give a chapter test (8 questions, 6 to pass) at the end of each unit and a readiness test (15 questions, 12 to pass) that lets the learner jump ahead.
- The system shall store every missed question in a practice list and bring it back at growing intervals (1, 3, 7, 16, 35 days).
- Each section shall end with a three-step project in which each step starts from the learner's own code.
- The system shall issue a public, verifiable, printable certificate when all lessons and chapter tests of a course are complete.

## **4. Motivation & Community**

- The system shall award XP, maintain daily goals and streaks, limit mistakes with hearts, award badges, and offer a daily challenge.
- The system shall let users follow friends by friend code or invite link and show a global league and a friends league each week.
- The system shall run a weekly timed contest for every course (same 10 questions for all, one 10-minute attempt) with a live leaderboard.
- The system shall send an optional streak reminder e-mail at most once per day, with a signed one-click unsubscribe link.

## **5. Administration & Privacy**

- The system shall provide an administration panel, visible only to administrators, to edit lessons, manage users and read the security audit log.
- Only an administrator shall be able to grant or remove administrator access, after re-entering a 2-step code; owner accounts shall always remain administrators.
- The system shall let a user download all of their data and delete their account permanently after confirming with a 2-step code.
- The system shall publish a Privacy Policy and Terms of Service.

### **3.2.2 Non-Functional Requirements**

Non-functional requirements describe the quality, performance, security and usability expected from the system.

## 1. Security

- Sessions must use HttpOnly, Secure, SameSite=Strict cookies with CSRF protection; secrets must be hashed or encrypted; every sensitive action must be rate-limited and audit-logged.
- Learner code must never run on the main page thread and must have no network access.

## 2. Performance

- Pages should load quickly and animations should use only compositor properties (transform and opacity) to stay smooth.
- Running a typical learner program in the browser should return in well under a second once the runtime is cached.

## 3. Usability

- The interface must be clean, friendly and usable from a phone to a 4K monitor, in a light theme (white, blue, black) and a dark theme (black, green).
- Feedback after every answer must be immediate and written in plain language.

## 4. Reliability

- Invalid input and unauthorized access must be handled gracefully with clear messages; failures in e-mail sending must not corrupt state.
- The system must stay consistent when several requests arrive at once (for example, the same answer submitted twice).

## 5. Maintainability

- The code must be modular (routers, services, models, views, components) and covered by automated tests; all curriculum content must be verifiable by script.

## 6. Scalability

- New courses, units and exercise types must be addable without code changes; the API must work with several server processes (shared rate limiting).

## 7. Accessibility & Compatibility

- Interactive elements must be keyboard-operable with visible focus, charts must have a table alternative, motion must respect the reduced-motion setting, and the application must run on current Chrome, Edge, Firefox and Safari.

## 8. Privacy & Compliance

- Collect only necessary data, never show e-mail addresses to other users, offer export and deletion, and keep no advertising or tracking cookies.

## 3.3 Software and Hardware Requirement

### Software Requirements

**Table 3.3.1: Software requirements**

| Software / Technology | Requirement / Purpose                              |
|-----------------------|----------------------------------------------------|
| Next.js 16, React 19  | User interface, pages, routing, security headers.  |
| Tailwind CSS 3        | Styling, responsive layout, light and dark themes. |

| Software / Technology                  | Requirement / Purpose                                                               |
|----------------------------------------|-------------------------------------------------------------------------------------|
| Recharts, Motion                       | Charts on the Progress page; animations.                                            |
| Python 3.11 or newer                   | Back-end language; curriculum definition and validation scripts.                    |
| FastAPI, Uvicorn, Pydantic             | REST API, validation, ASGI server.                                                  |
| SQLAlchemy 2, psycopg 3                | Data access layer and PostgreSQL driver.                                            |
| PostgreSQL 16 (SQLite for development) | Primary data storage.                                                               |
| Redis                                  | Shared rate limiting in production.                                                 |
| Pyodide, sql.js, TypeScript compiler   | Running learner Python, SQL and TypeScript in the browser.                          |
| Node.js 22 and npm                     | Building and running the front end; validating JavaScript/TypeScript/Git exercises. |
| Docker and nginx (optional)            | Single-server deployment; reverse proxy with TLS.                                   |
| Visual Studio Code, Git, GitHub        | Development, version control, continuous integration.                               |
| Web browser                            | Google Chrome, Microsoft Edge, Mozilla Firefox or Safari (current versions).        |

## Hardware Requirement

**Table 3.3.2: Hardware requirements**

| Component                     | Minimum Requirement                                        | Recommended            |
|-------------------------------|------------------------------------------------------------|------------------------|
| Learner device                | Any phone or computer with a modern browser, 2 GB RAM      | 4 GB RAM or more       |
| Network                       | 1 Mbps (the code runtimes are cached after the first load) | 5 Mbps or more         |
| Developer computer: processor | Intel Core i3 / AMD equivalent                             | Intel Core i5 or above |
| Developer computer: RAM       | 4 GB                                                       | 8 GB or more           |
| Developer computer: storage   | 2 GB free space (dependencies)                             | 20 GB SSD              |
| Display                       | 1366 x 768                                                 | Full HD (1920 x 1080)  |
| Server (production)           | 1 vCPU, 512 MB RAM (API); managed PostgreSQL and Redis     | 2 vCPU, 2 GB RAM       |

## 3.4 Preliminary Product Description

**CODEINGO** is a learning platform for students and self-learners and a management console for administrators. A learner follows a colourful path of units and lessons, writes and runs real

code, is tested at the end of each chapter, practises mistakes, builds projects, competes with friends and earns certificates. An administrator keeps the content correct and manages access from a hidden panel.

The preliminary product is divided into the following major modules:

1. **Authentication & Security Module.** Google sign-in, mandatory 2-step verification (e-mail code, authenticator, passkey), sessions, recovery codes, audit log.
2. **Learning Path Module.** Courses, sections, units, lessons, unlocking rules, the lesson player and the five exercise types.
3. **Code Execution & Grading Module.** Sandboxed in-browser runners, optional server runner, server-side grading, anti-copy protection.
4. **Tests & Practice Module.** Chapter tests, readiness tests, spaced-repetition practice, daily challenge.
5. **Projects & Certificates Module.** Multi-step carry-forward projects and public certificates.
6. **Gamification Module.** XP, daily goal, streaks, hearts, badges, progress charts.
7. **Social & Contest Module.** Friend codes, friends league, weekly contests, live leaderboard.
8. **Notification Module.** E-mail codes and optional streak reminders with unsubscribe.
9. **Administration & Content Module.** Hidden admin panel: lessons, users, roles, audit log.
10. **Account & Privacy Module.** Data export, account deletion, Privacy Policy and Terms.
11. **Progressive Web App Module.** Manifest, icons, service worker, offline page, install button.

### 3.5 Conceptual Model

The conceptual model represents the overall working flow of Codeingo. A learner opens the application and signs in with Google or with a passkey and completes the 2-step verification. After successful authentication the learner reaches the learning path, from where a lesson, a chapter test, practice or the social features can be chosen. Every answer is sent to the server, which grades it, awards XP, streak, hearts and badges, and issues a certificate when a course is complete.

The basic working flow of the system is: Codeingo -> Sign in -> 2-step verification -> Learning path -> Select activity (lesson / test / practice / contest) -> Answer -> Server-side grading -> Rewards -> Certificate check -> Continue learning.

The conceptual model can be represented using the following flowchart:

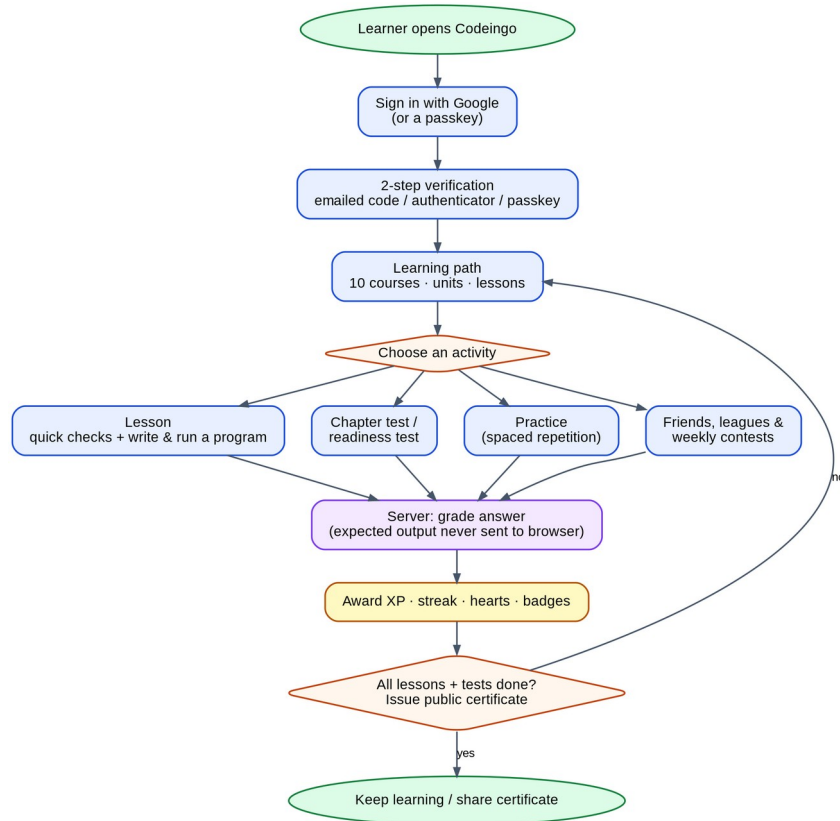


Figure 3.5: Conceptual model of Codeingo

## 3.6 Requirement Gathering Survey

### 3.6.1 Method

To base the feature list on evidence we created an online questionnaire, "Coding Platform Interest Survey" (Google Forms), and shared it with students and young learners in the week of 26 September 2026. The form had nine questions (Annexure A) about age, earlier experience with programming, reasons for difficulty, preferred learning style, platforms already used, the appeal of game-like learning, the first language of interest, motivation, solo versus community learning, and the biggest reason to stop. A total of 43 responses were collected and analysed; blank answers are excluded from the percentages of the question concerned. Names of respondents are not reproduced in this report.

Table 3.6.1: Summary of the survey results (n = 43)

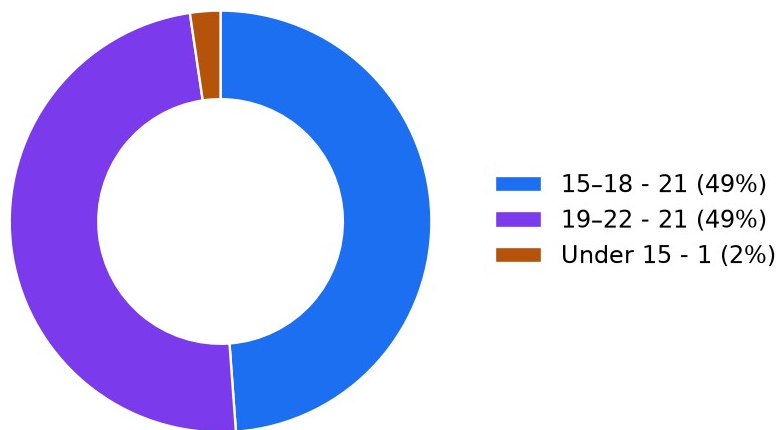
| Question                        | Most common answer      | Share                                                  |
|---------------------------------|-------------------------|--------------------------------------------------------|
| Age group                       | 15-18 and 19-22 (equal) | 21 + 21 of 43 (49% each); 1 under 15                   |
| Experience                      | Currently learning      | 20 of 43 (47%); gave up 12 (28%); never tried 11 (26%) |
| Difficulty (those who answered) | Confusing syntax        | 13 of 40 (33%)                                         |
| Preferred way to learn          | Watching videos         | 13 of 43 (30%); hands-on 12 (28%); games 10 (23%)      |

| Question                           | Most common answer     | Share                                                 |
|------------------------------------|------------------------|-------------------------------------------------------|
| Appeal of game-like learning (1-5) | Average 3.70 out of 5  | 28 of 43 (65%) answered 4 or 5                        |
| First language of interest         | Python                 | 14 of 43 (33%); Java 10 (23%); not sure 7 (16%)       |
| Strongest motivator                | Building real projects | 17 of 43 (40%); friends 10 (23%); points 7 (16%)      |
| Solo or community                  | Mix of both            | 22 of 43 (51%); community 11 (26%); solo 10 (23%)     |
| Biggest reason to stop             | Not enough guidance    | 12 of 43 (28%); no time 10 (23%); gets boring 8 (19%) |

### 3.6.2 Results

The charts below show the answers to each question.

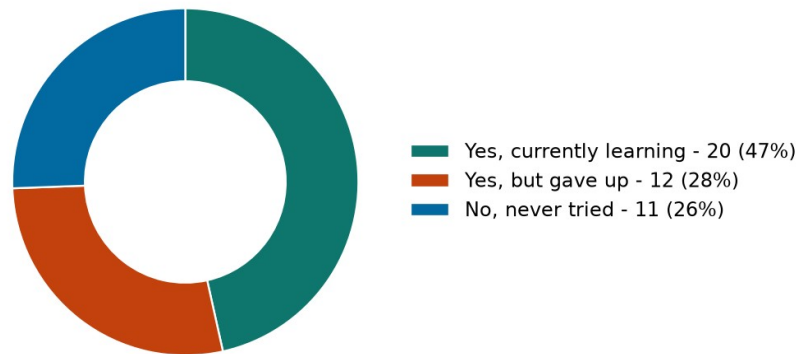
#### Q1. Age group of the respondents



*Figure 3.6.1: Age group of the respondents*

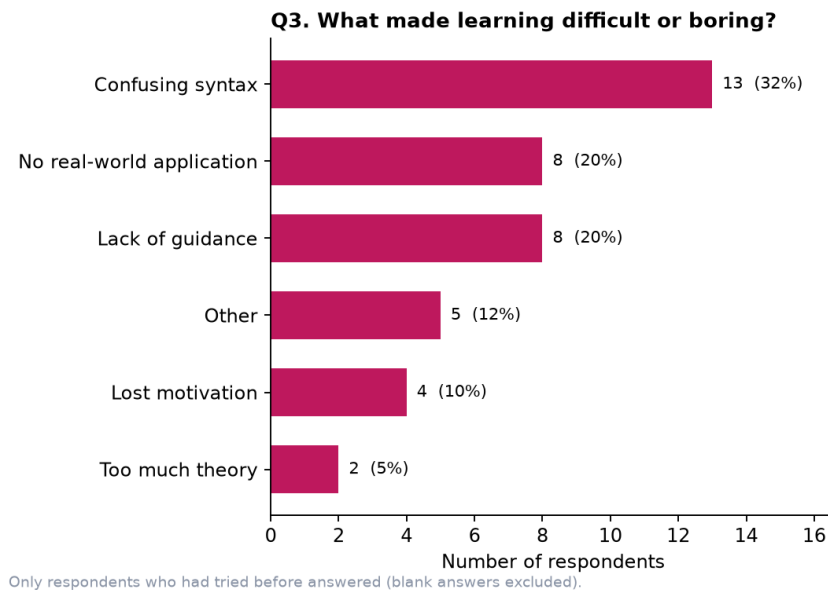
The respondents are school-leaving and early-college learners: 49% are 15-18 years old and 49% are 19-22, with one younger respondent. This is exactly the audience of a college field project and confirms the requirement for a friendly, phone-compatible interface and for Terms that allow users from 13 years of age.

**Q2. Have you tried learning programming before?**



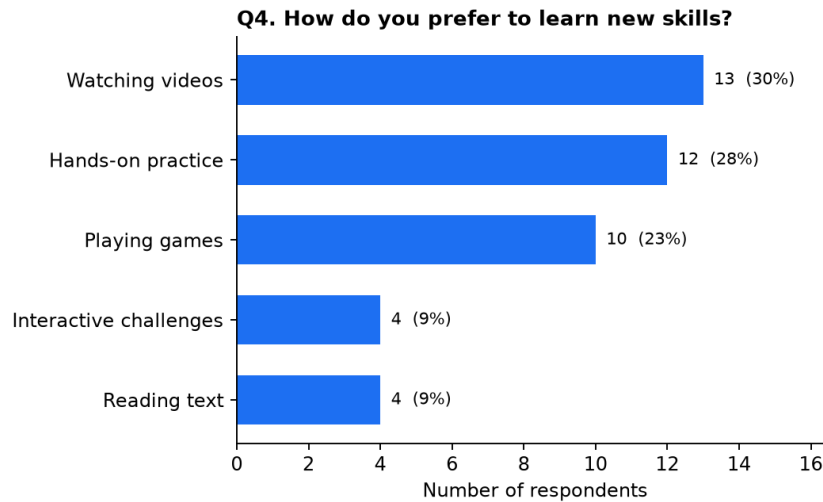
*Figure 3.6.2: Earlier experience with programming*

Almost half of the respondents are already learning (47%), but 28% tried and gave up, and 26% never started. Together, more than half of the sample either quit once or never began, which shows that the first weeks are the critical period and that a platform must work for absolute beginners.



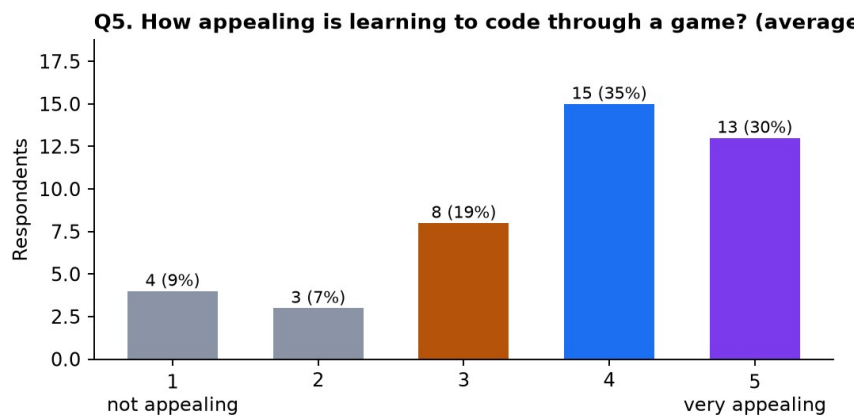
*Figure 3.6.3: Reasons why learning was difficult or boring*

Confusing syntax (13 respondents) was named most often, followed by no real-world application and lack of guidance (8 each). These three reasons together account for 29 of 40 answers (73%). They led directly to short explained lessons with instant feedback, hints, a runnable editor for every program, and projects that look like real tasks.



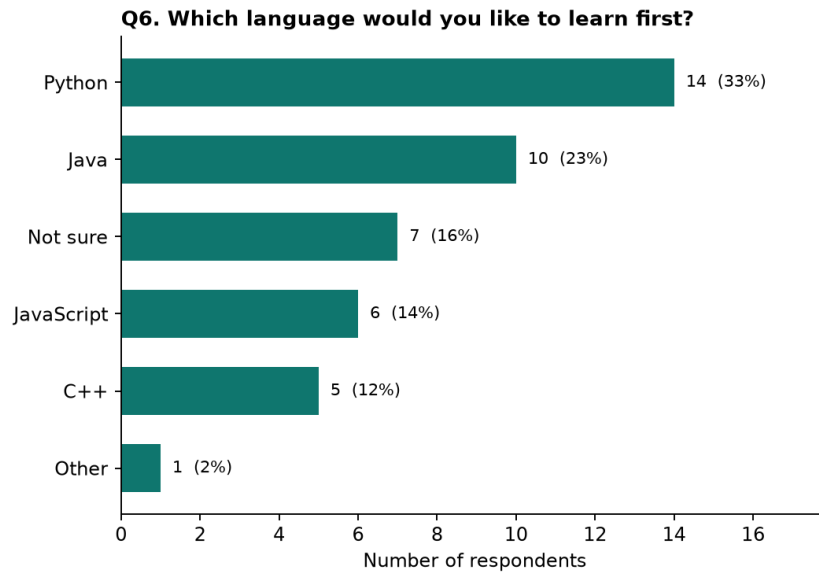
*Figure 3.6.4: Preferred way of learning new skills*

Videos are the most common preference (30%), but 60% of the respondents chose an active style: hands-on practice (28%), playing games (23%) and interactive challenges (9%). A platform that is hands-on and game-like therefore matches what most respondents want, while short concept explanations in each lesson still serve those who like to read or watch.



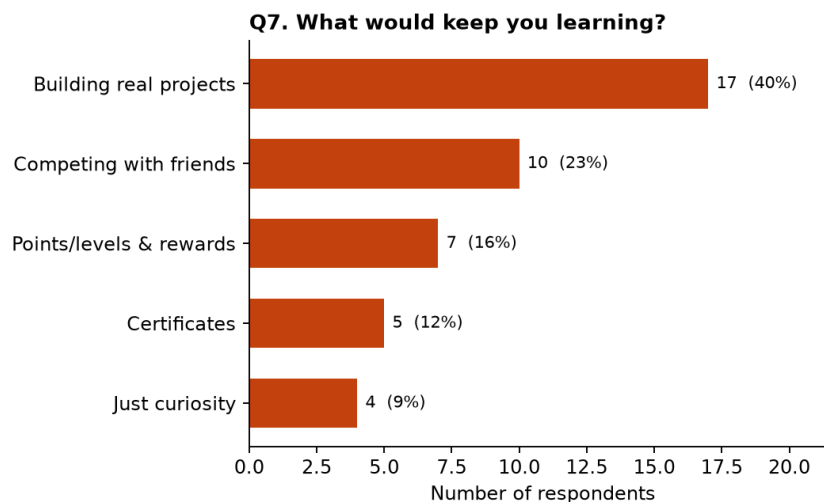
*Figure 3.6.5: How appealing is learning to code through a game?*

The idea of game-like learning received an average of 3.70 out of 5, and 28 of 43 respondents (65%) gave 4 or 5. Only 7 respondents (16%) gave 1 or 2. This supports the decision to build XP, streaks, hearts, badges and leagues into the core of the product.



*Figure 3.6.6: Language the respondents want to learn first*

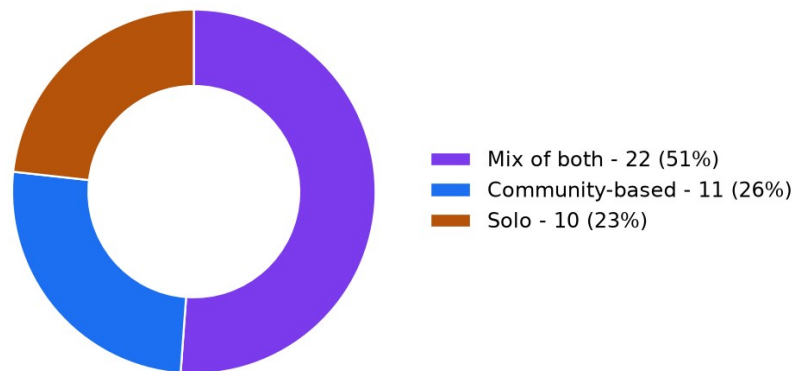
Python is the most requested first language (33%), then Java (23%), JavaScript (14%) and C++ (12%); 16% are not sure. The platform therefore starts with Python and offers Java, JavaScript and C++ as full courses, and adds C, SQL, HTML & CSS, TypeScript, Git and Data Structures & Algorithms. Learners who are unsure can try the daily challenge, which draws from all courses.



*Figure 3.6.7: What would keep respondents learning*

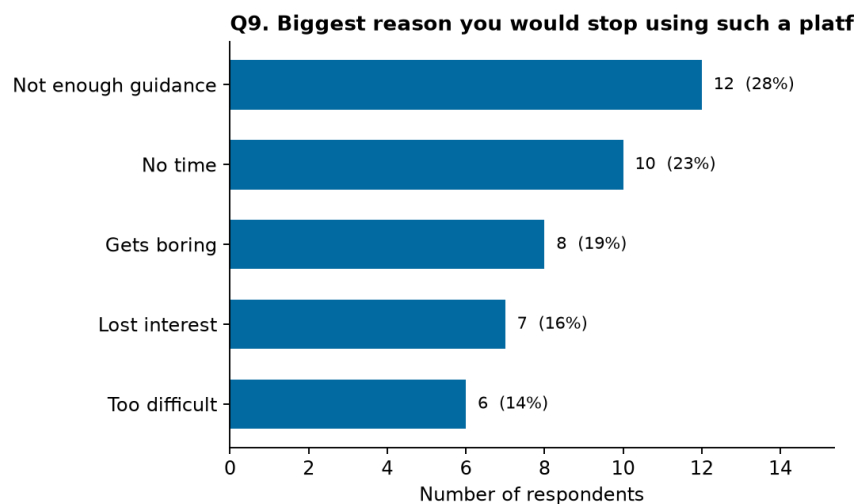
Building real projects is the strongest motivator (40%), ahead of competing with friends (23%), points and levels (16%) and certificates (12%). This is why every section ends with a project and every course with a certificate, and why friends, leagues and contests exist.

**Q8. Solo or community?**



*Figure 3.6.8: Learning alone or with a community*

Half of the respondents (51%) want a mix of both, 26% prefer a community and 23% prefer to study alone. Since 77% want at least some community, social features are included, but they are optional so that solo learners are not pushed into them.



*Figure 3.6.9: Biggest reason to stop using a learning platform*

Not enough guidance (28%) and no time (23%) are the main reasons to quit, followed by boredom (19%), loss of interest (16%) and difficulty (14%). The answers led to a clear unlocked path with tests and hints, five-minute lessons, adjustable daily goals, streak reminders, and variety in exercise types and contests.

The respondents were also asked which platforms they had used. Of those who named one, Codecademy (5 mentions) and freeCodeCamp (4) were the most common, followed by Scratch (2), LeetCode (1) and local compilers such as Turbo C (2); 15 wrote "No". This is consistent with the existing-systems study in Chapter 2.

### 3.6.3 From Findings to Features

**Table 3.6.2: Survey findings and the features that answer them**

| Finding                                          | Feature in Codeingo                                                                         | Where                          |
|--------------------------------------------------|---------------------------------------------------------------------------------------------|--------------------------------|
| 33% name confusing syntax as the main difficulty | Short explained lessons, instant feedback with explanation, AI-style hints, runnable editor | Lesson player, Codi hints      |
| 28% quit for lack of guidance                    | Unlocked path, chapter tests, readiness tests, hints, practice mode                         | Learning path, tests, practice |
| Real-world application missing (20%)             | 30 projects, each step continuing your own code                                             | End of every section           |
| Real projects motivate 40%                       | Projects and certificates                                                                   | Projects, profile              |
| Competing with friends motivates 23%             | Friends, friends league, weekly contests with live board                                    | Leagues, contests              |
| Points and levels motivate 16%                   | XP, streaks, hearts, badges, daily goal                                                     | Gamification                   |
| 65% like game-like learning                      | Game mechanics throughout                                                                   | All pages                      |
| 77% want at least some community                 | Optional social features; everything works solo                                             | Leagues                        |
| Python (33%) and Java (23%) most wanted          | Python as default first course; Java, JavaScript, C++ full courses                          | Courses                        |
| Not enough time (23%)                            | Five-minute lessons, daily goals, streak reminder e-mails (opt-out)                         | Lessons, profile               |
| Gets boring (19%)                                | Five exercise types, daily challenge, contests, colourful design                            | Exercises, contests            |

## GUIDE INTERACTION REPORT

*(To be maintained by the student and submitted as part of the Field Project documentation)*

|                                       |                        |
|---------------------------------------|------------------------|
| <b>Name of the Student</b>            | [STUDENT NAME]         |
| <b>Program /Semester</b>              | B.Sc I.T./ Semester V  |
| <b>Roll No/Seat No</b>                | [ROLL NO / SEAT NO]    |
| <b>Major Subject / Specialization</b> | Information Technology |
| <b>Field project Title</b>            | CODEINGO               |
| <b>Name of the Faculty mentor</b>     | [GUIDE NAME]           |

### Overview (Max 150 Words)

*(Brief summary of key activities, tasks and projects undertaken during the period)*

### CHAPTER 3: REQUIREMENT AND ANALYSIS

This chapter defines the problem: beginners depend on scattered videos and tools, quit early, and many platforms are weakly gamified or paid. It lists functional requirements (authentication, learning path, assessment, motivation, administration, privacy) and non-functional requirements (security, performance, usability, reliability, maintainability, scalability, accessibility, privacy), software and hardware tables, eleven modules, the conceptual flow, and a requirement survey of 43 learners with nine charts and a table that links each finding to a feature.

### Learning Outcomes (Max 100 words):

We learned to write a proper problem definition and proposed solution, and the difference between functional and non-functional requirements. We learned how to design a survey, read its charts, and turn answers into requirements: for example 40% wanting real projects led to a project at the end of every section. We learned to break a large system into modules and to draw a conceptual flow from sign-in to certificate.

### Challenges Faced (Max 100 words):

Writing requirements without repeating the same points was hard, and converting opinions from the survey into precise, testable statements took time. Some answers were blank or free text, so we had to clean the data and decide how to count them. Deciding which security rules are mandatory (for example 2-step verification for every account) and explaining why was another challenge, as was keeping the scope realistic for one semester.

### Mentor Feedback and Suggestions (Max 100 words):

[To be filled in by the guide]

Signature of the Internal Guide

# CHAPTER 4

## SYSTEM DESIGN

### 4.1 Basic Modules

#### 1. Authentication & Security Module

- Signs users in with Google (OpenID Connect with PKCE) or a passkey.
- Requires 2-step verification: emailed code, authenticator app (TOTP) or passkey; issues HttpOnly session cookies and rotating refresh tokens.
- Provides recovery codes, lock-out after five wrong codes, rate limiting and CSRF protection.
- Writes security events to the audit log.

#### 2. Learning Path Module

- Stores courses, units, lessons and exercises and synchronises the built-in curriculum at start-up.
- Decides which lessons are locked or unlocked, based on earlier lessons and passed chapter tests (old progress is grandfathered).
- Serves the lesson player with exercises, without the private solutions.

#### 3. Code Execution & Grading Module

- Runs Python, JavaScript, TypeScript, SQL, HTML/CSS and Git programs in sandboxed Web Workers in the browser; optionally runs Java, C and C++ in a server sandbox.
- Grades every answer on the server by comparing the reported output with expected output that is never sent to the browser; checks required patterns so a program cannot simply print the answer.
- Blocks paste/drop in answer boxes and marks revealed answers as non-copyable.

#### 4. Tests & Practice Module

- Chapter tests (8 questions, 6 to pass), readiness tests (15 questions, 12 to pass) with jump-ahead, no hearts lost and no hints during a test.
- Spaced-repetition practice: Leitner boxes with intervals of 0, 1, 3, 7, 16 and 35 days; sessions of ten questions.
- Daily challenge from a rotating course.

#### 5. Projects & Certificates Module

- Thirty three-step projects (the last lesson of each section). Each step starts from the learner's own code of the previous step.
- A certificate with a random public code is issued once when all lessons and chapter tests of a course are complete; a public page verifies and prints it.

#### 6. Gamification Module

- XP, daily goal, streaks, hearts that refill over time, fourteen badges.

- Progress page with charts of XP per day and lessons per language.

## **7. Social & Contest Module**

- Friend codes, invite links, follow and unfollow, friends league beside the global weekly league.
- Weekly contest for every course: same ten questions for all, one ten-minute attempt, ranked by score then time, with a live (polled) leaderboard.

## **8. Notification Module**

- Sends e-mail through SMTP over TLS: sign-in codes and optional streak reminders (once per day, atomic claim, signed one-click unsubscribe).

## **9. Administration & Content Module**

- Hidden from non-admins (the page and API answer 404). Edits lessons and exercises with server-side validation, disables users, searches users and audit log.
- Grants or removes admin access only after the acting admin re-enters a 2-step code; owner accounts (ADMIN\_EMAILS) cannot be demoted.

## **10. Account & Privacy Module**

- Download of all personal data as a JSON file; permanent account deletion (cascading) after a 2-step code.
- Privacy Policy and Terms of Service pages.

## **11. Progressive Web App Module**

- Web-app manifest and icons, a service worker that caches static files and shows an offline page, an install button.

## **4.2 Data Design**

Codeingo stores its data in a relational database (PostgreSQL in production, SQLite in development and tests) managed through SQLAlchemy models. There are 23 tables. Relations use foreign keys with ON DELETE CASCADE, so deleting a user removes everything that belongs to that user. The curriculum tables (courses, units, lessons, exercises) are synchronised from code at start-up and keyed by a stable key such as python/3/1, so learners' progress survives content updates. The Entity-Relationship diagram below shows the main tables; the field lists follow.

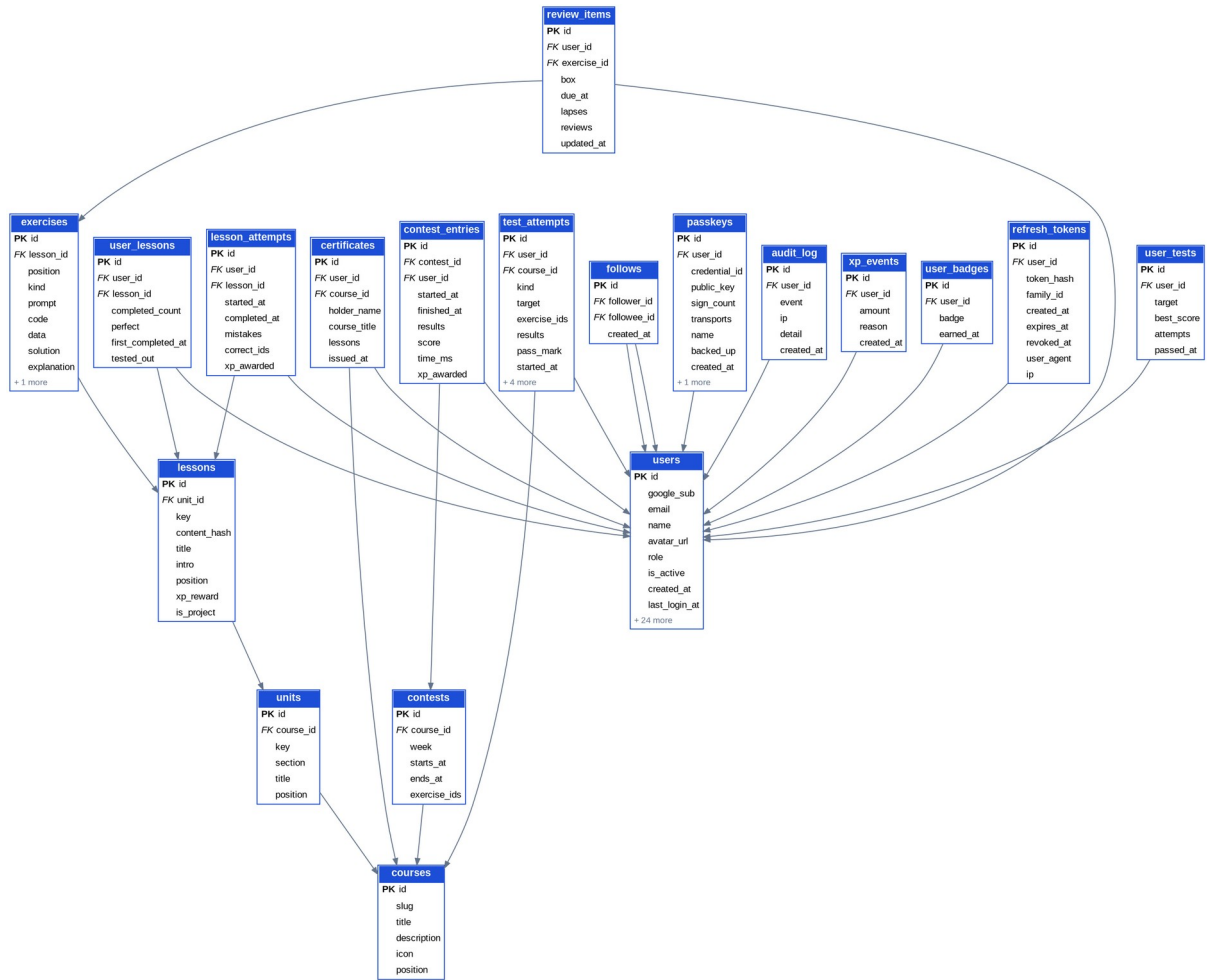


Figure 4.2: Entity-Relationship diagram (generated from the SQLAlchemy models)

## 1. users

Table 4.2.1: users - Account, security settings, gamification state

| Field         | Type / Key   | Description                                 |
|---------------|--------------|---------------------------------------------|
| id            | Integer / PK | Primary identifier                          |
| google_sub    | String       | Google account identifier                   |
| email         | String       | E-mail address (never shown to other users) |
| name          | String       | Display name                                |
| avatar_url    | String       | Profile picture URL                         |
| role          | String       | learner or admin                            |
| is_active     | Boolean      | False when the account is disabled          |
| created_at    | DateTime     | Creation timestamp                          |
| last_login_at | DateTime     | Last successful sign-in                     |
| token_version | Integer      | Increase to invalidate all access tokens    |
| mfa_enabled   | Boolean      | True when 2-step verification is set up     |
| mfa_method    | String       | email or totp                               |

| Field                   | Type / Key | Description                                      |
|-------------------------|------------|--------------------------------------------------|
| totp_secret_enc         | Text       | Authenticator secret, encrypted (Fernet)         |
| totp_pending_enc        | Text       | Secret during set-up, encrypted                  |
| totp_last_step          | Integer    | Last accepted TOTP time-step (replay protection) |
| mfa_failed_count        | Integer    | Consecutive wrong codes                          |
| mfa_locked_until        | DateTime   | Lock-out end time                                |
| email_code_hash         | String     | Keyed hash of the emailed code (never the code)  |
| email_code_expires_at   | DateTime   | Code expiry (10 minutes)                         |
| email_code_sent_at      | DateTime   | When the code was sent (resend cool-down)        |
| email_code_attempts     | Integer    | Wrong guesses for this code                      |
| email_code_window_start | DateTime   | Start of the hourly send window                  |
| email_code_window_count | Integer    | Codes sent in the window                         |
| xp_total                | Integer    | Total XP earned                                  |
| streak_current          | Integer    | Current streak in days                           |
| streak_best             | Integer    | Best streak                                      |
| last_active_date        | Date       | Last day with activity                           |
| hearts                  | Integer    | Hearts left                                      |
| hearts_updated_at       | DateTime   | Heart refill reference time                      |
| daily_goal              | Integer    | Daily XP goal                                    |
| friend_code             | String     | Shareable 8-character code                       |
| reminder_emails         | Boolean    | Whether streak reminder e-mails are on           |
| last_reminder_on        | Date       | Day of the last reminder (max one a day)         |

## 2. courses

**Table 4.2.2: courses - The ten courses**

| Field       | Type / Key   | Description                    |
|-------------|--------------|--------------------------------|
| id          | Integer / PK | Primary identifier             |
| slug        | String       | Short unique name used in URLs |
| title       | String       | Display title                  |
| description | String       | Short description              |
| icon        | String       | Emoji icon                     |
| position    | Integer      | Order position                 |

## 3. units

**Table 4.2.3: units - Units (chapters) of a course**

| Field | Type / Key   | Description        |
|-------|--------------|--------------------|
| id    | Integer / PK | Primary identifier |

| Field     | Type / Key                | Description                                       |
|-----------|---------------------------|---------------------------------------------------|
| course_id | Integer /<br>FK → courses | Course (courses.id)                               |
| key       | String                    | Stable curriculum key, e.g. python/3/1            |
| section   | String                    | Section name (Beginner / Intermediate / Advanced) |
| title     | String                    | Display title                                     |
| position  | Integer                   | Order position                                    |

## 4. lessons

**Table 4.2.4: lessons - Lessons and projects**

| Field        | Type / Key           | Description                                                   |
|--------------|----------------------|---------------------------------------------------------------|
| id           | Integer / PK         | Primary identifier                                            |
| unit_id      | Integer / FK → units | Unit (units.id)                                               |
| key          | String               | Stable curriculum key, e.g. python/3/1                        |
| content_hash | String               | Hash of the lesson content (detects changes at start-up sync) |
| title        | String               | Display title                                                 |
| intro        | Text                 | Concept explanation shown before the exercises                |
| position     | Integer              | Order position                                                |
| xp_reward    | Integer              | XP awarded for completing the lesson                          |
| is_project   | Boolean              | True for end-of-section project lessons                       |

## 5. exercises

**Table 4.2.5: exercises - Questions and programs**

| Field       | Type / Key                | Description                                                               |
|-------------|---------------------------|---------------------------------------------------------------------------|
| id          | Integer / PK              | Primary identifier                                                        |
| lesson_id   | Integer /<br>FK → lessons | Lesson (lessons.id)                                                       |
| position    | Integer                   | Order position                                                            |
| kind        | String                    | Exercise kind (mcq, fill, order, code, run) or test kind (unit / section) |
| prompt      | Text                      | Question text                                                             |
| code        | Text                      | Code shown with the question                                              |
| data        | JSON                      | Public part (options, starter code, tests) as JSON                        |
| solution    | JSON                      | Private answer data as JSON, never sent to the browser                    |
| explanation | Text                      | Explanation shown after answering                                         |
| hint        | Text                      | Author hint                                                               |

## 6. user\_lessons

**Table 4.2.6: user\_lessons - A learner's lesson progress**

| Field              | Type / Key                | Description                                           |
|--------------------|---------------------------|-------------------------------------------------------|
| id                 | Integer / PK              | Primary identifier                                    |
| user_id            | Integer / FK → users      | Owner of the record (users.id); deleted with the user |
| lesson_id          | Integer /<br>FK → lessons | Lesson (lessons.id)                                   |
| completed_count    | Integer                   | Times completed                                       |
| perfect            | Boolean                   | True if finished without mistakes                     |
| first_completed_at | DateTime                  | First completion time                                 |
| tested_out         | Boolean                   | True if skipped via a readiness test                  |

## 7. lesson\_attempts

**Table 4.2.7: lesson\_attempts - One play-through of a lesson**

| Field        | Type / Key                | Description                                           |
|--------------|---------------------------|-------------------------------------------------------|
| id           | String / PK               | Primary identifier                                    |
| user_id      | Integer / FK → users      | Owner of the record (users.id); deleted with the user |
| lesson_id    | Integer /<br>FK → lessons | Lesson (lessons.id)                                   |
| started_at   | DateTime                  | When it started                                       |
| completed_at | DateTime                  | When it was finished                                  |
| mistakes     | Integer                   | Number of wrong answers                               |
| correct_ids  | JSON                      | Exercises answered correctly                          |
| xp_awarded   | Integer                   | XP given for this attempt                             |

## 8. user\_tests

**Table 4.2.8: user\_tests - Best result per test**

| Field      | Type / Key           | Description                                           |
|------------|----------------------|-------------------------------------------------------|
| id         | Integer / PK         | Primary identifier                                    |
| user_id    | Integer / FK → users | Owner of the record (users.id); deleted with the user |
| target     | String               | unit:<id> or section:<course>:<name>                  |
| best_score | Integer              | Best score                                            |
| attempts   | Integer              | Number of attempts                                    |
| passed_at  | DateTime             | When it was first passed                              |

## 9. test\_attempts

**Table 4.2.9: test\_attempts - One sitting of a chapter or readiness test**

| Field        | Type / Key                | Description                                                               |
|--------------|---------------------------|---------------------------------------------------------------------------|
| id           | String / PK               | Primary identifier                                                        |
| user_id      | Integer / FK → users      | Owner of the record (users.id); deleted with the user                     |
| course_id    | Integer /<br>FK → courses | Course (courses.id)                                                       |
| kind         | String                    | Exercise kind (mcq, fill, order, code, run) or test kind (unit / section) |
| target       | String                    | unit:<id> or section:<course>:<name>                                      |
| exercise_ids | JSON                      | Questions in this attempt (JSON list)                                     |
| results      | JSON                      | Answers so far: {exercise id: correct}                                    |
| pass_mark    | Integer                   | Score needed to pass                                                      |
| started_at   | DateTime                  | When it started                                                           |
| completed_at | DateTime                  | When it was finished                                                      |
| score        | Integer                   | Final score                                                               |
| passed       | Boolean                   | True if passed                                                            |
| xp_awarded   | Integer                   | XP given for this attempt                                                 |

## 10. review\_items

**Table 4.2.10: review\_items - Spaced-repetition list**

| Field       | Type / Key                  | Description                                           |
|-------------|-----------------------------|-------------------------------------------------------|
| id          | Integer / PK                | Primary identifier                                    |
| user_id     | Integer / FK → users        | Owner of the record (users.id); deleted with the user |
| exercise_id | Integer /<br>FK → exercises | Exercise (exercises.id)                               |
| box         | Integer                     | Leitner box 0-5                                       |
| due_at      | DateTime                    | When the item is due again                            |
| lapses      | Integer                     | Times forgotten                                       |
| reviews     | Integer                     | Times reviewed                                        |
| updated_at  | DateTime                    | Last update timestamp                                 |

## 11. practice\_attempts

**Table 4.2.11: practice\_attempts - Practice sessions**

| Field   | Type / Key           | Description                                           |
|---------|----------------------|-------------------------------------------------------|
| id      | String / PK          | Primary identifier                                    |
| user_id | Integer / FK → users | Owner of the record (users.id); deleted with the user |

| Field         | Type / Key                | Description                            |
|---------------|---------------------------|----------------------------------------|
| course_id     | Integer /<br>FK → courses | Course (courses.id)                    |
| exercise_ids  | JSON                      | Questions in this attempt (JSON list)  |
| results       | JSON                      | Answers so far: {exercise id: correct} |
| started_at    | DateTime                  | When it started                        |
| completed_at  | DateTime                  | When it was finished                   |
| xp_awarded    | Integer                   | XP given for this attempt              |
| heart_awarded | Boolean                   | True if a heart was given back         |

## 12. daily\_claims

**Table 4.2.12: daily\_claims - Daily challenge answers**

| Field   | Type / Key           | Description                                           |
|---------|----------------------|-------------------------------------------------------|
| id      | Integer / PK         | Primary identifier                                    |
| user_id | Integer / FK → users | Owner of the record (users.id); deleted with the user |
| day     | Date                 | Calendar day                                          |
| correct | Boolean              | True if answered correctly                            |

## 13. certificates

**Table 4.2.13: certificates - Issued certificates**

| Field        | Type / Key                | Description                                           |
|--------------|---------------------------|-------------------------------------------------------|
| id           | String / PK               | Primary identifier                                    |
| user_id      | Integer / FK → users      | Owner of the record (users.id); deleted with the user |
| course_id    | Integer /<br>FK → courses | Course (courses.id)                                   |
| holder_name  | String                    | Name printed on the certificate (snapshot)            |
| course_title | String                    | Course name printed on the certificate                |
| lessons      | Integer                   | Number of lessons completed                           |
| issued_at    | DateTime                  | Issue timestamp                                       |

## 14. xp\_events

**Table 4.2.14: xp\_events - XP history**

| Field      | Type / Key           | Description                                           |
|------------|----------------------|-------------------------------------------------------|
| id         | Integer / PK         | Primary identifier                                    |
| user_id    | Integer / FK → users | Owner of the record (users.id); deleted with the user |
| amount     | Integer              | XP amount                                             |
| reason     | String               | Why the XP was awarded                                |
| created_at | DateTime             | Creation timestamp                                    |

## 15. user\_badges

**Table 4.2.15: user\_badges - Earned badges**

| Field     | Type / Key           | Description                                           |
|-----------|----------------------|-------------------------------------------------------|
| id        | Integer / PK         | Primary identifier                                    |
| user_id   | Integer / FK → users | Owner of the record (users.id); deleted with the user |
| badge     | String               | Badge key                                             |
| earned_at | DateTime             | When it was earned                                    |

## 16. follows

**Table 4.2.16: follows - Friend relations**

| Field       | Type / Key           | Description         |
|-------------|----------------------|---------------------|
| id          | Integer / PK         | Primary identifier  |
| follower_id | Integer / FK → users | User who follows    |
| followee_id | Integer / FK → users | User being followed |
| created_at  | DateTime             | Creation timestamp  |

## 17. contests

**Table 4.2.17: contests - Weekly contests**

| Field        | Type / Key                | Description                           |
|--------------|---------------------------|---------------------------------------|
| id           | Integer / PK              | Primary identifier                    |
| course_id    | Integer /<br>FK → courses | Course (courses.id)                   |
| week         | String                    | ISO week, e.g. 2026-W40               |
| starts_at    | DateTime                  | Contest start (Monday 00:00 UTC)      |
| ends_at      | DateTime                  | Contest end                           |
| exercise_ids | JSON                      | Questions in this attempt (JSON list) |

## 18. contest\_entries

**Table 4.2.18: contest\_entries - A learner's single timed run**

| Field       | Type / Key                 | Description                                           |
|-------------|----------------------------|-------------------------------------------------------|
| id          | Integer / PK               | Primary identifier                                    |
| contest_id  | Integer /<br>FK → contests | Contest (contests.id)                                 |
| user_id     | Integer / FK → users       | Owner of the record (users.id); deleted with the user |
| started_at  | DateTime                   | When it started                                       |
| finished_at | DateTime                   | When the run finished                                 |
| results     | JSON                       | Answers so far: {exercise id: correct}                |

| Field      | Type / Key | Description                |
|------------|------------|----------------------------|
| score      | Integer    | Final score                |
| time_ms    | Integer    | Time taken in milliseconds |
| xp_awarded | Integer    | XP given for this attempt  |

## 19. passkeys

**Table 4.2.19: passkeys - WebAuthn credentials**

| Field         | Type / Key           | Description                                           |
|---------------|----------------------|-------------------------------------------------------|
| id            | Integer / PK         | Primary identifier                                    |
| user_id       | Integer / FK → users | Owner of the record (users.id); deleted with the user |
| credential_id | String               | WebAuthn credential id (base64url)                    |
| public_key    | Text                 | Public key only (private key stays on the device)     |
| sign_count    | Integer              | Signature counter (clone detection)                   |
| transports    | JSON                 | Authenticator transports                              |
| name          | String               | Display name                                          |
| backed_up     | Boolean              | True for synced passkeys                              |
| created_at    | DateTime             | Creation timestamp                                    |
| last_used_at  | DateTime             | Last sign-in with this passkey                        |

## 20. webauthn\_challenges

**Table 4.2.20: webauthn\_challenges - One-time passkey challenges**

| Field      | Type / Key           | Description                                           |
|------------|----------------------|-------------------------------------------------------|
| id         | String / PK          | Primary identifier                                    |
| challenge  | String               | Random one-time challenge (base64url)                 |
| purpose    | String               | register or login                                     |
| user_id    | Integer / FK → users | Owner of the record (users.id); deleted with the user |
| expires_at | DateTime             | Expiry timestamp                                      |

## 21. recovery\_codes

**Table 4.2.21: recovery\_codes - Authenticator recovery codes**

| Field     | Type / Key           | Description                                           |
|-----------|----------------------|-------------------------------------------------------|
| id        | Integer / PK         | Primary identifier                                    |
| user_id   | Integer / FK → users | Owner of the record (users.id); deleted with the user |
| code_hash | String               | Hash of the recovery code                             |
| used_at   | DateTime             | When it was used                                      |

## 22. refresh\_tokens

**Table 4.2.22: refresh\_tokens - Rotating refresh tokens**

| Field      | Type / Key           | Description                                           |
|------------|----------------------|-------------------------------------------------------|
| id         | Integer / PK         | Primary identifier                                    |
| user_id    | Integer / FK → users | Owner of the record (users.id); deleted with the user |
| token_hash | String               | Hash of the refresh token                             |
| family_id  | String               | Token family (reuse detection)                        |
| created_at | DateTime             | Creation timestamp                                    |
| expires_at | DateTime             | Expiry timestamp                                      |
| revoked_at | DateTime             | When it was revoked                                   |
| user_agent | String               | Browser identification                                |
| ip         | String               | Client IP address                                     |

## 23. audit\_log

**Table 4.2.23: audit\_log - Security audit log**

| Field      | Type / Key           | Description                                           |
|------------|----------------------|-------------------------------------------------------|
| id         | Integer / PK         | Primary identifier                                    |
| user_id    | Integer / FK → users | Owner of the record (users.id); deleted with the user |
| event      | String               | Event name, e.g. login, mfa_failed                    |
| ip         | String               | Client IP address                                     |
| detail     | String               | Short extra detail                                    |
| created_at | DateTime             | Creation timestamp                                    |

## 4.3 Logic Design

The logic design shows how the layers of Codeingo are connected. The learner's browser runs the Next.js application and the sandboxed code runners. Requests reach the hosting edge (Vercel or nginx), which adds security headers and forwards /api requests to the FastAPI back end. The back end contains the authentication, learning, grading and administration services, and talks to PostgreSQL and Redis. External services are only Google sign-in and SMTP e-mail (and, optionally, an AI hint provider and a code sandbox).

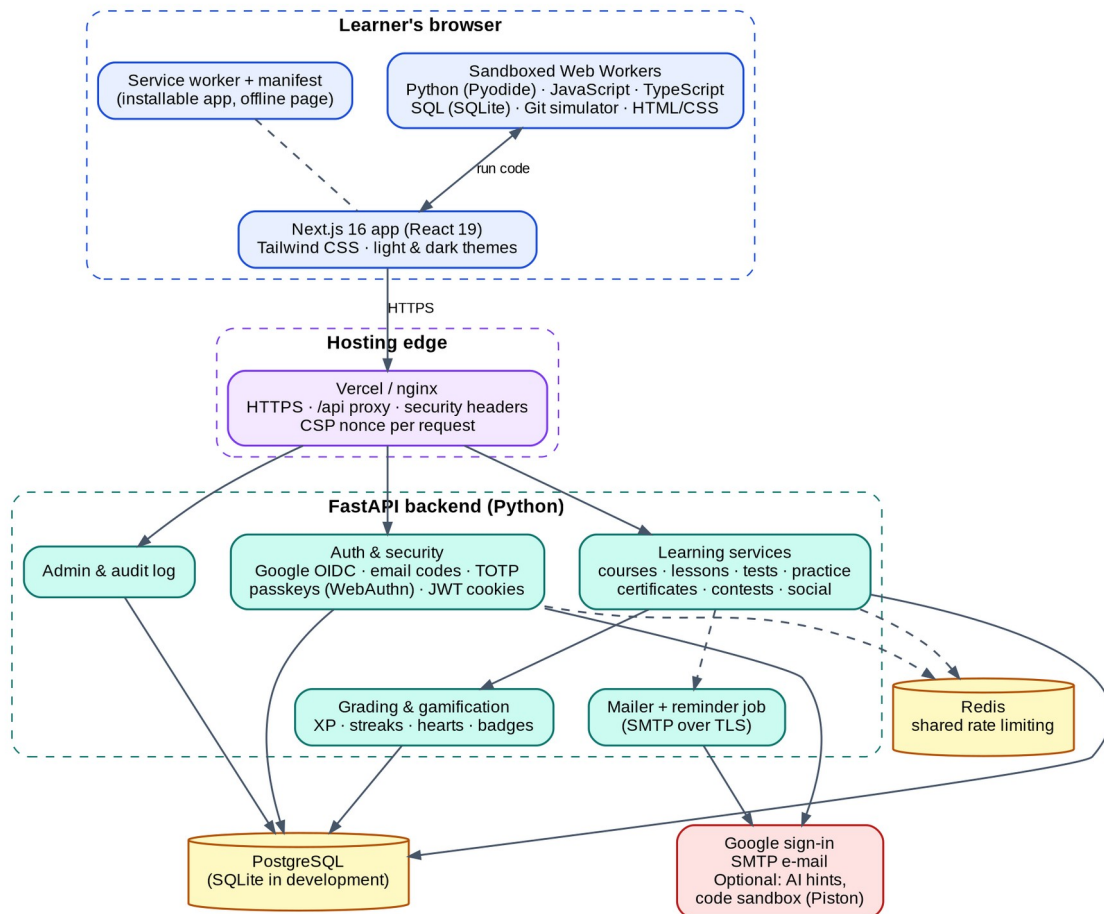


Figure 4.3: System architecture (logic diagram)

#### 4.3.1 Sign-in and 2-step verification flow

1. The learner chooses "Continue with Google" (or "Sign in with a passkey"). For Google, the server redirects with a random state, a nonce and a PKCE challenge; the callback verifies the ID token against Google's keys.
2. The server creates or finds the user and sets a short-lived MFA cookie that proves only that the first step is done; no session exists yet.
3. For e-mail codes the server generates a 6-digit code, stores only a keyed hash with a 10-minute expiry, and sends the code over SMTP. A wrong code counts toward a limit of five; a correct code is accepted once.
4. On success the server starts a session: a short-lived access token and a rotating refresh token in HttpOnly cookies, plus a CSRF token. A passkey sign-in (user verification required) skips the e-mailed code because it already combines device and biometric/PIN.

#### 4.3.2 Answer grading flow

1. The browser asks for a lesson. The server returns the questions without solutions (for run exercises the expected outputs are also withheld).
2. For a program the browser runs the learner's code in a worker, once for every test case, and sends the code and the printed outputs.

3. The server compares the outputs with the stored expected outputs, checks the required and forbidden patterns, and replies correct/incorrect with the explanation. The correct answer is revealed only after the answer.
4. The server records the result, updates the review list, awards XP, streak and badges, and, at the end of a course, issues the certificate. A minimum time per question and per lesson stops automation.

### 4.3.3 Security design

- Cookies: HttpOnly, Secure, SameSite=Strict; CSRF double-submit token and Origin check on every state-changing request.
- Content-Security-Policy with a fresh nonce per request; code workers get an even stricter policy (no network).
- Rate limiting per IP and per user (Redis); lock-out after five wrong 2-step codes; reuse detection for refresh tokens.
- Secrets: authenticator secrets encrypted with Fernet, e-mail codes and recovery codes stored only as hashes, passkeys store only public keys.
- Admin API answers 404 to non-admins; role changes need a fresh 2-step code and are audit-logged.

## 4.4 User Interface Design

The interface follows a Duolingo-style layout: a left sidebar on large screens and a bottom navigation bar on phones, a learning path in the centre and cards on the right. The diagram shows how the screens are connected.

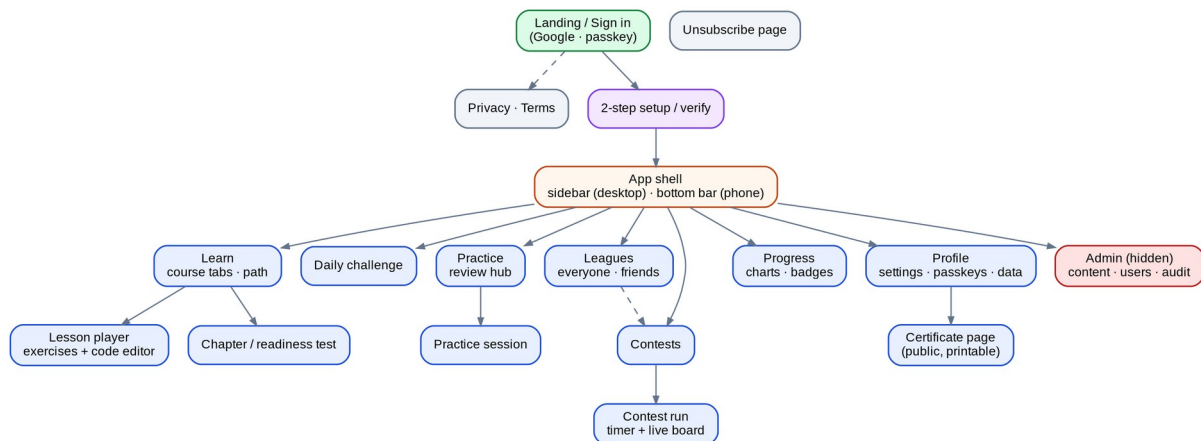


Figure 4.4: Screen map of Codeingo (user interface diagram)

### Design system

All colours are defined as CSS variables, so the two required themes swap by one class. Section accent colours (orange for Daily, violet for Practice, gold for Leagues, pink for Contests, teal for Progress, sky for Profile, red for Admin) re-colour the page using a tint class.

**Table 4.4.1: Main colour tokens of the two themes**

| Token            | Light theme                                           | Dark theme                | Use                             |
|------------------|-------------------------------------------------------|---------------------------|---------------------------------|
| Background       | White #FFFFFF                                         | Black #000000             | Page background                 |
| Primary          | Blue #1D6FF2                                          | Green #22C55E             | Buttons, active items, banners  |
| Text             | Near-black #0A0A0A                                    | Soft white #E8F5EC        | Body text                       |
| Surface / raised | #F4F7FF / white                                       | #0A0F0C / #101712         | Cards and panels                |
| Accents          | Violet, pink, orange, teal, sky, amber (deeper tones) | Same hues, brighter tones | Section colours, charts, badges |
| Danger / success | Red #EA2B2B                                           | Red #FF5454               | Mistakes, delete actions        |

### **Responsive and accessible behaviour**

- Layouts adapt from 320 px phones to 4K monitors; the root font size steps up on very large screens.
- All controls are reachable by keyboard with visible focus; charts have a table alternative; motion respects the reduced-motion setting.
- Text colours were chosen for contrast; light-theme accents are deeper so that white text on them stays readable.

## **4.5 Work Break-Down Structure**

The project was divided into seven work packages: planning and survey, design, back end, front end, content, testing and quality, and deployment and documentation. The work break-down structure shows the main tasks of each package.

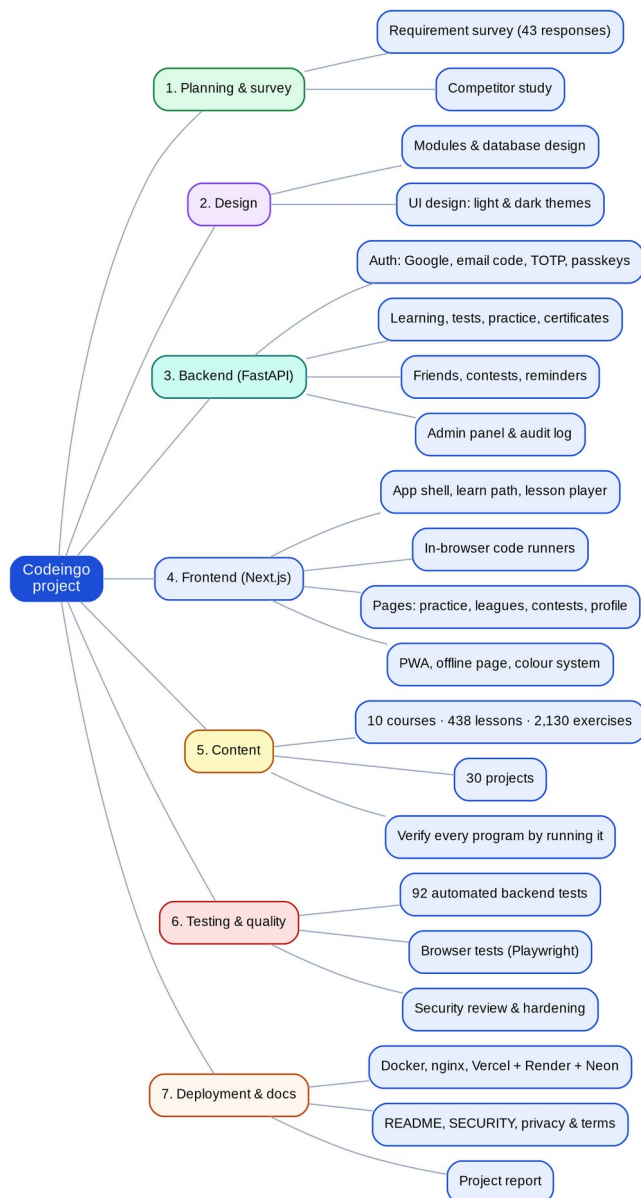


Figure 4.5: Work break-down structure of Codeingo

## GUIDE INTERACTION REPORT

*(To be maintained by the student and submitted as part of the Field Project documentation)*

|                                       |                        |
|---------------------------------------|------------------------|
| <b>Name of the Student</b>            | [STUDENT NAME]         |
| <b>Program /Semester</b>              | B.Sc I.T./ Semester V  |
| <b>Roll No/Seat No</b>                | [ROLL NO / SEAT NO]    |
| <b>Major Subject / Specialization</b> | Information Technology |
| <b>Field project Title</b>            | CODEINGO               |
| <b>Name of the Faculty mentor</b>     | [GUIDE NAME]           |

### Overview (Max 150 Words)

*(Brief summary of key activities, tasks and projects undertaken during the period)*

### CHAPTER 4: SYSTEM DESIGN

This chapter shows the design of Codeingo: eleven modules (authentication, learning path, code execution and grading, tests and practice, projects and certificates, gamification, social and contests, notifications, administration, privacy, progressive web app), the data design with 23 relational tables and an ER diagram generated from the code, the logic design with architecture, sign-in flow, grading flow and security design, the user-interface design with a screen map and colour tokens for both themes, and a work break-down structure.

### Learning Outcomes (Max 100 words):

We learned to design a system step by step and to make each module responsible for one thing. We learned relational design: foreign keys, unique constraints, cascading deletes and why a stable curriculum key keeps progress safe when lessons change. We understood spaced repetition (Leitner boxes), the idea of a security design with several layers, and how to describe flows in numbered steps. We learned to keep design colours in tokens so two themes can share one code base.

### Challenges Faced (Max 100 words):

Designing 23 tables and their relations without duplication was difficult. Deciding what must be deleted when a user deletes an account (and what must stay, such as an anonymous audit entry) needed several discussions. Designing the unlocking rules so that old progress is not lost, and designing the grading flow so that the answers never reach the browser, were the hardest logic problems. Drawing clear diagrams that fit on a page also took time.

### Mentor Feedback and Suggestions (Max 100 words):

[To be filled in by the guide]

Signature of the Internal Guide

## 5.1 Coding

This section shows the structure of the code and selected excerpts of the main modules. The complete source code is available in the project repository.

### 5.1.0 Project structure

```
codeingo/
|-- backend/                                FastAPI application (Python)
|   |-- app/
|   |   |-- main.py                        application factory, middleware, routers
|   |   |-- config.py, db.py              settings (validated in production), database engine
|   |   |-- models.py                    23 SQLAlchemy tables
|   |   |-- routers/                     auth, learn, checkpoints, practice, contests,
|   |   |                               social, passkeys, account, stats, ai, admin
|   |   |-- services/                   grading, progress, review, contests, social,
|   |   |                               certificates, reminders, mailer, gamification
|   |   |-- security/                   tokens, mfa, passkeys, ratelimit, middleware
|   |   |-- curriculum/                 10 courses + 30 projects (Python data)
|   |-- scripts/validate_curriculum.py    runs every reference solution
|   |-- tests/                          automated tests (pytest)
|-- frontend/                             Next.js application
|   |-- app/                             routes (App Router) + manifest
|   |-- views/                           page components (Learn, Lesson, Practice, ...)
|   |-- components/                      shared UI (AppShell, Exercise, QuizRunner, ...)
|   |-- lib/                             api client, auth, theme, PWA, WebAuthn, runners
|   |-- public/runners/                  sandboxed workers: Python, JS, TS, SQL, Git
|   |-- proxy.js                         per-request Content-Security-Policy
|   |-- e2e/                             browser tests (Playwright)
|-- deploy/                              nginx, Caddy, Vercel/Render guide
|-- docker-compose.yml, render.yaml
|-- README.md, SECURITY.md
```

**Table 5.1.1: Size of the code base (lines)**

| Part                                      | Lines | Language             |
|-------------------------------------------|-------|----------------------|
| Back end application (without curriculum) | 4641  | Python               |
| Curriculum (10 courses, 30 projects)      | 8545  | Python data          |
| Back end tests                            | 1670  | Python (pytest)      |
| Front end (pages, components, libraries)  | 6390  | JavaScript / JSX     |
| In-browser code runners                   | 1157  | JavaScript (modules) |

#### 5.1.1 Application factory - backend/app/main.py

```
task.cancel()
with contextlib.suppress(asyncio.CancelledError):
    await task

def create_app() -> FastAPI:
    s = get_settings()
    app = FastAPI(
        title="Codeingo API",
        lifespan=lifespan,
        # Don't publish the API map in production.
        docs_url=None if s.is_production else "/api/docs",
```

```

        redoc_url=None,
        openapi_url=None if s.is_production else "/api/openapi.json",
    )

    # Starlette runs the LAST added middleware first (outermost).
    app.add_middleware(CSRFMiddleware)
    app.add_middleware(BodySizeLimitMiddleware)
    app.add_middleware(GlobalRateLimitMiddleware)
    app.add_middleware(TrustedHostMiddleware, allowed_hosts=s.allowed_hosts)
    app.add_middleware(SecurityHeadersMiddleware)
    # No CORSMiddleware on purpose: the SPA is served from the same origin, so
    browsers
    # block every cross-origin read of the API by default.

    for r in (account.router, auth.router, passkeys.router, learn.router,
              checkpoints.router, contests.router, practice.router, social.router, stats.router,
              ai.router, admin.router):
        app.include_router(r)

    @app.get("/api/health")
    def health():
        return {"status": "ok"}

    @app.exception_handler(RequestValidationError)
    async def validation_error(_: Request, exc: RequestValidationError):
        # Report which fields failed without echoing the submitted values back.
        errors = [{"loc": e.get("loc"), "msg": e.get("msg")} for e in exc.errors()]
[:10]
        return JSONResponse({"detail": "Invalid request", "errors": errors},
                             status_code=422)

    @app.exception_handler(Exception)
    async def unhandled(_: Request, exc: Exception):
        log.exception("unhandled error", exc_info=exc)
        return JSONResponse({"detail": "Internal server error"}, status_code=500) #
    never leak stack traces

```

### 5.1.2 E-mail 2-step code: issue and verify - backend/app/security/mfa.py

```

def issue_email_code(user: User, *, resend: bool = False) -> str | None:
    """Creates a fresh 6-digit code, or returns None when the code already sent should
    be used:
    - without `resend` (e.g. the sign-in page loading or reloading) whenever a code is
    still valid,
    so a reload never silently replaces the code the learner is about to type;
    - with `resend` (the learner asked for a new code) only during the short cooldown.
    Raises EmailCodeThrottled when the hourly limit is reached."""
    s = get_settings()
    now = datetime.now(timezone.utc)
    if email_code_resend_in(user) > 0 or (not resend and _live_email_code(user)):
        return None
    window = aware(user.email_code_window_start)
    if window is None or now - window >= timedelta(hours=1):
        user.email_code_window_start, user.email_code_window_count = now, 0
    if user.email_code_window_count >= s.email_code_max_per_hour:
        raise EmailCodeThrottled(int((aware(user.email_code_window_start) +
timedelta(hours=1) - now).total_seconds()) + 1)
    code = f"{secrets.randbelow(1_000_000):06d}"
    user.email_code_hash = _email_code_hash(user, code)

```

```

        user.email_code_expires_at = now + timedelta(minutes=s.email_code_minutes)
        user.email_code_sent_at = now
        user.email_code_attempts = 0
        user.email_code_window_count += 1
        return code

def verify_email_code(user: User, code: str) -> bool:
    """Single use, expires, and only a few guesses per code."""
    code = _clean(code)
    if not _live_email_code(user):
        return False
    user.email_code_attempts += 1
    ok = len(code) == 6 and code.isdigit() and
    hmac.compare_digest(user.email_code_hash, _email_code_hash(user, code))
    if ok or user.email_code_attempts >= get_settings().email_code_max_attempts:
        user.email_code_hash = user.email_code_expires_at = None # used up (or too
many guesses)
    return ok

```

### 5.1.3 Server-side grading - backend/app/services/grading.py

```

def grade(ex: Exercise, answer) -> bool:
    sol = ex.solution or {}
    try:
        if ex.kind == "mcq":
            return isinstance(answer, int) and not isinstance(answer, bool) and answer
== sol.get("index")
        if ex.kind == "fill":
            return isinstance(answer, str) and len(answer) <= 200 and _norm(answer) in
{_norm(a) for a in sol.get("accepted", [])}
        if ex.kind == "order":
            n = len(ex.data.get("lines", []))
            if not isinstance(answer, list) or len(answer) != n or not
all(isinstance(i, int) for i in answer):
                return False
            return answer in sol.get("alternatives", [list(range(n))])
        if ex.kind == "code":
            if not isinstance(answer, str) or len(answer) > 2000:
                return False
            return _patterns_ok(answer, sol.get("patterns", []), sol.get("forbid",
[]))
        if ex.kind == "run":
            return _grade_run(ex, answer)
    except TimeoutError:
        return False
    return False

```

### 5.1.4 Spaced repetition (Leitner boxes) - backend/app/services/review.py

```

def record(db: Session, user: User, ex: Exercise, correct: bool, *, practice: bool =
False) -> None:
    """Update the schedule after any answer. Program-writing exercises aren't
reviewed."""
    if ex.kind not in QUIZ_KINDS:
        return
    item = _item(db, user, ex)
    now = _now()
    if item is None:

```

```

        item = ReviewItem(user_id=user.id, exercise_id=ex.id, box=0, lapses=0,
reviews=0)
        db.add(item)
        if correct: # first meeting, got it right: check back tomorrow
            item.box, item.due_at, item.updated_at = 1, now +
timedelta(days=INTERVAL_DAYS[1]), now
            return
        if practice:
            item.reviews += 1
        if not correct:
            item.box, item.due_at = 0, now
            item.lapses += 1
        elif practice:
            item.box = min(MAX_BOX, item.box + 1)
            item.due_at = now + timedelta(days=INTERVAL_DAYS[item.box])
        # a correct retry inside a lesson doesn't count as "learned" - practice decides
that
        item.updated_at = now

```

### 5.1.5 Passkey sign-in verification - backend/app/security/passkeys.py

```

def authenticate(db: Session, request: Request, response: Response, credential: dict)
-> User:
    rp_id, origin = rp()
    challenge = _take_challenge(db, request, response, "login", None)
    fail = HTTPException(400, "That passkey isn't registered here. Sign in another
way, then add it in your profile.")
    raw_id = credential.get("rawId") or credential.get("id")
    if not isinstance(raw_id, str) or len(raw_id) > 1400:
        raise fail
    pk = db.scalar(select(Passkey).where(Passkey.credential_id == raw_id.rstrip("=")))
    if pk is None:
        raise fail
    user = db.get(User, pk.user_id)
    if user is None or not user.is_active:
        raise HTTPException(403, "This account is disabled.")
    handle = (credential.get("response") or {}).get("userHandle")
    if handle and handle.rstrip("=") != bytes_to_base64url(user_handle(user)):
        raise fail
    try:
        v = verify_authentication_response(
            credential=credential, expected_challenge=challenge, expected_rp_id=rp_id,
expected_origin=origin,
            credential_public_key=base64url_to_bytes(pk.public_key),
credential_current_sign_count=pk.sign_count,
            require_user_verification=True,
        )
    except (InvalidAuthenticationResponse, ValueError, KeyError, TypeError) as exc:
        raise HTTPException(400, "That passkey couldn't be verified.") from exc
    pk.sign_count = v.new_sign_count
    pk.last_used_at = datetime.now(timezone.utc)
    return user

```

### 5.1.6 Contest scoring and ranking - backend/app/services/contests.py

```

def finish(db: Session, entry: ContestEntry, contest: Contest, user: User, now:
datetime | None = None) -> list[str]:
    """Close a run (on request or when its time is up) and award XP once. Returns new

```

```

badges."""
    if entry.finished_at is not None:
        return []
    now = now or datetime.now(timezone.utc)
    end = min(now, deadline(entry, contest))
    entry.finished_at = end
    entry.time_ms = int((end - gamification.aware(entry.started_at)).total_seconds() *
1000)
    entry.score = sum(1 for v in entry.results.values() if v)
    entry.xp_awarded = entry.score * XP_PER_CORRECT + (PERFECT_BONUS if entry.score ==
len(contest.exercise_ids) else 0)
    gamification.award_xp(db, user, entry.xp_awarded, "contest")
    earned: list[str] = []
    gamification.grant_badge(db, user, "contender", earned)
    return earned

def ranking(db: Session, contest: Contest) -> list[tuple[ContestEntry, User]]:
    """Finished runs first (score desc, then time asc), then runs still in progress
(live score)."""
    rows = db.execute(
        select(ContestEntry, User).join(User, User.id ==
ContestEntry.user_id).where(ContestEntry.contest_id == contest.id,
User.is_active.is_(True))
    ).all()

    def key(row):
        e = row[0]
        live = sum(1 for v in e.results.values() if v)
        return (e.finished_at is None, -(e.score if e.finished_at else live),
e.time_ms if e.finished_at else 0, e.id)

    return sorted(rows, key=key)

```

### 5.1.7 TypeScript type-checking in the browser - frontend/public/runners/ts-shared.mjs

```

export function compileTs(ts, libs, source) {
    const files = { ...libs, "runner.d.ts": RUNNER_DTS };
    let js = "";
    const strip = (n) => n.replace(/^\/+/, "");
    const host = {
        getSourceFile(name, languageVersion) {
            name = strip(name);
            if (name === FILE) return ts.createSourceFile(FILE, source, languageVersion,
true);
            if (!(name in files)) return undefined;
            if (!cache.has(name)) cache.set(name, ts.createSourceFile(name, files[name],
languageVersion, true));
            return cache.get(name);
        },
        writeFile(name, text) {
            if (name.endsWith(".js")) js = text;
        },
        getDefaultLibFileName: () => ROOT_LIB,
        useCaseSensitiveFileNames: () => true,
        getCanonicalFileName: (n) => n,
        getCurrentDirectory: () => "",
        getNewLine: () => "\n",
        fileExists: (n) => strip(n) === FILE || strip(n) in files,
        readFile: (n) => (strip(n) === FILE ? source : files[strip(n)]),
        directoryExists: () => true,
    };

```

```

    getDirectories: () => [],
  };
  const program = ts.createProgram([FILE, ...Object.keys(files)], options(ts), host);
  const main = program.getSourceFile(FILE);
  const diags =
  [...program.getSyntacticDiagnostics(main), ...program.getSemanticDiagnostics(main)];
  if (diags.length) {
    return {
      errors: diags.slice(0, 5).map((d) => {
        const msg = ts.flattenDiagnosticMessageText(d.messageText, "\n");
        if (d.file && d.start !== undefined) return `Line $
[d.file.getLineAndCharacterOfPosition(d.start).line + 1]: ${msg}`;
        return msg;
      }),
    };
  }
  program.emit(main);
  // a file with import/export would emit module syntax the runner can't execute
  if (/^\s*(import|export)\s/m.test(js)) return { errors: ["import/export aren't
available here - keep everything in one file."] };
  return { js };
}

```

### 5.1.8 Per-request Content-Security-Policy - frontend/proxy.js

```

export function proxy(request) {
  const nonce = Buffer.from(crypto.randomUUID()).toString("base64");
  const isDev = process.env.NODE_ENV === "development";

  const csp = [
    "default-src 'self'",
    `script-src 'self' 'nonce-${nonce}' 'strict-dynamic'${isDev ? " 'unsafe-eval'" :
    ""}`,
    // Inline style attributes are needed by charts/animations; styles can't execute
    // code.
    "style-src 'self' 'unsafe-inline'",
    "img-src 'self' data: blob: https://lh3.googleusercontent.com",
    "font-src 'self'",
    `connect-src 'self'${isDev ? " ws:" : ""}`,
    // code runners (see lib/runners.js) - each worker script carries its own stricter
    CSP
    "worker-src 'self'",
    // HTML/CSS exercises: sandboxed, script-less preview frames built from srcdoc
    "frame-src 'self'",
    "object-src 'none'",
    "base-uri 'none'",
    "form-action 'self' https://accounts.google.com",
    "frame-ancestors 'none'",
    ...(isDev ? [] : ["upgrade-insecure-requests"]),
  ].join("; ");

  const requestHeaders = new Headers(request.headers);
  requestHeaders.set("x-nonce", nonce);
  requestHeaders.set("Content-Security-Policy", csp);

  const response = NextResponse.next({ request: { headers: requestHeaders } });
  response.headers.set("Content-Security-Policy", csp);
  if (!isDev) response.headers.set("Strict-Transport-Security", "max-age=63072000;
includeSubDomains; preload");
  return response;
}

```

```
}
```

## 5.2 Testing

Testing combines four layers: automated back-end tests (pytest) that call the real API with an in-process client, a validation script that executes the reference solution of every exercise, manual and automated browser checks of the main flows, and security-oriented tests (CSRF, token forgery, rate limits, replay, hidden admin area). The whole back-end suite is run on SQLite and on PostgreSQL 16.

**Table 5.2.1: Automated back-end tests by file**

| Test file                     | Tests | What it covers                                     |
|-------------------------------|-------|----------------------------------------------------|
| test_security.py              | 16    | Session, CSRF, tokens, headers, admin, rate limits |
| test_learning.py              | 15    | Courses, lessons, answers, hearts, daily challenge |
| test_email_codes.py           | 14    | E-mail 2-step codes and mailer                     |
| test_social.py                | 11    | Friends, leagues, reminders, unsubscribe           |
| test_practice.py              | 9     | Spaced repetition and practice sessions            |
| test_passkeys.py              | 8     | WebAuthn passkeys                                  |
| test_checkpoints.py           | 6     | Chapter and readiness tests                        |
| test_contests.py              | 6     | Weekly contests                                    |
| test_projects_certificates.py | 4     | Projects and certificates                          |
| test_account.py               | 3     | Data export and deletion                           |
| test_admin_roles.py           | 3     | Admin role management                              |
| Total                         | 95    | All pass on SQLite and on PostgreSQL 16            |

### 5.2.1 Test Cases

The table on the following pages lists the main test cases. Every row corresponds to one or more automated tests in the repository that were executed, and all of them passed.

**Table 5.2.2: Test cases**

| Test ID | Test Cases            | Test Description                                            | Input                                                              | Expected Output                                               | Actual Output                                                 | Result |
|---------|-----------------------|-------------------------------------------------------------|--------------------------------------------------------------------|---------------------------------------------------------------|---------------------------------------------------------------|--------|
| T01     | Email Code Testing #1 | A correct emailed 6-digit code signs the user in.           | Code from the e-mail                                               | Sign-in completes; session cookies are set.                   | Sign-in completes; session cookies are set.                   | Pass   |
| T02     | Email Code Testing #2 | A wrong code is refused.                                    | Code: 000000                                                       | Error "Invalid verification code"; no session.                | Error "Invalid verification code"; no session.                | Pass   |
| T03     | Email Code Testing #3 | An expired code is refused.                                 | Code used after 10 minutes                                         | Rejected; the user is asked to request a new code.            | Rejected; the user is asked to request a new code.            | Pass   |
| T04     | Email Code Testing #4 | A code can be used only once.                               | The same correct code twice                                        | Second use is rejected.                                       | Second use is rejected.                                       | Pass   |
| T05     | Email Code Testing #5 | Lock-out after repeated wrong guesses.                      | Five wrong codes in a row                                          | Further attempts are refused (HTTP 429) for a period.         | Further attempts are refused (HTTP 429) for a period.         | Pass   |
| T06     | Email Code Testing #6 | Resend cool-down and hourly cap.                            | Request codes quickly and more than 6 per hour                     | Cool-down message; hourly limit stops further sends.          | Cool-down message; hourly limit stops further sends.          | Pass   |
| T07     | Email Code Testing #7 | Reloading the page keeps the code the user is typing valid. | Open the code page, reload, enter the first code                   | The first code still works; only "send new code" replaces it. | The first code still works; only "send new code" replaces it. | Pass   |
| T08     | Email Code Testing #8 | E-mail header injection is impossible.                      | Address containing a line break                                    | Message is rejected; no extra headers are sent.               | Message is rejected; no extra headers are sent.               | Pass   |
| T09     | Email Code Testing #9 | Production refuses to start without SMTP.                   | ENV=production without SMTP settings                               | Start-up error naming the missing setting.                    | Start-up error naming the missing setting.                    | Pass   |
| T10     | Session Security #1   | Protected data needs a session.                             | GET /api/courses without sign-in                                   | HTTP 401 Unauthorized.                                        | HTTP 401 Unauthorized.                                        | Pass   |
| T11     | Session Security #2   | CSRF token is required for changes.                         | POST without the X-CSRF-Token header                               | HTTP 403; nothing is changed.                                 | HTTP 403; nothing is changed.                                 | Pass   |
| T12     | Session Security #3   | Forged and wrong-type tokens are rejected.                  | Token signed with another key / refresh token used as access token | HTTP 401.                                                     | HTTP 401.                                                     | Pass   |
| T13     | Session Security #4   | Sign-in endpoints are rate-limited.                         | 25 sign-in requests in a minute                                    | HTTP 429 appears.                                             | HTTP 429 appears.                                             | Pass   |
| T14     | Passkey Testing #1    | A passkey can be added and used to sign in without a code.  | Software authenticator (real P-256 signatures)                     | Registered; sign-in returns the same account.                 | Registered; sign-in returns the same account.                 | Pass   |

| Test ID | Test Cases             | Test Description                                                | Input                                                            | Expected Output                                                    | Actual Output                                                      | Result |
|---------|------------------------|-----------------------------------------------------------------|------------------------------------------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------|--------|
| T15     | Passkey Testing #2     | A captured sign-in response cannot be replayed.                 | Send the same signed response twice                              | Second attempt fails (challenge already used).                     | Second attempt fails (challenge already used).                     | Pass   |
| T16     | Passkey Testing #3     | A phishing site's origin is rejected.                           | Assertion with origin https://evil.example                       | HTTP 400; not signed in.                                           | HTTP 400; not signed in.                                           | Pass   |
| T17     | Passkey Testing #4     | User verification is mandatory.                                 | Assertion without the user-verified flag                         | Rejected.                                                          | Rejected.                                                          | Pass   |
| T18     | Passkey Testing #5     | A wrong private key is rejected.                                | Signature made by another key                                    | Rejected.                                                          | Rejected.                                                          | Pass   |
| T19     | Passkey Testing #6     | Unknown credentials and disabled accounts are refused.          | Unregistered authenticator / disabled user                       | HTTP 400 / HTTP 403.                                               | HTTP 400 / HTTP 403.                                               | Pass   |
| T20     | Learning Testing #1    | Lessons unlock in order and the path shows sections and tests.  | GET course path as a new learner                                 | Only the first lesson is unlocked; chapter tests are locked.       | Only the first lesson is unlocked; chapter tests are locked.       | Pass   |
| T21     | Learning Testing #2    | Answers are graded on the server and not revealed in advance.   | Request lesson questions                                         | No solution fields in the response.                                | No solution fields in the response.                                | Pass   |
| T22     | Learning Testing #3    | Finishing a lesson too fast is refused.                         | Complete a lesson instantly                                      | HTTP 400 "suspiciously fast".                                      | HTTP 400 "suspiciously fast".                                      | Pass   |
| T23     | Learning Testing #4    | A lesson attempt belongs to its owner.                          | Another user completes the attempt                               | HTTP 404.                                                          | HTTP 404.                                                          | Pass   |
| T24     | Test Testing #1        | A chapter test needs 6 of 8 to pass and unlocks the next unit.  | Answer 8 correctly / all wrong                                   | Passed, +20 XP, next unit open / failed, retry with new questions. | Passed, +20 XP, next unit open / failed, retry with new questions. | Pass   |
| T25     | Test Testing #2        | A readiness test lets the learner jump ahead.                   | Pass the 15-question test (12 to pass)                           | Earlier lessons marked tested out; next section unlocked.          | Earlier lessons marked tested out; next section unlocked.          | Pass   |
| T26     | Test Testing #3        | AI hints are blocked during an open test or contest.            | Ask for a hint for a test question                               | HTTP 403; after the test, hints work again.                        | HTTP 403; after the test, hints work again.                        | Pass   |
| T27     | Practice Testing #1    | Missed questions enter the practice list; practice moves boxes. | Miss a question, then answer it correctly in practice            | Item moves up a Leitner box; due date grows.                       | Item moves up a Leitner box; due date grows.                       | Pass   |
| T28     | Practice Testing #2    | Practice reward limits.                                         | Finish 6 practice sessions in a day                              | +10 XP and a heart for the first five only.                        | +10 XP and a heart for the first five only.                        | Pass   |
| T29     | Project Testing #1     | Every section ends with a three-step project.                   | Inspect all lessons flagged as projects                          | 30 projects, 3 run steps each, steps continue the learner's code.  | 30 projects, 3 run steps each, steps continue the learner's code.  | Pass   |
| T30     | Certificate Testing #1 | A certificate is issued only when everything is complete.       | Complete all lessons but not the last chapter test; then pass it | No certificate first; then one certificate (issued once).          | No certificate first; then one certificate (issued once).          | Pass   |

| Test ID | Test Cases             | Test Description                                          | Input                                                                  | Expected Output                                              | Actual Output                                                | Result |
|---------|------------------------|-----------------------------------------------------------|------------------------------------------------------------------------|--------------------------------------------------------------|--------------------------------------------------------------|--------|
| T31     | Certificate Testing #2 | Public verification.                                      | Open<br>/api/public/certificates/<code><br>signed out; try a fake code | Certificate data returned; fake code gives 404.              | Certificate data returned; fake code gives 404.              | Pass   |
| T32     | Social Testing #1      | Follow a friend by friend code (case and dashes ignored). | Code "abcd-2345"                                                       | Friend added; Friendly badge awarded once.                   | Friend added; Friendly badge awarded once.                   | Pass   |
| T33     | Social Testing #2      | Invalid follow requests.                                  | Own code / unknown code / disabled account / bad characters            | HTTP 400 / 404 / 404 / 422.                                  | HTTP 400 / 404 / 404 / 422.                                  | Pass   |
| T34     | Social Testing #3      | Leagues never expose e-mail addresses.                    | Read global and friends leagues                                        | Only name, picture, code, streak and XP.                     | Only name, picture, code, streak and XP.                     | Pass   |
| T35     | Reminder Testing #1    | One streak reminder per evening.                          | Run the job twice on the evening the streak would end                  | One e-mail; second run sends nothing.                        | One e-mail; second run sends nothing.                        | Pass   |
| T36     | Reminder Testing #2    | No reminder when it is not needed.                        | Already active today / streak lost / opted out / disabled              | No e-mail is sent.                                           | No e-mail is sent.                                           | Pass   |
| T37     | Reminder Testing #3    | A failed send is retried; unsubscribe is signed.          | Mail server down, then up; tampered link                               | Retried later the same day; tampered link HTTP 400.          | Retried later the same day; tampered link HTTP 400.          | Pass   |
| T38     | Contest Testing #1     | Everyone gets the same ten questions.                     | Two users enter the same contest                                       | Identical question list; resume keeps the same run.          | Identical question list; resume keeps the same run.          | Pass   |
| T39     | Contest Testing #2     | Score and ranking; no answers revealed live.              | Finish with 10/10 and with 1/10                                        | Ranked by score then time; no correct answer in responses.   | Ranked by score then time; no correct answer in responses.   | Pass   |
| T40     | Contest Testing #3     | Time limit and anti-automation.                           | Answer instantly / after 11 minutes                                    | HTTP 429 / HTTP 410 and the run is closed.                   | HTTP 429 / HTTP 410 and the run is closed.                   | Pass   |
| T41     | Admin Testing #1       | Admin area is hidden from learners.                       | Learner calls /api/admin/tree                                          | HTTP 404.                                                    | HTTP 404.                                                    | Pass   |
| T42     | Admin Testing #2       | Granting admin needs a valid 2-step code.                 | Wrong code, then the right code                                        | HTTP 400, then role changed; both audit-logged.              | HTTP 400, then role changed; both audit-logged.              | Pass   |
| T43     | Admin Testing #3       | Owners and self are protected.                            | Demote an owner; change own role; disable an owner                     | HTTP 403 / 400 / 403.                                        | HTTP 403 / 400 / 403.                                        | Pass   |
| T44     | Admin Testing #4       | Only finished accounts can become admins.                 | Promote an account without 2-step set up                               | HTTP 400.                                                    | HTTP 400.                                                    | Pass   |
| T45     | Privacy Testing #1     | Data export has no secrets.                               | GET /api/account/export                                                | JSON of own data; no tokens, hashes or other users' e-mails. | JSON of own data; no tokens, hashes or other users' e-mails. | Pass   |

| Test ID | Test Cases            | Test Description                                               | Input                                                 | Expected Output                                                              | Actual Output                                                                | Result |
|---------|-----------------------|----------------------------------------------------------------|-------------------------------------------------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------|--------|
| T46     | Privacy Testing #2    | Account deletion is complete.                                  | Delete with a valid 2-step code                       | User and all linked rows removed; anonymous audit entry kept; session ended. | User and all linked rows removed; anonymous audit entry kept; session ended. | Pass   |
| T47     | Privacy Testing #3    | Deletion needs the right code; owners cannot self-delete.      | Wrong code / owner account                            | HTTP 400 / HTTP 403; account stays.                                          | HTTP 400 / HTTP 403; account stays.                                          | Pass   |
| T48     | Database Testing #1   | The whole suite passes on PostgreSQL as well as SQLite.        | Run all tests with TEST_DATABASE_URL on PostgreSQL 16 | All tests pass on both databases.                                            | All tests pass on both databases.                                            | Pass   |
| T49     | Curriculum Testing #1 | Every reference solution is executed (492 runnable exercises). | scripts/validate_curriculum.py                        | All 438 lessons / 2130 exercises valid; starters do not pass.                | All 438 lessons / 2130 exercises valid; starters do not pass.                | Pass   |

### 5.2.2 Curriculum validation

Content quality is tested by `scripts/validate_curriculum.py`. For every exercise it checks the structure (options, a single blank, valid patterns, tests matching expected outputs) and that the exercise's own reference answer is accepted by the real grader. For each of the 492 program exercises it executes the reference solution with the real toolchain (python3, Node with the browser runners' own code, the TypeScript compiler, the Git simulator, sql.js, Chromium for HTML/CSS, javac, gcc and g++) and compares the output with the expected output, and it also checks that the empty starter code does not pass. The result for the final curriculum is: 10 courses, 438 lessons, 2130 exercises, 30 projects - all valid. The same script runs in the continuous-integration pipeline on every push.

**Table 5.2.3: Curriculum size verified by execution**

| Course                       | Units | Lessons | Projects |
|------------------------------|-------|---------|----------|
| Python                       | 16    | 51      | 3        |
| JavaScript                   | 16    | 51      | 3        |
| TypeScript                   | 8     | 27      | 3        |
| Java                         | 16    | 51      | 3        |
| C++                          | 16    | 51      | 3        |
| C                            | 16    | 51      | 3        |
| SQL                          | 16    | 51      | 3        |
| HTML & CSS                   | 16    | 51      | 3        |
| Git                          | 8     | 27      | 3        |
| Data Structures & Algorithms | 8     | 27      | 3        |
| Total                        | 136   | 438     | 30       |

### 5.2.3 Observations from browser testing

- The complete flow was driven in a real Chromium browser: sign-up with an e-mailed code, a lesson with a deliberate wrong answer, a chapter test, practice, daily challenge, leagues, a two-learner friend flow, a contest with a live board on a second device, certificate issue and public verification, hidden admin 404, role promotion with a confirmation code, account deletion.
- The first TypeScript run (which loads the compiler into the worker) took about 0.65 s; later runs took about 0.07 s.
- Type errors in TypeScript exercises are shown with line numbers and stop the run, as intended.
- The in-browser Git terminal showed the full session (commands and output) while only the check commands were graded.
- No horizontal scrolling appeared at a 390 px phone width on the pages tested, in both themes.

## 5.2.4 Defects found and fixed during testing

**Table 5.2.4: Notable defects found by testing and how they were fixed**

| No. | Defect                                                                                                                                        | Fix                                                                                                                                |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| 1   | A hydration warning appeared in the browser console (the mascot's blink delay used a random number on the server and another in the browser). | The delay is now derived from a stable id, so server and browser render the same value; verified by temporarily reverting the fix. |
| 2   | Reloading the e-mail code page replaced the code the learner was typing, so the correct-looking code was refused.                             | A page load now reuses a still-valid code; only an explicit "send a new code" replaces it.                                         |
| 3   | The resend cool-down stayed active after a code had been used, blocking the next sign-in.                                                     | The cool-down now applies only while a live code exists.                                                                           |
| 4   | Certificates were issued but not listed because SQLite returned time stamps without a zone.                                                   | Dates are normalised to UTC before comparison.                                                                                     |
| 5   | Automated sign-ins were blocked by the reverse proxy's rate limit during tests.                                                               | Test scripts pace their requests; the number of start-up requests was reduced.                                                     |
| 6   | The first TypeScript run timed out because the compiler was loaded before the message handler existed.                                        | The worker now attaches its handler first and waits for the compiler inside it.                                                    |
| 7   | SQLite ignored cascading deletes, so account deletion left rows behind in development.                                                        | Foreign-key enforcement is switched on for SQLite connections; the account tests pass on both databases.                           |

## GUIDE INTERACTION REPORT

*(To be maintained by the student and submitted as part of the Field Project documentation)*

|                                       |                        |
|---------------------------------------|------------------------|
| <b>Name of the Student</b>            | [STUDENT NAME]         |
| <b>Program /Semester</b>              | B.Sc I.T./ Semester V  |
| <b>Roll No/Seat No</b>                | [ROLL NO / SEAT NO]    |
| <b>Major Subject / Specialization</b> | Information Technology |
| <b>Field project Title</b>            | CODEINGO               |
| <b>Name of the Faculty mentor</b>     | [GUIDE NAME]           |

### Overview (Max 150 Words)

*(Brief summary of key activities, tasks and projects undertaken during the period)*

### CHAPTER 5: IMPLEMENTATION AND TESTING

This chapter shows the implementation: the project structure and selected code excerpts (application factory, e-mail code issue and verification, server-side grading, spaced repetition, passkey verification, contest ranking, in-browser TypeScript type checking, per-request content-security policy), and the testing: automated back-end tests on two databases, about fifty documented test cases with input, expected and actual output, execution of every reference solution, browser observations and a table of defects that were found and fixed.

### Learning Outcomes (Max 100 words):

We learned how a real project is organised in routers, services, models and views, and how to write code so that it can be tested. We learned test-case writing with ID, description, input, expected and actual output, and the value of security tests such as replay, wrong origin and token forgery. We learned that automated tests catch regressions when features are added, and that running the same tests on PostgreSQL finds differences that SQLite hides.

### Challenges Faced (Max 100 words):

Coding was the longest part. A hydration error caused by random values in the interface, the e-mail code that changed when the page reloaded, and the first TypeScript run that never started were difficult to find. Running the compiler and a Git simulator inside a locked-down worker was new for us. Writing tests for time (expiry, cool-downs, contest limits) needed fake clocks. Keeping all tests green after each change took discipline.

### Mentor Feedback and Suggestions (Max 100 words):

[To be filled in by the guide]

Signature of the Internal Guide

# CHAPTER 6

## RESULTS AND DISCUSSION

### 6.1 Screenshots of the system

This chapter shows the finished system. The screenshots were captured from the running application with demonstration data. Each screen is followed by a short discussion.

#### 6.1.1 User sign-in and entry

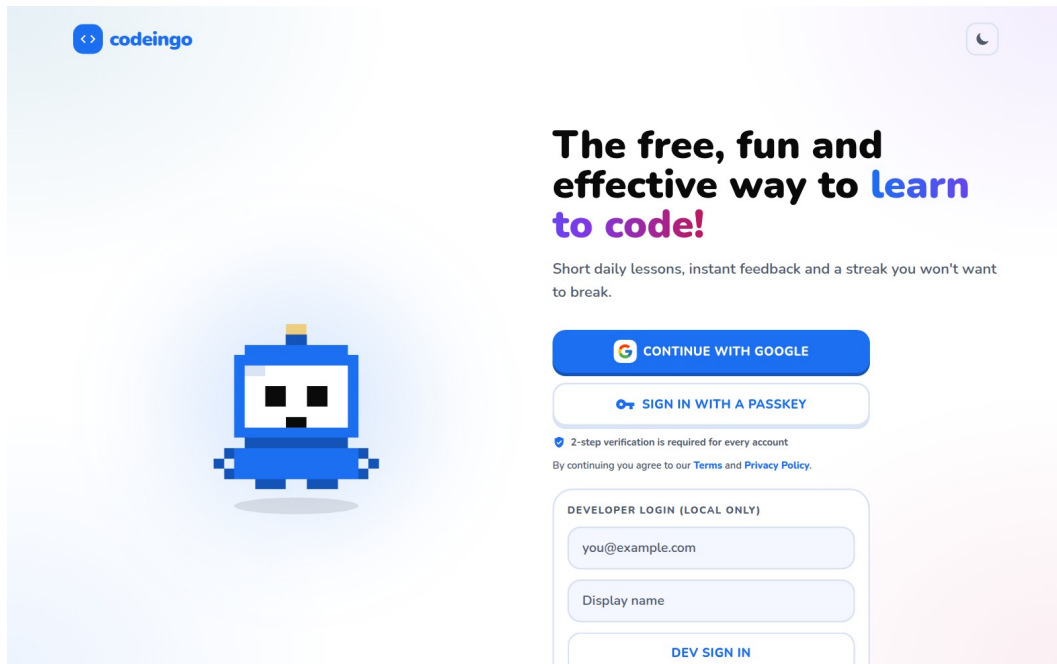


Figure 6.1: Landing page (light theme)

The first page offers Google sign-in, a passkey button and the consent line with links to the Privacy Policy and Terms. The gradient headline and blue accent follow the light theme.

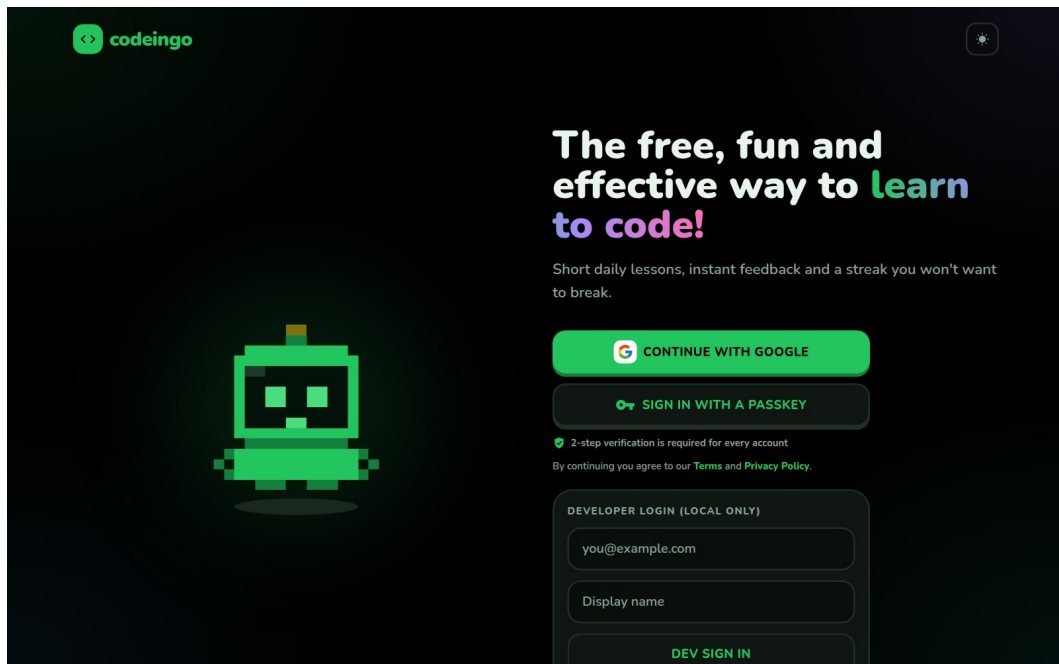


Figure 6.2: Landing page (dark theme)

The same page in the black/green dark theme. The theme can be switched at any time and is remembered.

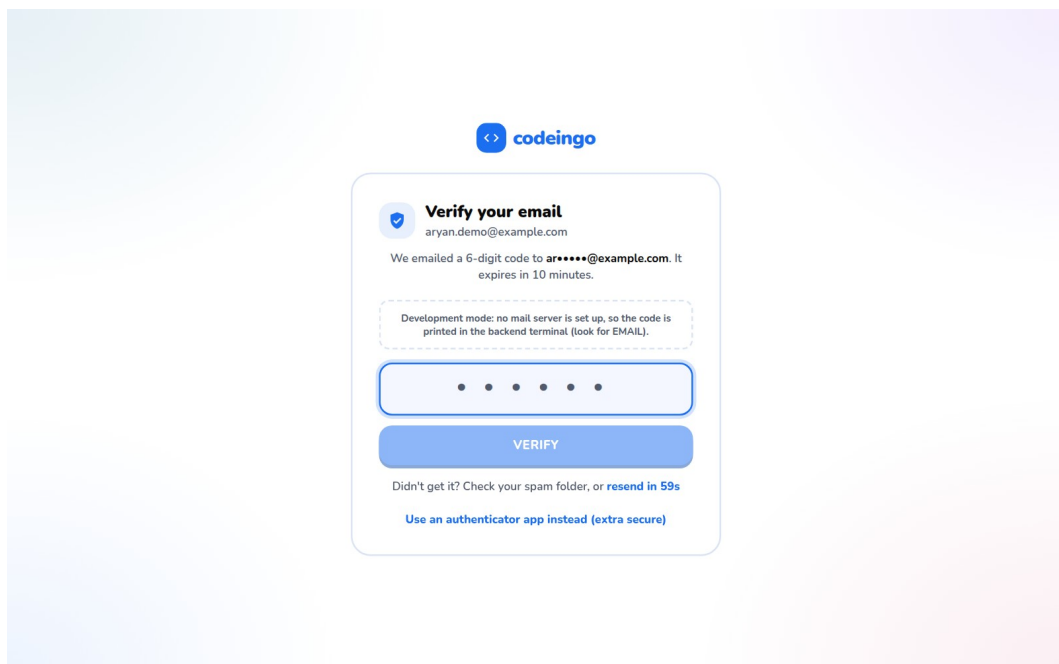


Figure 6.3: 2-step verification by e-mailed code

After Google sign-in the learner enters the 6-digit code sent to the e-mail address. Authenticator app and passkey are alternatives.

## 6.1.2 Learning

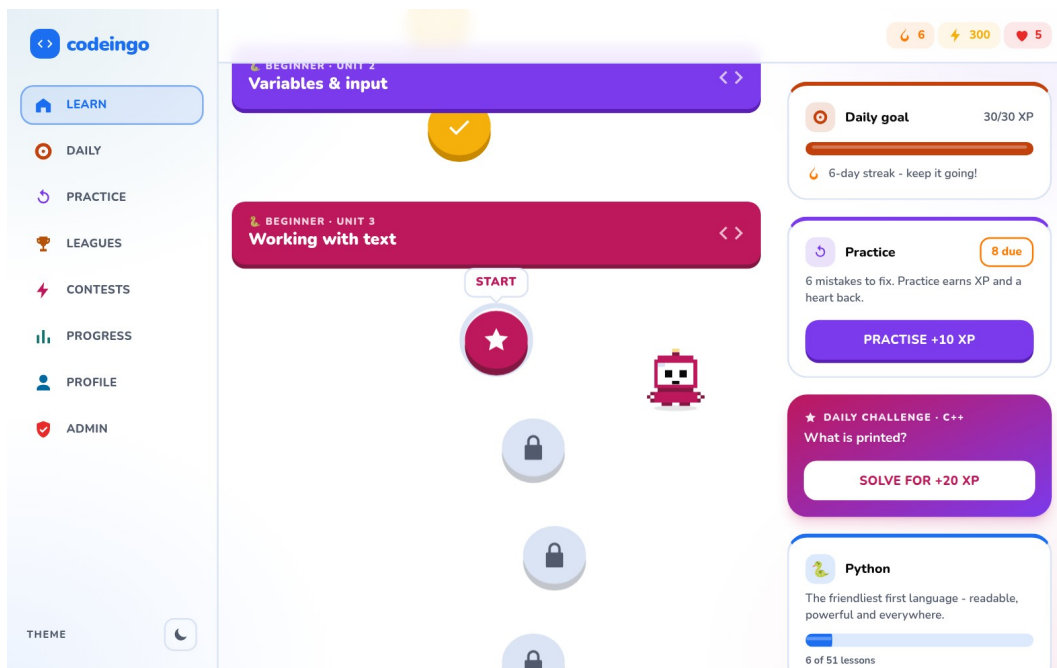


Figure 6.4: Course list (light)

All ten courses with colour-coded tiles and progress.

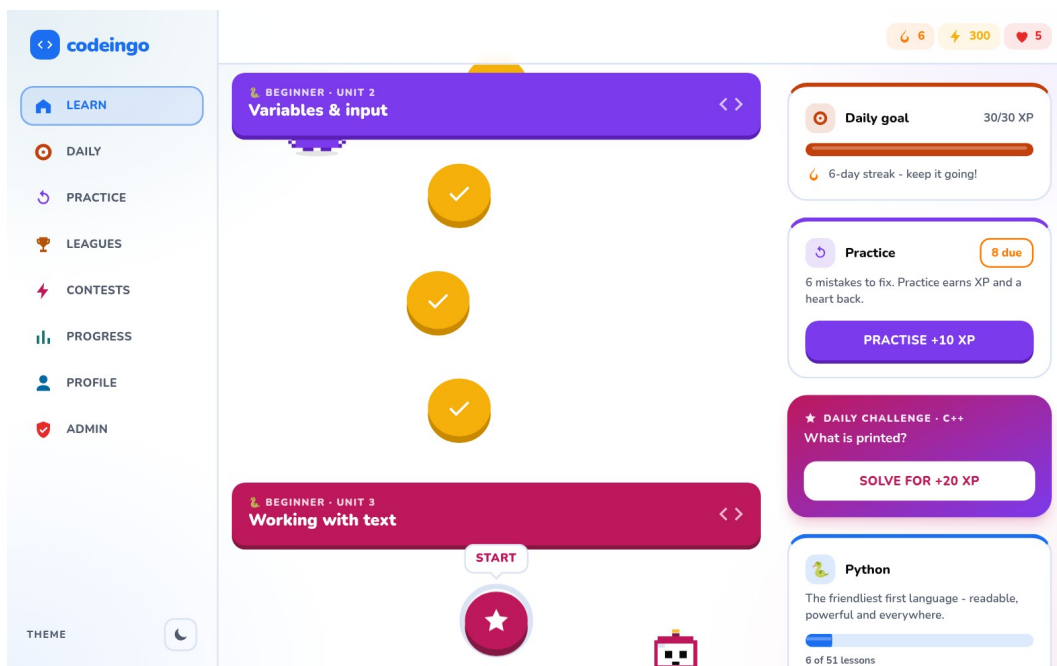


Figure 6.5: Learning path

Units and lessons in order; later lessons and chapter tests are locked until the learner reaches them.

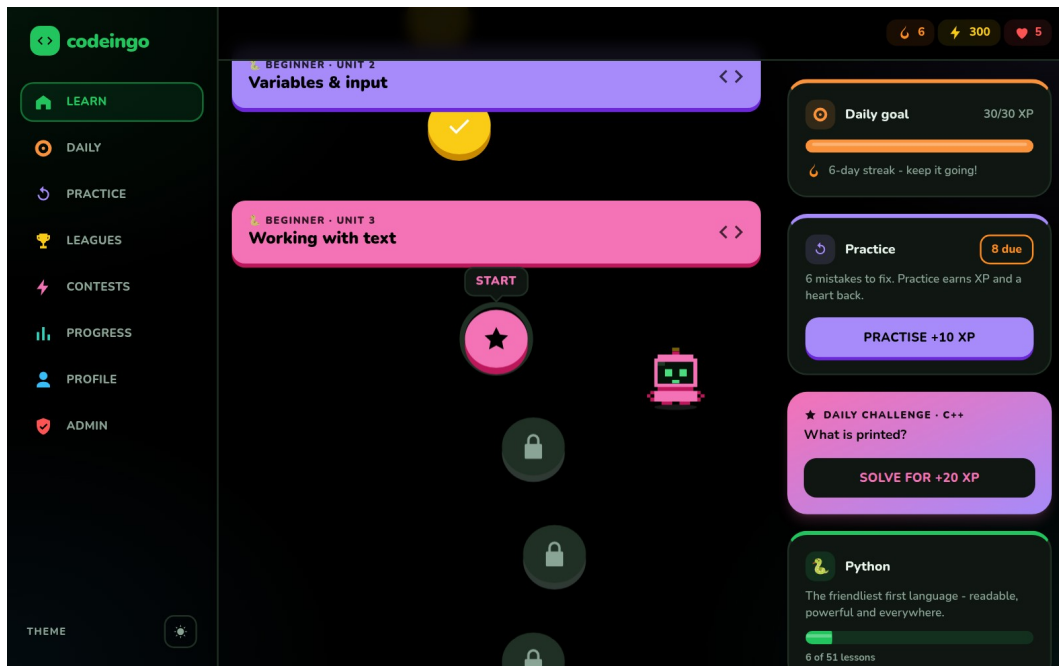


Figure 6.6: Learning path (dark)

The path in the dark theme with per-section colours.

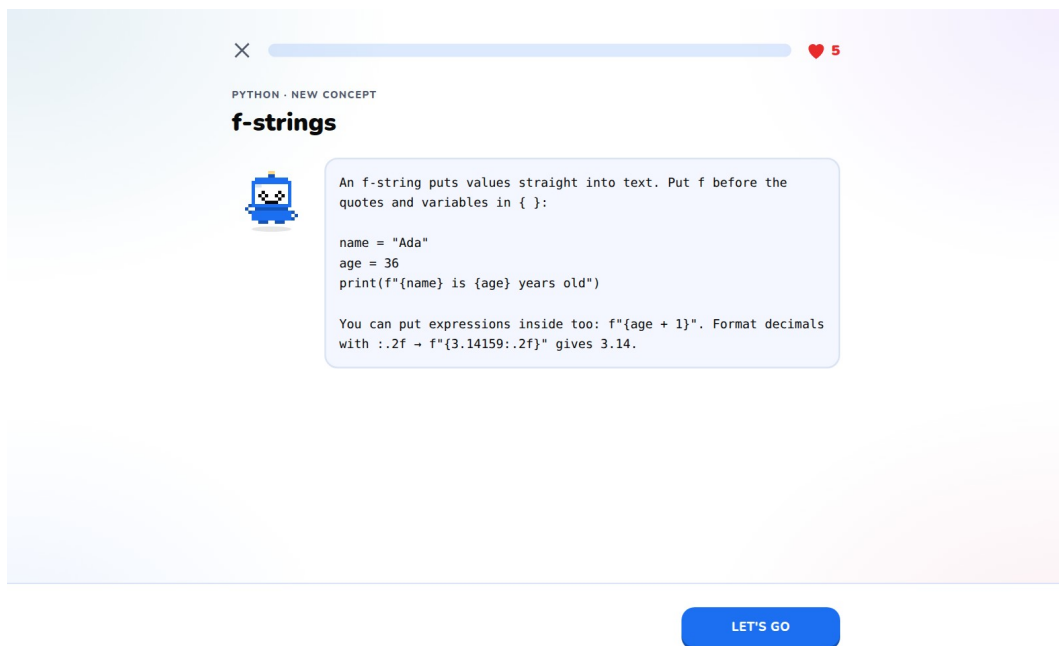


Figure 6.7: Lesson introduction

Each lesson begins with a short explanation and an example.

× ❤️ 5

SELECT THE CORRECT ANSWER

**What is printed?**

```
name = "Ada"
print(f"Hi {name}")
```

1 Hi Ada

2 Hi {name}

3 f"Hi Ada"

4 An error

💡 Stuck? Ask Codi for a hint

HINT

CHECK

Figure 6.8: Lesson question

Questions are multiple choice, fill-in-the-blank or code exercises.

× ❤️ 4

SELECT THE CORRECT ANSWER

**What is printed?**

```
name = "Ada"
print(f"Hi {name}")
```

1 Hi Ada

2 Hi {name}

3 f"Hi Ada"

4 An error

 **Not quite**  
Correct answer:  
Hi Ada  
The f prefix makes {name} be replaced.

CONTINUE

Figure 6.9: Wrong answer feedback

A wrong answer costs a heart and shows an explanation; an AI hint is available outside tests.



Figure 6.10: Running code in the browser

The learner's program runs in a sandbox in the browser and the output is compared with the expected result on the server.

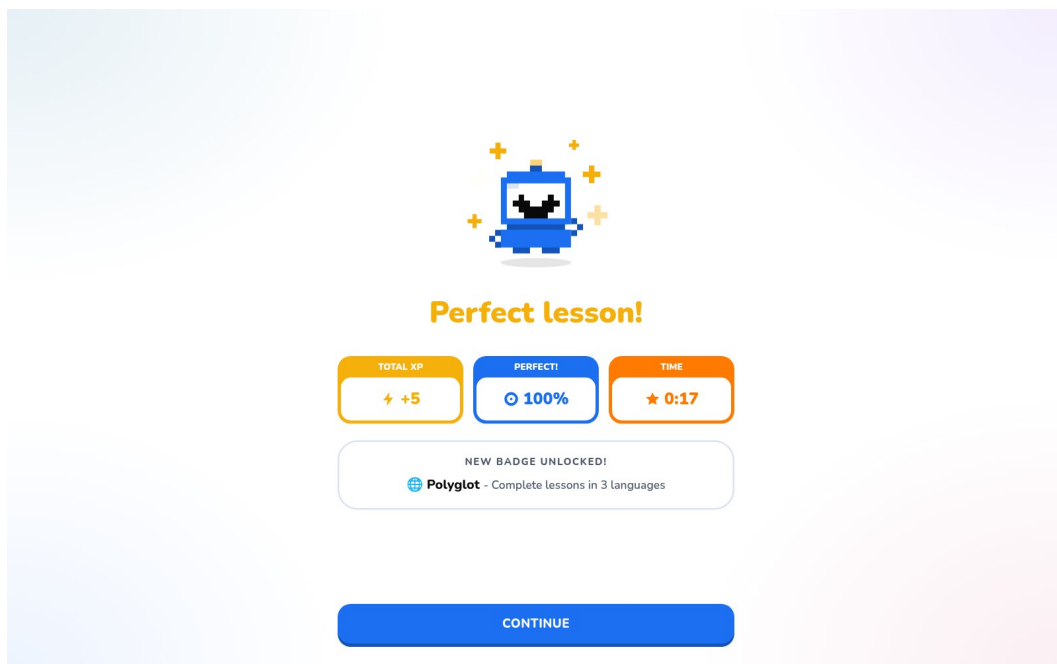


Figure 6.11: Lesson complete

XP, streak and badge rewards are shown at the end of a lesson.

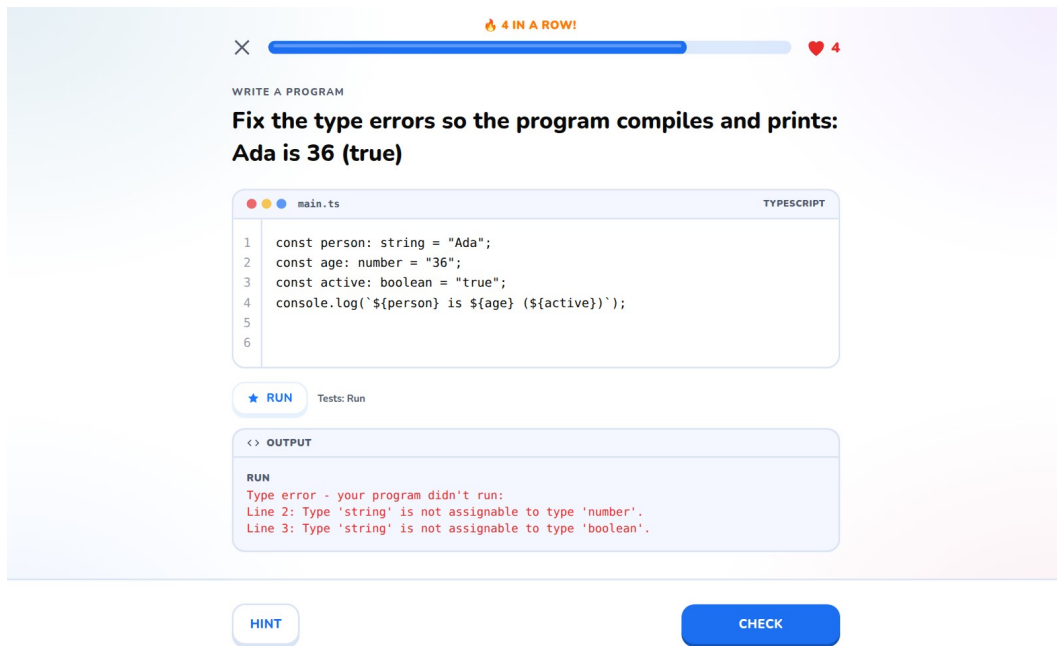


Figure 6.12: TypeScript type error

The TypeScript course type-checks the program with the real compiler and reports errors with line numbers.

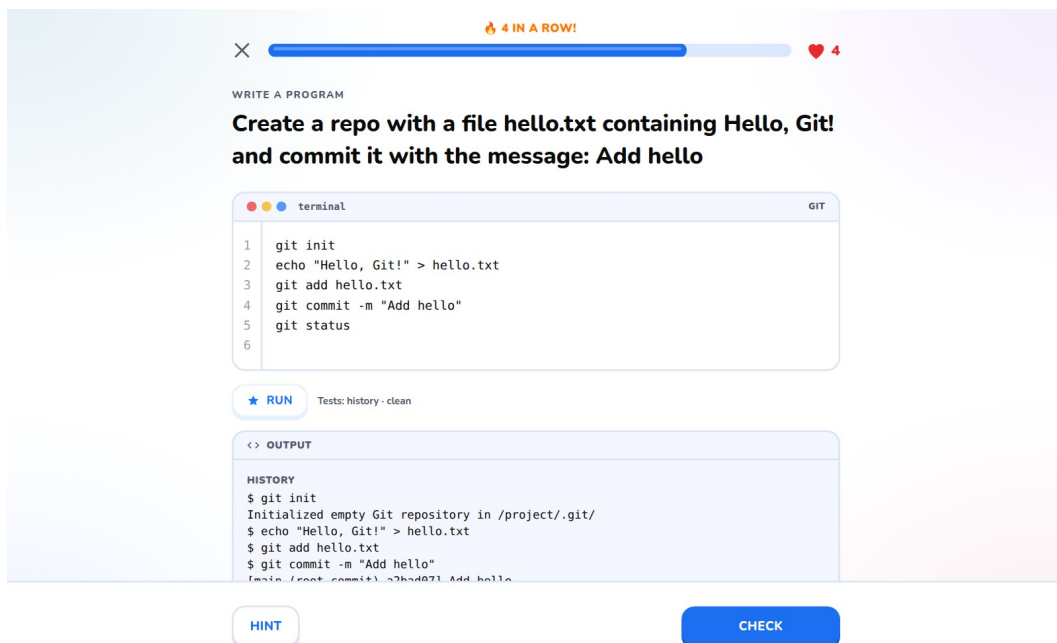


Figure 6.13: Git terminal

The Git course offers a terminal that simulates a repository in the browser.

### 6.1.3 Tests, practice and daily challenge

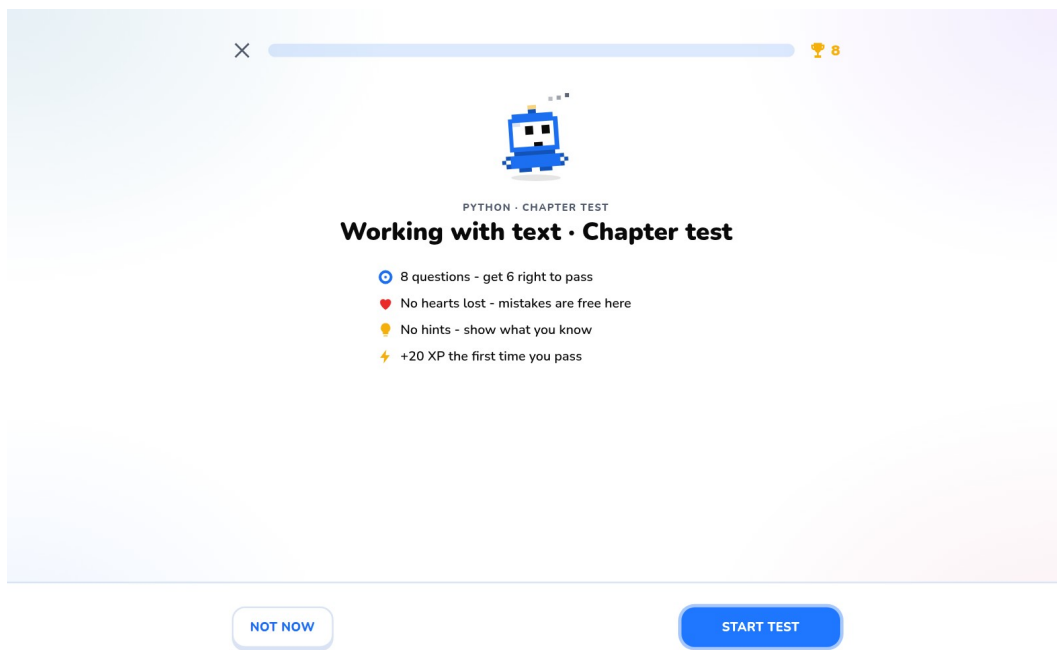


Figure 6.14: Chapter test introduction

A chapter test has eight questions; six correct answers unlock the next unit.

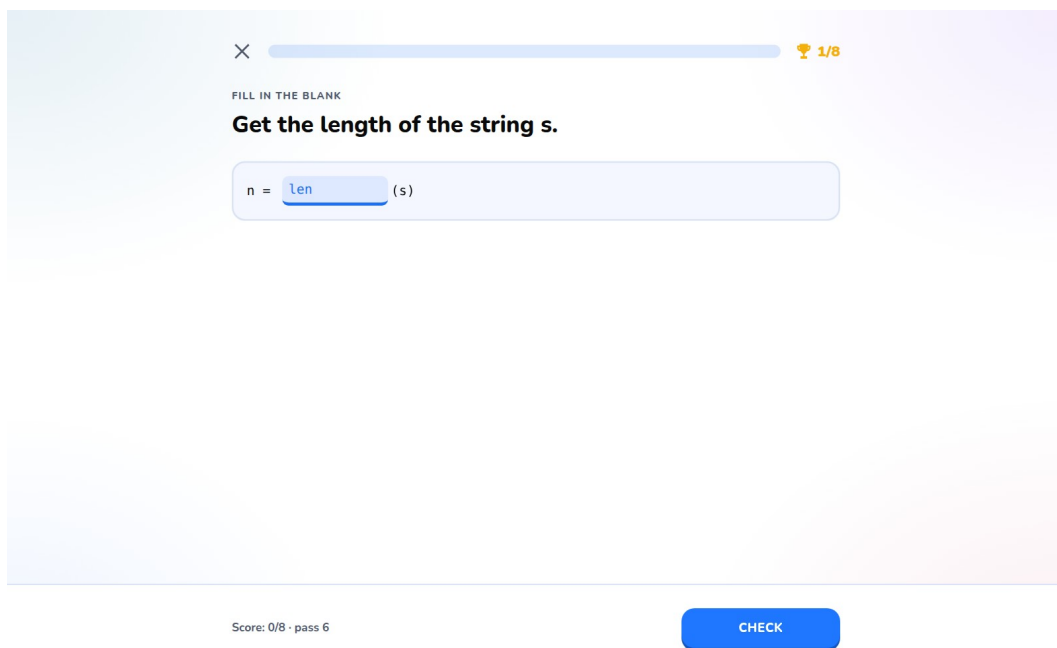


Figure 6.15: Chapter test question

During a test hints are disabled and a countdown is shown.

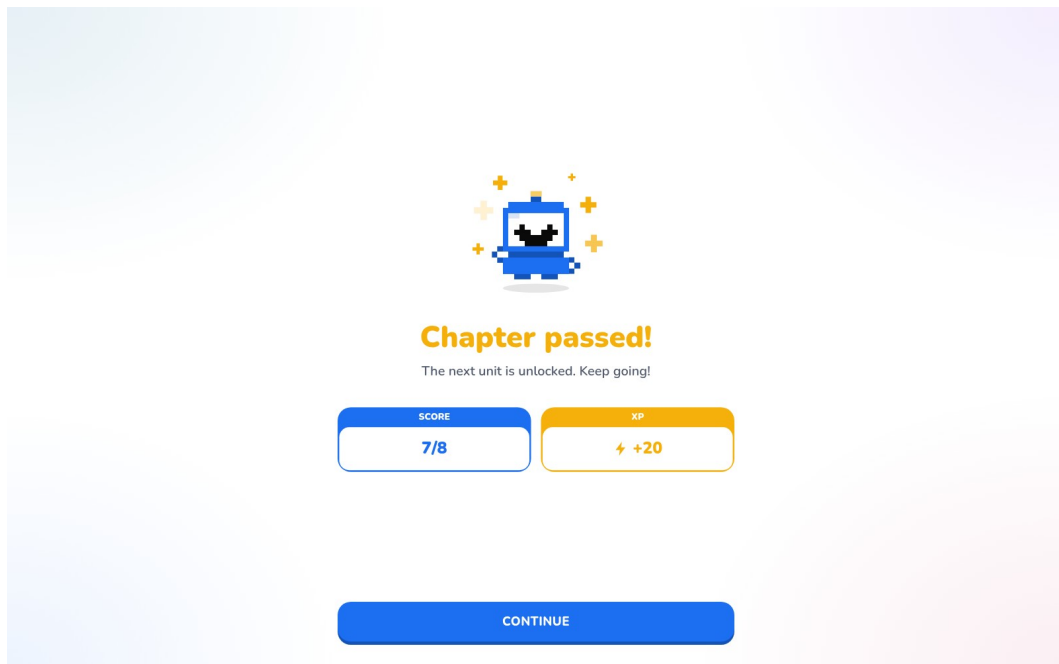


Figure 6.16: Chapter test result

The result page shows the score and XP earned.

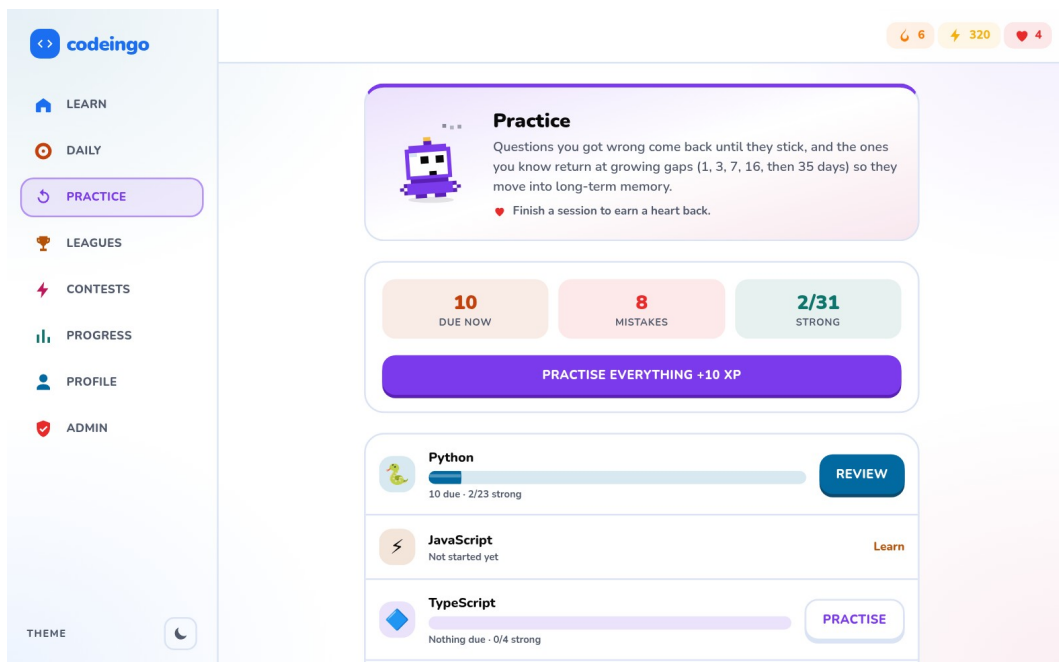


Figure 6.17: Practice hub

Missed questions are collected and scheduled for review using Leitner boxes.

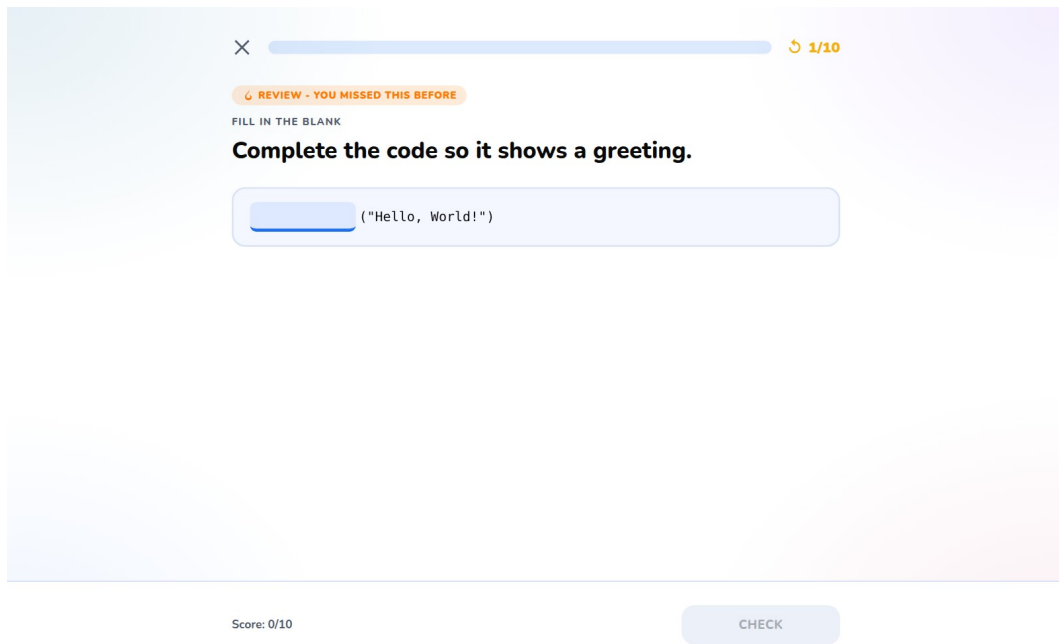


Figure 6.18: Practice question

Practice sessions give small rewards.

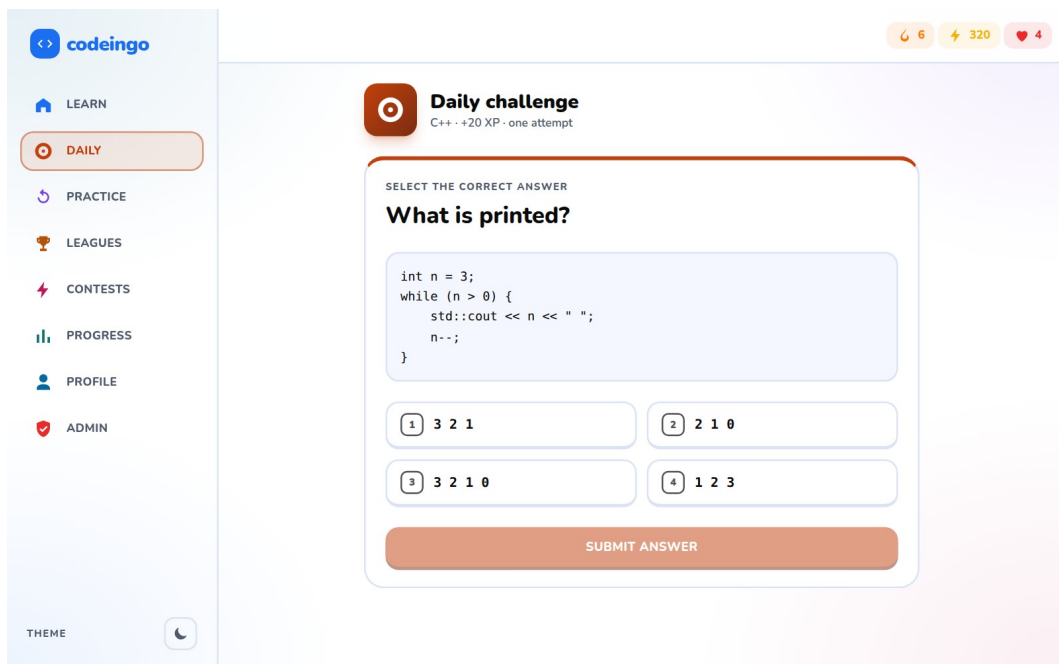


Figure 6.19: Daily challenge

A new challenge each day keeps the streak going.

## 6.1.4 Social features and contests

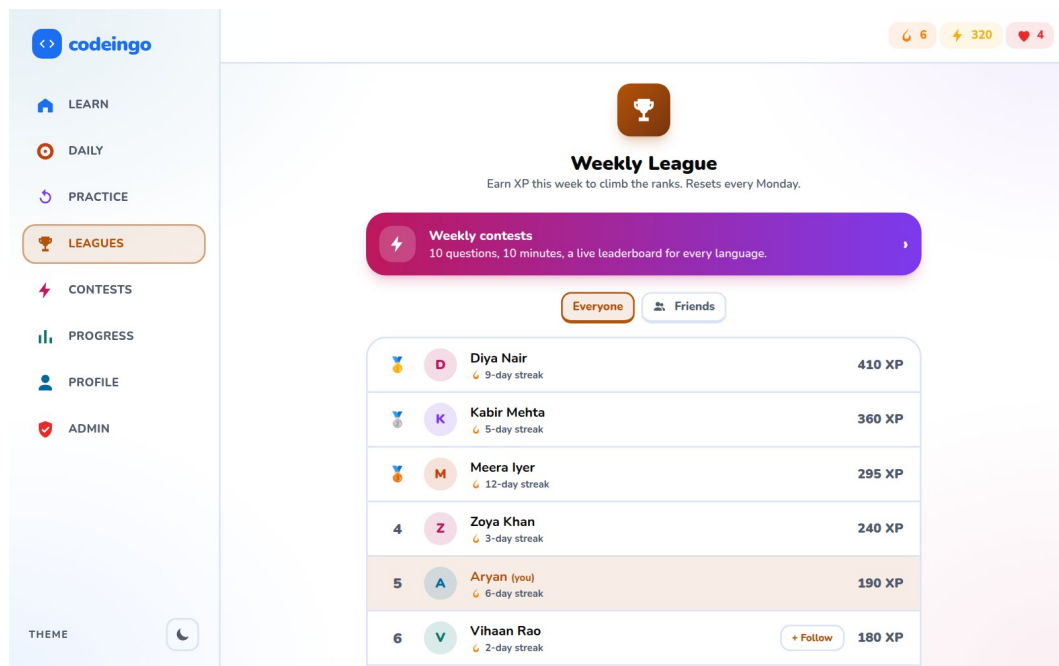


Figure 6.20: Weekly league (everyone)

Learners are ranked by XP earned this week; no e-mail addresses are shown.

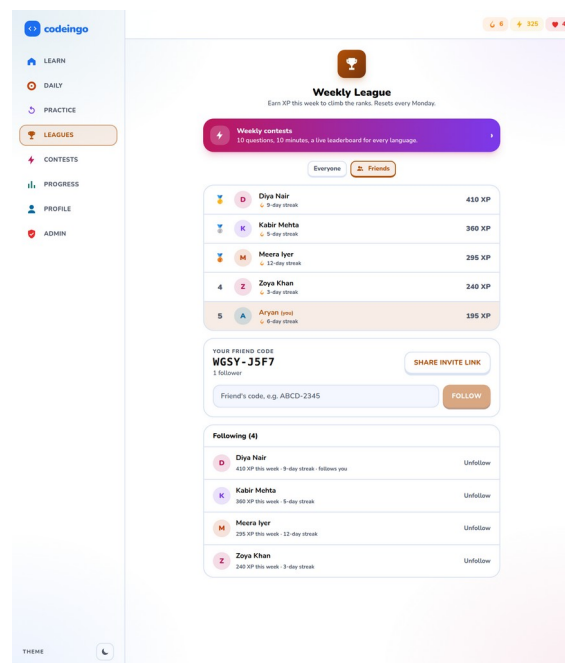


Figure 6.21: Weekly league (friends)

Adding a friend by friend code lets learners compete with friends.

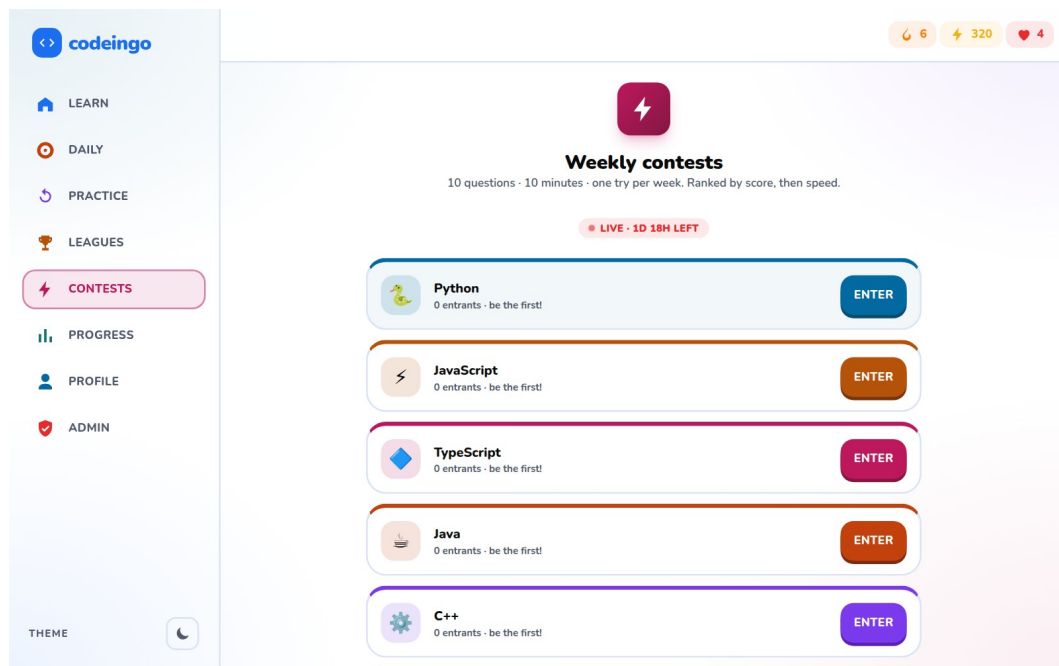


Figure 6.22: Contests list

A timed contest of ten questions opens every week.

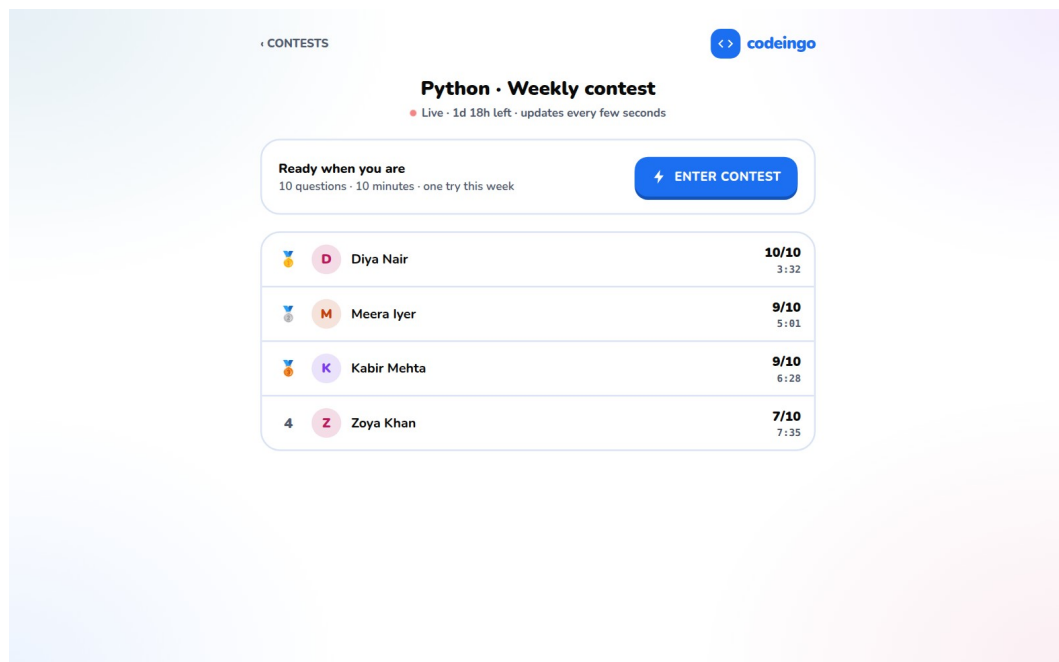


Figure 6.23: Contest leaderboard

The board ranks by score and then by time.

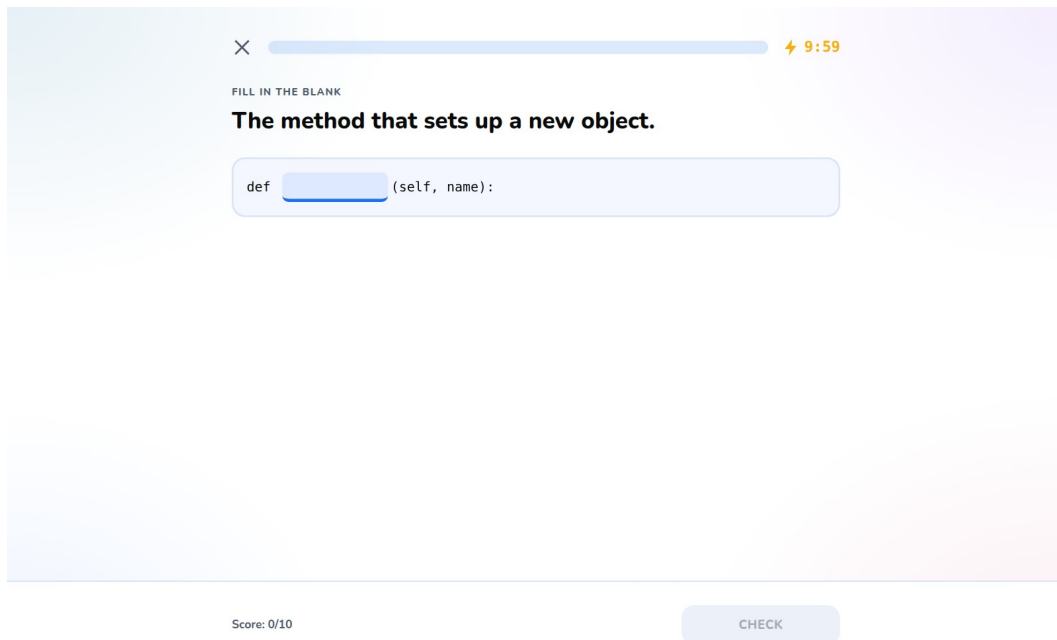


Figure 6.24: Contest in progress

Everyone gets the same questions and a time limit.

### 6.1.5 Progress, profile and certificate

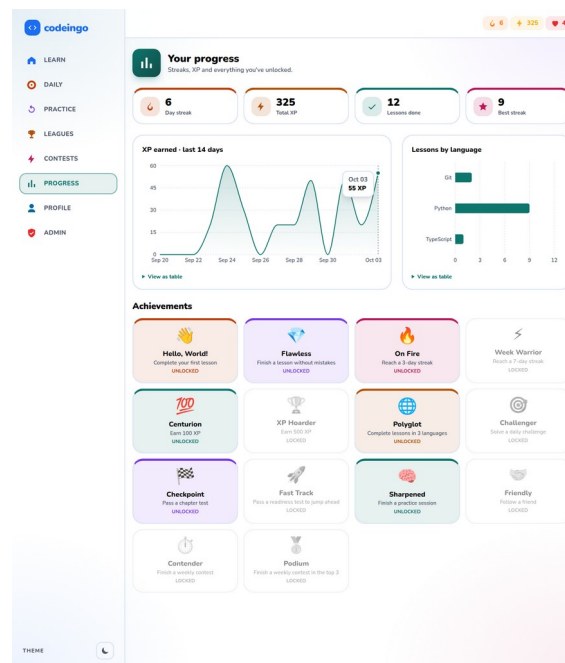


Figure 6.25: Progress (light)

XP, streak, badges and activity of the learner.

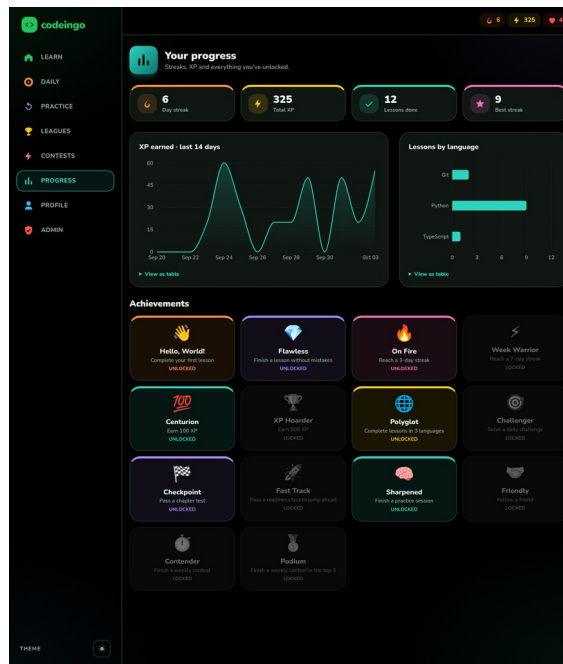


Figure 6.26: Progress (dark)

The progress page in the dark theme.

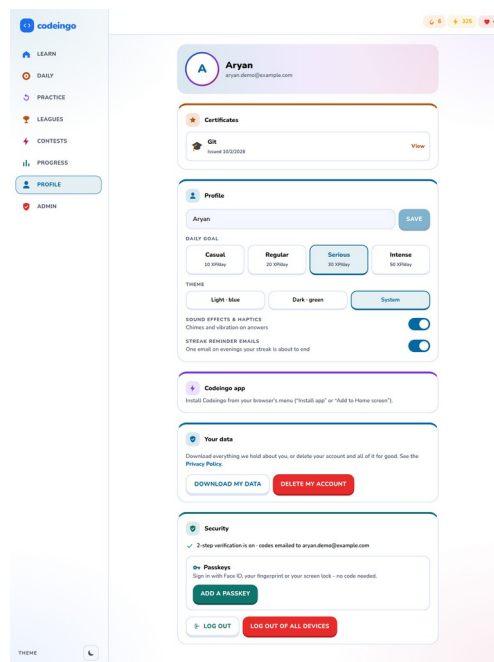


Figure 6.27: Profile and security

Passkeys, authenticator app, install-as-app, reminder e-mails and the data and privacy section.

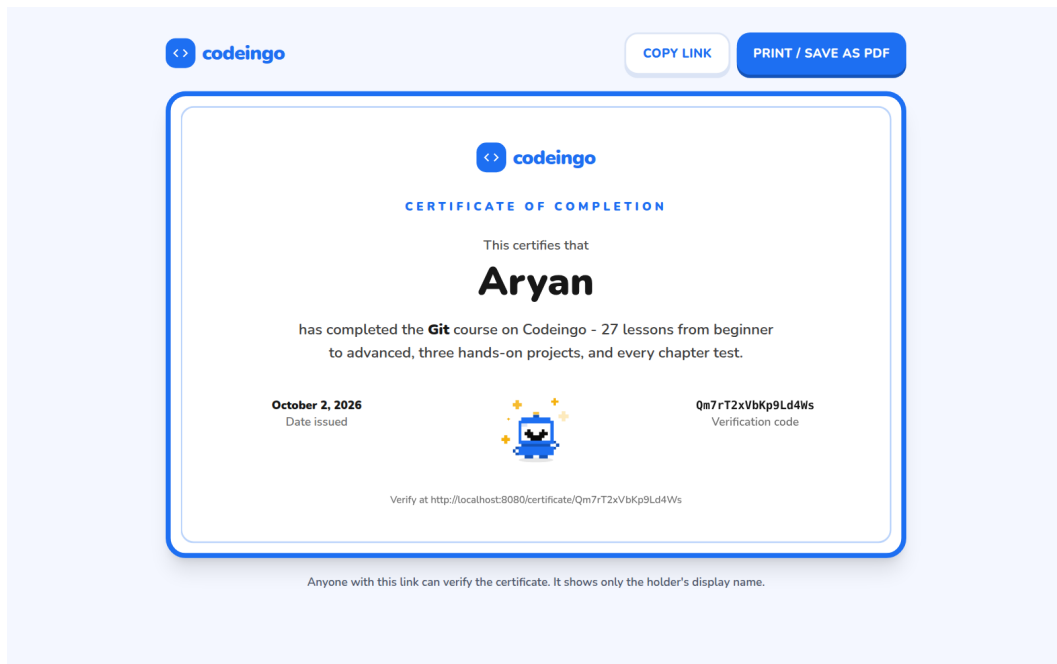


Figure 6.28: Certificate

A certificate is issued when a course is complete and can be verified publicly by its code.

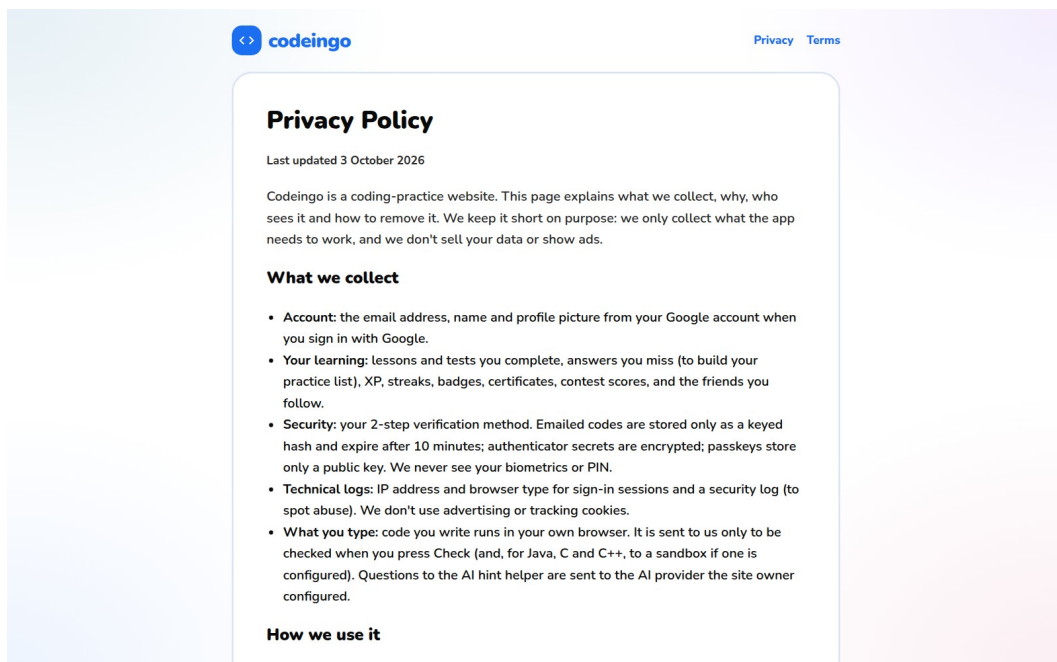


Figure 6.29: Your data (export and delete)

The learner can download all personal data or delete the account after a 2-step code.

## 6.1.6 Administration

**codeingo**

LEARN  
DAILY  
PRACTICE  
LEAGUES  
CONTESTS  
PROGRESS  
PROFILE  
**ADMIN**

THEME

**Admin**  
Content, users and the security log.

Content **Users** Security log

Search by name or email **SEARCH**

Admins can edit lessons and manage users. Owners (the emails in ADMIN\_EMAILS on the server) are always admins and can't be removed here. Every change needs your 2-step code and is written to the audit log.

| User                                         | Role    | XP  | 2FA | Last login            |            |         |
|----------------------------------------------|---------|-----|-----|-----------------------|------------|---------|
| <b>Aryan (you)</b><br>aryan.demo@example.com | owner   | 320 | ✓   | 10/3/2026, 5:15:46 AM |            |         |
| <b>Ishaan Gupta</b><br>ishaan@example.com    | learner | 120 | ✓   | never                 | Make admin | Disable |
| <b>Vihaan Rao</b><br>vihaan@example.com      | learner | 180 | ✓   | never                 | Make admin | Disable |
| <b>Zoya Khan</b><br>zoya@example.com         | learner | 240 | ✓   | never                 | Make admin | Disable |
| <b>Meera Iyer</b><br>meera@example.com       | learner | 295 | ✓   | never                 | Make admin | Disable |
| <b>Kabir Mehta</b><br>kabir@example.com      | learner | 360 | ✓   | never                 | Make admin | Disable |
| <b>Diya Nair</b><br>diya@example.com         | learner | 410 | ✓   | never                 | Make admin | Disable |

Figure 6.30: Admin: users and roles

Only an admin sees this area. Granting admin needs a confirmation code and is logged.

**codeingo**

LEARN  
DAILY  
PRACTICE  
LEAGUES  
CONTESTS  
PROGRESS  
PROFILE  
**ADMIN**

THEME

**Admin**  
Content, users and the security log.

Content Users **Security log**

Search by name or email **SEARCH**

| Time                  | Event           | User | IP        | Detail |
|-----------------------|-----------------|------|-----------|--------|
| 10/3/2026, 5:15:46 AM | mfa_enabled     | 1    | 127.0.0.1 | email  |
| 10/3/2026, 5:15:44 AM | email_code_sent | 1    | 127.0.0.1 |        |
| 10/3/2026, 5:15:44 AM | dev_login       | 1    | 127.0.0.1 |        |

Figure 6.31: Admin: audit log

Security-relevant events (sign-ins, role changes, deletions) are listed.

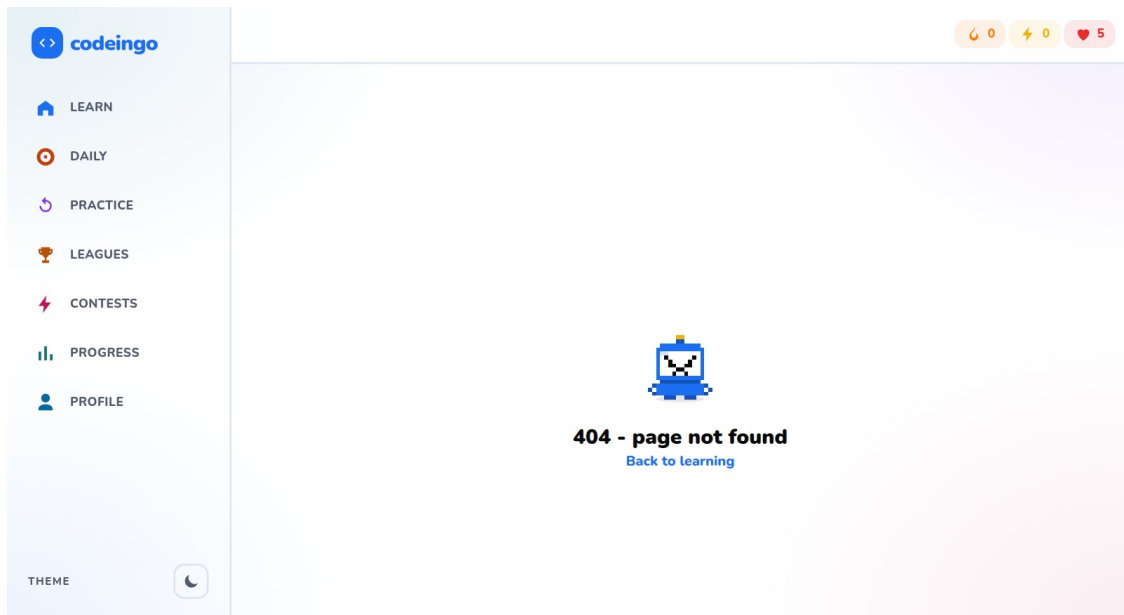


Figure 6.32: Admin area hidden from learners

A learner who opens an admin address sees an ordinary "not found" page.

users (SELECT id, email, name, role, mfa\_method, xp\_total, streak\_current FROM users)

| id | email            | name         | role    | mfa_method | xp_total | streak_current | friend_code |
|----|------------------|--------------|---------|------------|----------|----------------|-------------|
| 2  | d***@example.com | Diya Nair    | learner | email      | 410      | 9              | RkXVB6H4    |
| 3  | k***@example.com | Kabir Mehta  | learner | email      | 360      | 5              | JVYKSENS    |
| 1  | a***@example.com | Aryan        | admin   | email      | 325      | 6              | WGSYJ5F7    |
| 4  | m***@example.com | Meera Iyer   | learner | email      | 295      | 12             | UYWBZ7WU    |
| 5  | z***@example.com | Zoya Khan    | learner | email      | 240      | 3              | DTDNBKQS    |
| 6  | v***@example.com | Vihaan Rao   | learner | email      | 180      | 2              | 2CZSCVKK    |
| 7  | i***@example.com | Ishaan Gupta | learner | email      | 120      | 1              | YUXVTAM4    |
| 8  | d***@example.com | Diya         | learner | email      | 0        | 0              | FNIXAAGH    |

courses and their size (JOIN units / lessons)

| slug       | title                        | units | lessons | projects |
|------------|------------------------------|-------|---------|----------|
| python     | Python                       | 16    | 51      | 3        |
| javascript | JavaScript                   | 16    | 51      | 3        |
| typescript | TypeScript                   | 8     | 27      | 3        |
| java       | Java                         | 16    | 51      | 3        |
| cpp        | C++                          | 16    | 51      | 3        |
| c          | C                            | 16    | 51      | 3        |
| sql        | SQL                          | 16    | 51      | 3        |
| html-css   | HTML & CSS                   | 16    | 51      | 3        |
| git        | Git                          | 8     | 27      | 3        |
| dsa        | Data Structures & Algorithms | 8     | 27      | 3        |

audit\_log (latest security events)

| id | user_id | event           | created_at                 |
|----|---------|-----------------|----------------------------|
| 12 | 1       | login           | 2026-10-03 05:18:52.056013 |
| 11 | 1       | email_code_sent | 2026-10-03 05:18:51.275523 |
| 10 | 1       | dev_login       | 2026-10-03 05:18:51.209941 |
| 9  | 1       | login           | 2026-10-03 05:18:28.305770 |
| 8  | 1       | email_code_sent | 2026-10-03 05:18:27.543738 |
| 7  | 1       | dev_login       | 2026-10-03 05:18:27.474482 |

certificates

| code               | holder_name | course_title | lessons | issued_at                  |
|--------------------|-------------|--------------|---------|----------------------------|
| Qm7rT2xVbKp9Ld4Wls | Aryan       | Git          | 27      | 2026-10-02 05:15:48.525216 |

Figure 6.33: Database tables

A view of the PostgreSQL tables created by the application.

### 6.1.7 Mobile view

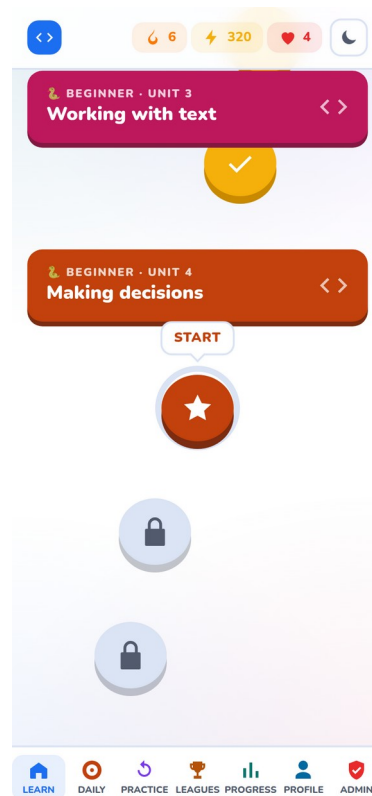


Figure 6.34: Phone: learning path

The layout adapts to a phone screen (390 px).

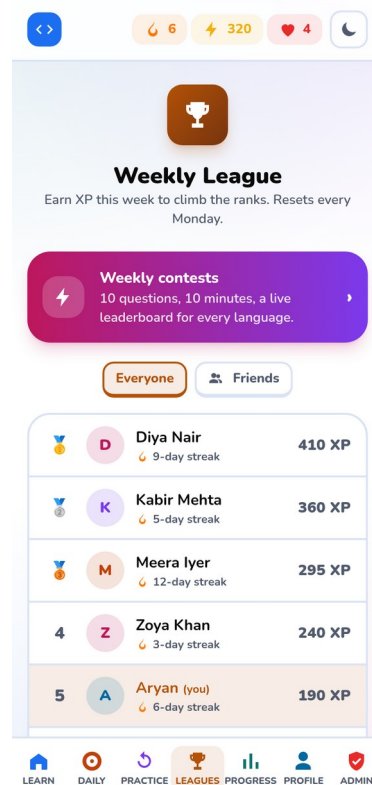


Figure 6.35: Phone: leagues

Leagues on a phone.

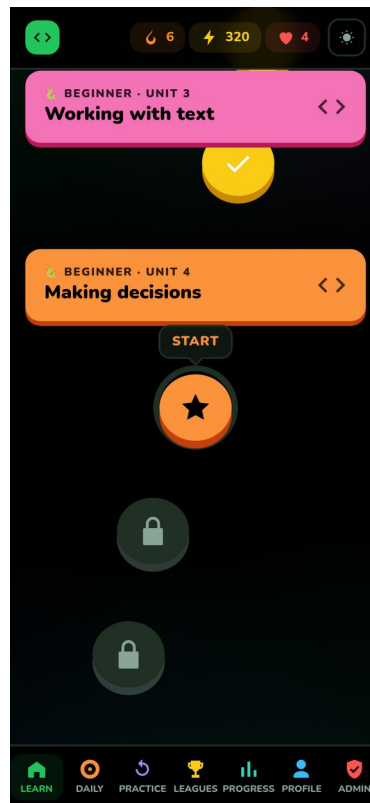


Figure 6.36: Phone: dark theme

The dark theme on a phone.

## 6.2 Discussion of results

The objectives set in Chapter 1 were met. The platform provides ten courses with real code execution in the browser, game elements that the survey found appealing, projects at the end of every section, and social features. The security design (mandatory 2-step verification, hidden admin area, rate limiting, signed tokens) was checked by automated tests.

**Table 6.2.1: Survey finding and how the finished system answers it**

| Survey finding                                     | Result in Codeingo                                                                |
|----------------------------------------------------|-----------------------------------------------------------------------------------|
| Confusing syntax is the main difficulty (13 of 40) | Short lessons, hints, instant feedback with explanations and in-browser practice. |
| Lack of guidance and no real-world use (8 + 8)     | A fixed learning path, readiness tests and three-step projects in every section.  |
| Real projects are the best motivator (17 of 43)    | 30 projects, each building on the learner's own code.                             |
| Friends and community (32 of 43 want some)         | Friend codes, leagues and weekly contests.                                        |
| Videos preferred by many (13)                      | Not provided; listed as future scope.                                             |

## GUIDE INTERACTION REPORT

*(To be maintained by the student and submitted as part of the Field Project documentation)*

|                                       |                        |
|---------------------------------------|------------------------|
| <b>Name of the Student</b>            | [STUDENT NAME]         |
| <b>Program /Semester</b>              | B.Sc I.T./ Semester V  |
| <b>Roll No/Seat No</b>                | [ROLL NO / SEAT NO]    |
| <b>Major Subject / Specialization</b> | Information Technology |
| <b>Field project Title</b>            | CODEINGO               |
| <b>Name of the Faculty mentor</b>     | [GUIDE NAME]           |

### Overview (Max 150 Words)

*(Brief summary of key activities, tasks and projects undertaken during the period)*

### CHAPTER 6: RESULTS AND DISCUSSION

In this period the finished system was run end to end and screenshots of all main screens were prepared in both themes and on a phone. The results were compared with the survey findings and the objectives of Chapter 1.

#### Learning Outcomes (Max 100 words):

We learned to present results clearly with captions and short discussions, to check the work against the original requirements, and to see the system through the eyes of a first-time user.

#### Challenges Faced (Max 100 words):

Taking clean screenshots needed demonstration data and a repeatable script. Some screens looked different in the dark theme and needed adjustment.

#### Mentor Feedback and Suggestions (Max 100 words):

[To be filled in by the guide]

Signature of the Internal Guide

# CHAPTER 7

## CONCLUSION AND FUTURE WORK

### 7.1 Conclusion

Codeingo shows that a game-like, project-oriented coding course can be delivered entirely as a web application that is secure and does not need the learner to install anything. The learner's code runs in the browser, so the server stays small and safe; grading on the server prevents answer copying; and a layered security design protects accounts and data.

Starting from a survey of 43 learners, the project delivered ten courses with 438 lessons, 2,130 exercises and 30 projects, seven kinds of gamification, friends and contests, certificates, a progressive web app, and an administration area. It is verified by 95 automated tests (on SQLite and PostgreSQL) and by running all 492 reference solutions.

### 7.2 Limitations

- Java, C and C++ programs cannot run in the browser; they are checked on the server and need a sandboxed runtime to be installed for the full experience.
- The AI hint feature depends on an external AI service and an API key.
- Contests are not live in the sense of instant push updates; the board refreshes at short intervals.
- Learning content was written by the project team only and has not yet been reviewed by external teachers.
- The survey had 43 respondents from one region and age group, so its results are indicative rather than statistical.
- The platform has been tested with demonstration users and not yet with a large public audience.

### 7.3 Future Scope

- Video explanations and audio for lessons, as many survey respondents prefer videos.
- Live contests with push updates, team contests and a public contest archive.
- Native mobile apps and offline lesson downloads.
- More courses (Rust, Go, Kotlin, data science) and longer advanced tracks.
- Teacher dashboards and classrooms for colleges and schools.
- Content review by educators and adaptive difficulty based on learner performance.
- Accessibility audit and translations into Indian languages.

### 7.4 Bibliography

1. FastAPI documentation, <https://fastapi.tiangolo.com>
2. Next.js documentation, <https://nextjs.org/docs>
3. SQLAlchemy 2.0 documentation, <https://docs.sqlalchemy.org>

4. PostgreSQL 16 documentation, <https://www.postgresql.org/docs/16/>
5. W3C Web Authentication (WebAuthn) Level 2 specification, <https://www.w3.org/TR/webauthn-2/>
6. OWASP Cheat Sheet Series (Authentication, CSRF Prevention, Content Security Policy), <https://cheatsheetseries.owasp.org>
7. Pyodide documentation, <https://pyodide.org>
8. TypeScript compiler API, <https://www.typescriptlang.org/docs>
9. S. Leitner, So lernt man lernen, Herder, 1972 (Leitner system for spaced repetition).
10. Playwright documentation, <https://playwright.dev>
11. Mozilla Developer Network, Progressive Web Apps, [https://developer.mozilla.org/docs/Web/Progressive\\_web\\_apps](https://developer.mozilla.org/docs/Web/Progressive_web_apps)
12. Pressman R. S., Software Engineering: A Practitioner's Approach, McGraw-Hill.

## GUIDE INTERACTION REPORT

*(To be maintained by the student and submitted as part of the Field Project documentation)*

|                                       |                        |
|---------------------------------------|------------------------|
| <b>Name of the Student</b>            | [STUDENT NAME]         |
| <b>Program /Semester</b>              | B.Sc I.T./ Semester V  |
| <b>Roll No/Seat No</b>                | [ROLL NO / SEAT NO]    |
| <b>Major Subject / Specialization</b> | Information Technology |
| <b>Field project Title</b>            | CODEINGO               |
| <b>Name of the Faculty mentor</b>     | [GUIDE NAME]           |

### Overview (Max 150 Words)

*(Brief summary of key activities, tasks and projects undertaken during the period)*

### CHAPTER 7: CONCLUSION AND FUTURE WORK

The project was summarised, its limits were listed honestly and ideas for future work were written. The bibliography and annexures were prepared and the report was finalised.

### Learning Outcomes (Max 100 words):

We learned to judge our own work fairly, to separate what is finished from what is planned, and to cite sources.

### Challenges Faced (Max 100 words):

Deciding which limitations to state, and keeping the report consistent with the final code, took several revisions.

### Mentor Feedback and Suggestions (Max 100 words):

[To be filled in by the guide]

Signature of the Internal Guide

# ANNEXURES

## A Survey questionnaire

A Google Form titled "Coding Platform Interest Survey" was answered by 43 people. The questions were:

1. Age group (under 15, 15-18, 19-22, 23+).
2. Have you tried to learn coding before? (currently learning / tried and gave up / never tried).
3. What was the hardest part of learning to code?
4. Which learning style do you prefer? (videos, hands-on, games, interactive, reading).
5. How appealing is a game-like coding app? (1 to 5).
6. Which programming language would you like to learn first?
7. What would motivate you most? (real projects, friends, points, certificates, curiosity).
8. Would you prefer to learn alone, with a community, or a mix?
9. If you quit a course before, why?
10. Which learning platforms have you used before?

## B REST API endpoints

The back end exposes 71 endpoints. Interactive documentation is generated automatically by FastAPI in development.

**Table B.1: REST API endpoints**

| Method | Path                              | Group   |
|--------|-----------------------------------|---------|
| POST   | /api/account/confirm-code         | account |
| POST   | /api/account/delete               | account |
| GET    | /api/account/export               | account |
| GET    | /api/admin/audit                  | admin   |
| POST   | /api/admin/confirm-code           | admin   |
| POST   | /api/admin/courses                | admin   |
| POST   | /api/admin/lessons                | admin   |
| DELETE | /api/admin/lessons/{lesson_id}    | admin   |
| GET    | /api/admin/lessons/{lesson_id}    | admin   |
| PUT    | /api/admin/lessons/{lesson_id}    | admin   |
| GET    | /api/admin/tree                   | admin   |
| POST   | /api/admin/units                  | admin   |
| GET    | /api/admin/users                  | admin   |
| POST   | /api/admin/users/{user_id}/active | admin   |

| Method | Path                                   | Group    |
|--------|----------------------------------------|----------|
| POST   | /api/admin/users/{user_id}/role        | admin    |
| POST   | /api/ai/hint                           | ai       |
| GET    | /api/ai/status                         | ai       |
| POST   | /api/auth/2fa/email/enable             | auth     |
| POST   | /api/auth/2fa/email/send               | auth     |
| POST   | /api/auth/2fa/enable                   | auth     |
| POST   | /api/auth/2fa/setup                    | auth     |
| GET    | /api/auth/2fa/status                   | auth     |
| POST   | /api/auth/2fa/verify                   | auth     |
| GET    | /api/auth/config                       | auth     |
| GET    | /api/auth/csrf                         | auth     |
| POST   | /api/auth/dev-login                    | auth     |
| GET    | /api/auth/google/callback              | auth     |
| GET    | /api/auth/google/login                 | auth     |
| POST   | /api/auth/logout                       | auth     |
| POST   | /api/auth/logout-all                   | auth     |
| GET    | /api/auth/me                           | auth     |
| POST   | /api/auth/recovery-codes               | auth     |
| POST   | /api/auth/refresh                      | auth     |
| GET    | /api/contests                          | contests |
| POST   | /api/contests/{contest_id}/answer      | contests |
| POST   | /api/contests/{contest_id}/enter       | contests |
| POST   | /api/contests/{contest_id}/finish      | contests |
| GET    | /api/contests/{contest_id}/leaderboard | contests |
| POST   | /api/attempts/{attempt_id}/answer      | learn    |
| POST   | /api/attempts/{attempt_id}/complete    | learn    |
| GET    | /api/certificates                      | learn    |
| GET    | /api/courses                           | learn    |
| GET    | /api/courses/{slug}                    | learn    |
| GET    | /api/daily                             | learn    |
| POST   | /api/daily/answer                      | learn    |
| POST   | /api/lessons/{lesson_id}/start         | learn    |
| GET    | /api/public/certificates/{code}        | learn    |
| POST   | /api/run                               | learn    |
| GET    | /api/health                            | misc     |

| Method | Path                                | Group    |
|--------|-------------------------------------|----------|
| GET    | /api/auth/passkeys                  | passkeys |
| POST   | /api/auth/passkeys/login            | passkeys |
| POST   | /api/auth/passkeys/login/options    | passkeys |
| POST   | /api/auth/passkeys/register         | passkeys |
| POST   | /api/auth/passkeys/register/options | passkeys |
| DELETE | /api/auth/passkeys/{passkey_id}     | passkeys |
| GET    | /api/practice/overview              | practice |
| POST   | /api/practice/start                 | practice |
| GET    | /api/practice/summary               | practice |
| POST   | /api/practice/{attempt_id}/answer   | practice |
| POST   | /api/practice/{attempt_id}/complete | practice |
| GET    | /api/friends                        | social   |
| POST   | /api/friends                        | social   |
| GET    | /api/friends/lookup                 | social   |
| DELETE | /api/friends/{code}                 | social   |
| POST   | /api/public/unsubscribe             | social   |
| GET    | /api/leaderboard                    | stats    |
| PATCH  | /api/profile                        | stats    |
| GET    | /api/stats                          | stats    |
| POST   | /api/tests/start                    | tests    |
| POST   | /api/tests/{attempt_id}/answer      | tests    |
| POST   | /api/tests/{attempt_id}/complete    | tests    |

## C Security controls

| Control                 | How it is done                                                                                               |
|-------------------------|--------------------------------------------------------------------------------------------------------------|
| Sign-in                 | Google account + mandatory 2-step verification (e-mail code, authenticator app or passkey).                  |
| Sessions                | Short-lived signed access token and rotating refresh token in HttpOnly, Secure, SameSite cookies.            |
| CSRF                    | Double-submit token header on every state-changing request.                                                  |
| Content-Security-Policy | Per-request nonce; no inline script without nonce.                                                           |
| Rate limiting           | Per-IP and per-account limits, backed by Redis in production.                                                |
| Secrets at rest         | Authenticator secrets encrypted with Fernet; e-mail codes stored hashed.                                     |
| Admin area              | Hidden (404) for non-admins; role changes need a 2-step code; owners protected; all actions in an audit log. |

| Control           | How it is done                                                                  |
|-------------------|---------------------------------------------------------------------------------|
| Privacy           | Data export, account deletion with cascading removal, signed unsubscribe links. |
| Answer protection | Expected answers never leave the server; very fast completions are refused.     |

## D Installation and user guide

### Running locally

1. Install Python 3.11+, Node.js 20+ and (optionally) Docker.
2. Back end: create a virtual environment, run `pip install -r requirements.txt`, copy `.env.example` to `.env` and start `uvicorn app.main:app`.
3. Front end: run `npm install` and `npm run dev` inside the frontend folder.
4. Open `http://localhost:3000`, sign in and verify with the code (in development it is written to the server console).

### Deployment

The repository contains `render.yaml` (API and Redis), a Vercel guide (front end) and `docker-compose.yml`. Production needs a PostgreSQL database, SMTP credentials, Google OAuth keys and an encryption key; the application refuses to start if a required secret is missing.

### Using the platform

1. Sign in with Google and complete 2-step verification.
2. Pick a course and follow the path; finish lessons to earn XP and keep your streak.
3. Pass chapter tests to unlock new units; take the readiness test to jump ahead.
4. Use Practice for mistakes, the Daily challenge and the weekly Contest.
5. Add friends with your friend code and watch the leagues.
6. Complete a course to receive a certificate with a public verification link.